



Universidad de Valladolid

Escuela de Ingeniería Informática

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Tecnologías de la Información

**Sistema para la gestión y despliegue de
topologías de redes basadas en software
sobre Containerlab**

Autor:

Francisco Iván San Segundo Álvarez

Tutor:

Juan Alberto Muñoz Cristóbal

Dedicatoria

A mi madre y a mi abuela, que ejercieron con un amor incondicional el papel de madre.

A mis tíos, por haber sido esa figura paterna que me impulsó a seguir adelante.

Gracias a ellos, hoy alcanzo esta meta; cuando todo era oscuro, fueron mi luz, y cuando el camino estaba despejado, fueron los que me enseñaron a recorrerlo.

Y a mi abuelo, mi ángel de la guarda, que desde arriba me protege y me guía en cada paso que doy.

Agradecimientos

En primer lugar y más importante, quiero agradecer a mi tutor, Juan, por su valiosa guía y su apoyo crítico a lo largo del desarrollo de este trabajo. Su experiencia y conocimientos han sido fundamentales para orientar este proyecto de la mejor manera.

En segundo lugar, quiero agradecer a mis compañeros de carrera y amigos, por hacer mi camino universitario mucho más llevadero, por las jornadas de estudio compartido y por la paciencia y apoyo prestados tanto durante la realización de las prácticas como en los momentos de mayor tensión.

Por último, a mi familia, cuya comprensión y apoyo incondicional han sido clave para culminar con éxito esta etapa de mi vida.

Resumen

En el panorama tecnológico actual, la creciente complejidad de las infraestructuras de red demanda soluciones avanzadas que superen las limitaciones de los métodos tradicionales. Históricamente, la configuración de topologías de red ha dependido de procesos manuales altamente propensos a errores humanos, con una limitada capacidad de portabilidad entre entornos y una monitorización de recursos insuficiente. Esta carencia de automatización supone un obstáculo crítico para la agilidad operativa y la escalabilidad en entornos de emulación modernos.

Ante esta problemática, este proyecto presenta el diseño e implementación de una plataforma *full-stack* innovadora, orientada a la orquestación automatizada de redes sobre el motor de virtualización Containerlab. La arquitectura de la solución integra un *backend* robusto desarrollado en FastAPI, encargado de centralizar la lógica de orquestación, y una interfaz web construida en React, que permite a los usuarios gestionar el despliegue y la monitorización en tiempo real. Un aspecto diferenciador de este sistema es la incorporación de un modelo de persistencia híbrido; este garantiza la integridad de los datos mediante una sincronización entre una base de datos centralizada y archivos de configuración YAML, asegurando así la portabilidad total del escenario entre diferentes infraestructuras.

El desarrollo se ha articulado bajo una metodología de trabajo iterativa e incremental. Este enfoque ha permitido la evolución del sistema desde un prototipo inicial basado en línea de comandos hasta construir un ecosistema web completo y escalable. Los resultados obtenidos validan la eficacia del sistema para gestionar escenarios complejos.

En conclusión, este trabajo provee una herramienta versátil y segura que optimiza el ciclo de vida de las redes virtuales, mejorando la experiencia del usuario y estableciendo una base sólida para futuras implementaciones en el ámbito de la automatización de las redes.

Abstract

In today's technological landscape, the growing complexity of network infrastructures demands advanced solutions that overcome the limitations of traditional methods. Historically, network topology configuration has relied on manual processes highly prone to human error, with limited portability across environments and insufficient resource monitoring. This lack of automation represents a critical obstacle to operational agility and scalability in modern emulation environments.

To address this challenge, this project presents the design and implementation of an innovative *full-stack* platform aimed at the automated orchestration of networks on the Containerlab virtualization engine. The solution's architecture integrates a robust *backend* developed in FastAPI, responsible for centralizing the orchestration logic, and a web interface built with React, which allows users to manage deployment and real-time monitoring. A distinguishing feature of this system is the incorporation of a hybrid persistence model; this ensures data integrity through synchronization between a centralized database and YAML configuration files, thereby guaranteeing full scenario portability across different infrastructures.

Development was carried out under an iterative and incremental work methodology. This approach enabled the system's evolution from an initial command-line prototype into a complete, scalable web ecosystem. The results obtained validate the system's effectiveness in managing complex scenarios.

In conclusion, this work provides a versatile and secure tool that optimizes the lifecycle of virtual networks, enhancing user experience and establishing a solid foundation for future developments in the field of network automation.

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Estructura de la memoria	4
2. Estado de la cuestión	7
2.1. Arquitectura y componentes de los sistemas SDN	7
2.2. Taxonomía y clasificación de las redes SDN	10
2.2.1. Según la arquitectura del controlador:	11
2.2.2. Según el modelo tecnológico subyacente:	11
2.2.3. Según la gestión de los flujos:	12
2.2.4. Según la topología de red:	13
2.3. Ejemplos de uso de las redes SDN	14
2.4. Tecnologías para la emulación de redes virtualizadas	15
2.4.1. Análisis de entornos de emulación de red	15
2.4.2. Herramientas y lenguajes para la automatización del despliegue y gestión de re- des virtualizadas	16
2.5. Plataformas y herramientas para el ciclo de vida de diseño y gestión de redes virtualizadas	17
3. Metodología y Planificación	21
3.1. Metodología	21
3.1.1. Fundamentos del diseño iterativo e incremental	21
3.1.2. Organización de tareas con Kanban	22
3.1.3. Integración de Kanban en Notion	22
3.1.4. Priorización mediante la técnica MoSCoW	23
3.1.5. Justificación y aplicación del enfoque	24
3.2. Planificación inicial	24
3.2.1. Fase inicial	24
Iteraciones del desarrollo del proyecto	25
Primera iteración (23/10/2025 - 20/11/2025)	27
Segunda iteración (11/02/2026 - 02/03/2026)	27
Tercera iteración (02/03/2026 - 28/03/2026)	28
Cuarta iteración (28/03/2026 - 08/05/2026)	28

Quinta iteración (08/05/2026 - 18/05/2026)	29
Sexta iteración (19/05/2026 - 30/05/2026)	29
3.2.2. Análisis de riesgos	30
3.2.3. Metodología de análisis y priorización	31
3.2.4. Estimación de costes	41
3.2.5. Presupuesto y coste real del proyecto	43
3.3. Comparación respecto a la planificación inicial	44
3.4. Lecciones aprendidas de la planificación	45
3.5. Tecnologías utilizadas	46
3.5.1. Herramientas de comunicación	46
3.5.2. Herramientas para el análisis, diseño y documentación	47
3.5.3. Herramientas para la gestión y el control del proyecto	47
3.5.4. Herramientas para el desarrollo del proyecto	48
3.5.5. Declaración sobre el uso de la Inteligencia Artificial	53
4. Análisis	55
4.1. Requisitos del sistema	55
4.1.1. Requisitos funcionales	55
4.1.2. Requisitos no funcionales	58
4.1.3. Requisitos de información	59
4.2. Criterios de aceptación	60
4.3. Casos de uso	61
4.4. Modelo de dominio	69
4.5. Diagramas de secuencia	71
5. Diseño del sistema	75
5.1. Arquitectura del Sistema	75
5.1.1. Diagrama de paquetes	75
5.1.2. Patrón arquitectónico MVVM	77
5.2. Estructura de la base de datos	78
5.3. Integridad y Restricciones de Sistema	79
5.4. Patrones de diseño	80
5.4.1. Patrón de Inyección de Dependencias	80
5.4.2. Patrón DTO	81
5.4.3. Patrón Observer	81
5.5. Decisiones de diseño	82
5.5.1. Entorno de desarrollo: React y Vite	82
5.5.2. Tailwind CSS	82
5.5.3. Identidad Visual: Nomenclatura, Logo, Tipografía y Paleta de Colores	83
5.5.4. Control de acceso y gestión de identidades: Google Auth	84
5.5.5. Mecanismo de seguridad en la eliminación: Confirmación por entrada de texto	85
5.6. Especificación de la interfaz de usuario	86
5.6.1. Criterios de diseño y usabilidad	86

5.6.2. Prototipado y definición de la interfaz	86
5.6.3. Diseño de vistas y catálogo de pantallas	86
Bloque de acceso y gestión de cuenta	87
Bloque de administración de topologías de red	88
Bloque de monitorización operativa e interacción con nodos	91
5.7. Diagrama de despliegue	93
6. Descripción de las iteraciones del sistema	95
6.1. Iteración 1: Núcleo del orquestador en CLI y Automatización de red	95
6.1.1. Objetivo	95
6.1.2. Desarrollo y especificaciones de la implementación	95
6.1.3. Contratiempos encontrados y soluciones adoptadas	96
6.2. Iteración 2: Diseño e integración de la persistencia en base de datos	98
6.2.1. Objetivo	98
6.2.2. Desarrollo y especificaciones de la implementación	98
6.2.3. Contratiempos encontrados y soluciones adoptadas	98
6.3. Iteración 3: Diseño de las topologías de red y creación de las plantillas corporativas	99
6.3.1. Objetivo	99
6.3.2. Desarrollo y especificaciones de la implementación	99
6.3.3. Contratiempos encontrados y soluciones adoptadas	100
6.4. Iteración 4: Integración del sistema full-stack y validación del producto final	101
6.4.1. Objetivo	101
6.4.2. Desarrollo y especificaciones de la implementación	101
6.4.3. Contratiempos encontrados y soluciones adoptadas	102
6.5. Estructura del proyecto	105
7. Pruebas	109
7.1. Pruebas de aceptación	109
7.1.1. Gestión de identidad	110
7.1.2. Orquestación y despliegue	112
7.1.3. Monitorización y telemetría	117
7.1.4. Gestión de escenarios	118
7.1.5. Validación de requisitos no funcionales	120
7.2. Pruebas de usabilidad	121
7.2.1. Perfil de los participantes	122
7.2.2. Resultados cualitativos y observaciones	122
Evaluación cuantitativa: System Usability Scale (SUS)	123
8. Conclusiones	125
8.1. Conclusiones	125
8.1.1. Objetivos del proyecto	125
8.1.2. Objetivos personales y de aprendizaje	127
8.2. Líneas de trabajo futuras	128
8.2.1. Ampliación de funcionalidades y mejoras en la experiencia del usuario	128

8.2.2. Mejoras técnicas y de calidad del software	130
8.3. Valoración personal	130
Referencias	131
Anexos	143
A. Manual de despliegue	143
A.1. Requisitos previos	143
A.2. Instalación de herramientas y configuración	144
A.2.1. Preparación del entorno (HOST)	144
A.2.2. Configuración de las variables de entorno	144
A.2.3. Configuración de la base de datos	145
A.2.4. Instalación del Frontend	145
A.3. Despliegue del proyecto	145
A.4. Verificación del despliegue	146
B. Manual de uso	147
B.1. Inicio de sesión	147
B.2. Pantalla principal: Mis topologías	148
B.3. Creación de una nueva topología	150
B.4. Eliminación de una topología	151
B.5. Biblioteca de plantillas	151
B.6. Ajustes de Perfil	152
B.7. Monitor de Red	153
B.8. Detalles de un nodo	154
B.9. Consola del nodo	156
C. Repositorio del código fuente	159

Índice de figuras

2.1. Arquitectura de referencia de los sistemas SDN. Fuente: adaptado de [17].	10
2.2. Representación del enrutamiento Underlay frente a Overlay. Fuente: basado en [21]. . . .	13
3.1. Tablero Kanban empleado durante el desarrollo del proyecto.	23
3.2. Diagrama de Gantt con la planificación global del proyecto	26
3.3. Diagrama de Gantt de la primera iteración	27
3.4. Diagrama de Gantt de la segunda iteración	28
3.5. Diagrama de Gantt de la tercera iteración	28
3.6. Diagrama de Gantt de la cuarta iteración	29
3.7. Diagrama de Gantt de la quinta iteración	29
3.8. Diagrama de Gantt de la sexta iteración	30
4.1. Diagrama de CCUU del sistema	62
4.2. Modelo de dominio del sistema	70
4.3. Diagrama de secuencia del CU02: Registrar Usuario	72
4.4. Diagrama de secuencia del CU09: Desplegar Topología	72
4.5. Diagrama de secuencia del CU10: Monitorizar Nodos	73
4.6. Diagrama de secuencia del CU13: Consultar logs de actividad	73
5.1. Diagrama de paquetes del sistema	77
5.2. Estructura de la base de datos	79
5.3. Logo de la interfaz de usuario	84
5.4. Paleta de colores	84
5.5. Vista de inicio de sesión	87
5.6. Vista de gestión del perfil	88
5.7. Vista de la selección de plantillas	89
5.8. Vista de las topologías del usuario	89
5.9. Vista de creación de una topología	90
5.10. Vista de confirmación de eliminación de una topología	90
5.11. Vista de la monitorización de la red sin terminal	91
5.12. Vista de la monitorización de la red con terminal	92
5.13. Vista de la monitorización de un nodo concreto	92
5.14. Vista de la consola de un nodo en concreto	93
5.15. Diagrama de despliegue del sistema	94

B.1. Pantalla de inicio de sesión de S2 Orchestrator.	148
B.2. Pantalla principal de la aplicación: Mis Topologías.	149
B.3. Ventana de creación de una nueva topología.	150
B.4. Ventana de confirmación para la eliminación de una topología.	151
B.5. Biblioteca de plantillas de topologías preconfiguradas.	152
B.6. Pantalla de ajustes del perfil de usuario.	153
B.7. Pantalla del monitor de red con la topología desplegada y los logs.	154
B.8. Panel de detalles y configuración de un nodo seleccionado.	155
B.9. Panel de detalles de un nodo de tipo conmutador (switch).	156
B.10. Consola de un nodo con registros disponibles.	157
B.11. Consola de un nodo sin registros disponibles.	157

Índice de tablas

3.1. Identificación de los principales riesgos del proyecto	32
3.2. Matriz de decisiones según Probabilidad e Impacto.	32
3.3. Riesgo RSK001: Errores al estimar las duraciones de las tareas.	33
3.4. Riesgo RSK002: Cambios en los requisitos durante el proyecto.	33
3.5. Riesgo RSK003: Curva de aprendizaje tecnológica excesiva.	34
3.6. Riesgo RSK004: Baja disponibilidad por otros compromisos.	35
3.7. Riesgo RSK005: Gold-Plating.	36
3.8. Riesgo RSK006: Inestabilidad en el entorno de desarrollo.	37
3.9. Riesgo RSK007: Baja por enfermedad del desarrollador o familiares.	38
3.10. Riesgo RSK008: Parada por análisis.	39
3.11. Riesgo RSK009: Pérdida de información.	40
3.12. Riesgo RSK010: Documentación técnica insuficiente.	41
3.13. Cálculo de retenciones y salario	42
3.14. Resumen del presupuesto total del proyecto	43
3.15. Resumen del presupuesto real del proyecto	44
3.16. Comparativa de esfuerzo temporal: Planificación inicial vs. Desarrollo real.	45
4.1. Requisitos funcionales del sistema	56
4.2. Requisitos no funcionales del sistema	58
4.3. Requisitos de información del sistema	59
4.4. Criterios de aceptación del sistema	60
4.5. Especificación del Caso de Uso: Iniciar sesión	63
4.6. Especificación del Caso de Uso: Registrar usuario	63
4.7. Especificación del Caso de Uso: Modificar perfil	64
4.8. Especificación del Caso de Uso: Consultar perfil	64
4.9. Especificación del Caso de Uso: Eliminar perfil	65
4.10. Especificación del Caso de Uso: Importar topología	65
4.11. Especificación del Caso de Uso: Gestionar repositorio de topologías	66
4.12. Especificación del Caso de Uso: Utilizar plantilla	66
4.13. Especificación del Caso de Uso: Desplegar topología	67
4.14. Especificación del Caso de Uso: Monitorizar nodos	67
4.15. Especificación del Caso de Uso: Gestionar topología activa	68
4.16. Especificación del Caso de Uso: Eliminar topología activa	68
4.17. Especificación del Caso de Uso: Consultar logs	69

6.1. Definición de los endpoints de la API y WebSockets	103
7.1. CP01. Registro	110
7.2. CP02. Inicio de sesión	110
7.3. CP02-1. Acceso no autorizado a rutas protegidas	111
7.4. CP03. Modificación del perfil	111
7.5. CP03-1. Modificación del perfil (Formato del avatar no valido)	112
7.6. CP04. Carga de topología YAML válida	112
7.7. CP04-1. Carga de topología YAML inválida (Sintaxis incorrecta)	113
7.8. CP04-2. Carga de topología YAML inválida (Formato no válido)	113
7.9. CP05. Despliegue y validación de tiempos	114
7.10. CP05-1. Despliegue y validación de tiempos (Fallo en el despliegue)	114
7.11. CP06. Parada de topología y eliminación de recursos	115
7.12. CP07. Reinicio de nodo	115
7.13. CP08. Modificación de los parámetros de un nodo	116
7.14. CP08-1. Modificación de los parámetros de un nodo (Formato no válido)	116
7.15. CP09. Visualización de logs en tiempo real	117
7.16. CP10. Monitorización de métricas de nodos	117
7.17. CP11. Monitorización de métricas de salud de la red	118
7.18. CP12. Importación de plantilla predefinida	118
7.19. CP13. Clonación de escenarios	119
7.20. CP14. Consulta del historial de topologías	119
7.21. CP15. Exportación completa de topología	120
7.22. CP16. Tiempos de carga de la interfaz	120
7.23. CP17. Eficiencia en el flujo de despliegue	121
7.24. CP18. Aislamiento de acceso entre usuarios	121
7.25. Respuestas en escala Likert (1-5) de los participantes al test SUS.	123

Glosario de términos

API (Application Programming Interface) : Conjunto de definiciones y protocolos que permiten que diferentes aplicaciones de software se comuniquen entre sí, como la conexión entre el frontend de React y el backend en Python.

Backend : Capa lógica del sistema encargada del procesamiento de datos y la comunicación con el motor de orquestación, implementada en Python en este sistema.

CLI (Command Line Interface) : Interfaz de línea de comandos tradicional utilizada para la gestión directa de dispositivos, la cual el sistema busca automatizar y abstraer mediante una interfaz gráfica.

Containerlab : Herramienta de orquestación de código abierto basada en contenedores Docker que permite desplegar infraestructuras de red virtuales de manera rápida y escalable mediante archivos YAML.

CRUD : Siglas de Create, Read, Update y Delete. Representa las funciones básicas de gestión de datos en aplicaciones informáticas. Se utiliza para describir la interfaz necesaria para administrar los recursos del sistema de orquestación de red.

Despliegue : Proceso de instalación automática de la infraestructura de red.

Docker : Plataforma de código abierto basada en contenedores que permite ejecutar los nodos de red de forma aislada, compartiendo el kernel del sistema anfitrión para optimizar recursos.

DTO (Data Transfer Object) : Objeto diseñado exclusivamente para el transporte de datos entre el frontend y el backend, optimizando la comunicación y seguridad del sistema.

Frontend : Parte de la aplicación con la que interactúa el usuario directamente, desarrollada en este proyecto mediante la biblioteca React.

IaC (Infrastructure as Code) : Práctica de gestionar y aprovisionar la infraestructura mediante archivos de definición legibles por la máquina, en lugar de configuraciones manuales o herramientas interactivas.

JSON (JavaScript Object Notation) : Formato ligero de intercambio de datos utilizado para la comunicación entre la interfaz en React y el controlador en Python.

Log : Registros de salida estándar (stdout/stderr) de los contenedores Docker que representan los nodos de la red.

MVP (Minimum Viable Product) : Versión inicial del sistema que incluye las funcionalidades esenciales para satisfacer los requisitos básicos de despliegue y orquestación.

MVVM (Model-View-ViewModel) : Patrón arquitectónico utilizado para desacoplar la interfaz de usuario (React) de la lógica de negocio, facilitando la reactividad en tiempo real.

MySQL : Sistema de gestión de base de datos relacional utilizado para la persistencia de información del sistema.

NetDevOps : Enfoque que aplica las filosofías de DevOps (automatización, integración y entrega continua) a la gestión y operación de redes.

Orquestador : Componente encargado de coordinar el ciclo de vida de los recursos virtuales (nodos y enlaces), asegurando que el estado real de la red coincida con el diseño definido en el software.

React : Biblioteca de JavaScript utilizada para construir la interfaz de usuario del sistema mediante una arquitectura basada en componentes.

SDN (Software Defined Networking) : Paradigma que propone la separación del plano de control (lógica) del plano de datos (reenvío), centralizando la gestión de la red en una entidad programable.

SPA (Single Page Application) : Arquitectura de aplicación web que ejecuta la carga de una sola página y actualiza el contenido dinámicamente sin necesidad de recargar el navegador.

SSOT (Single Source of Truth) : Principio de diseño que establece que un sistema debe poseer un único origen de datos autorizado para cualquier pieza de información. En el contexto de redes, asegura que todas las herramientas y procesos consulten el mismo registro de estado para evitar inconsistencias entre la configuración deseada y la real.

Topología : Conjunto de nodos (contenedores) y enlaces virtuales que forman un diseño de red específico dentro del sistema.

YAML (YAML Ain't Markup Language) : Formato de serialización de datos legible por los humanos utilizado en el proyecto para definir la estructura de las topologías de red y las configuraciones de los nodos.

Capítulo 1

Introducción

1.1. Contexto

Históricamente, el sector de las comunicaciones se ha basado en un modelo rígido y fuertemente ligado al hardware propietario [1]. Durante años, la administración y gestión de estas infraestructuras ha dependido de una gestión basada en dispositivos individuales. A pesar de la existencia de protocolos de red estándar para el intercambio de datos y de interfaces remotas (como GUIs o acceso vía SSH), su configuración y despliegue han seguido vinculados a un modelo operativo manual y atómico. En la mayoría de casos, esto requiere el uso de las interfaces de comandos (CLI) con sintaxis propietaria de cada fabricante, lo que genera una fuerte dependencia del administrador de red respecto a la infraestructura específica. Este modelo ha sido funcional en redes de todos los tamaños desde el inicio de Internet y sus primeras infraestructuras, sin embargo, debido al exponencial crecimiento tecnológico que ha experimentado el mundo a lo largo de las últimas décadas, ha empezado a mostrar deficiencias críticas tales como la falta de agilidad operativa, la alta posibilidad de error humano en las configuraciones repetitivas y la dificultad para mantener una visión unificada de la red en entornos heterogéneos, las cuales representan las principales causas de las constantes brechas de seguridad e interrupciones de servicios que se están viviendo hoy en día [2].

Esta limitación se ha hecho más evidente con la consolidación de nuevos paradigmas como el Cloud Computing o la virtualización de sistemas. En los entornos actuales, es posible desplegar cientos de máquinas y contenedores en cuestión de segundos, sin embargo, si la infraestructura de red sigue dependiendo de configuraciones manuales, se obtiene un desfase entre la rapidez con la que estos sistemas se pueden desplegar y la capacidad de respuesta de la infraestructura de red subyacente. Mientras que el almacenamiento y el cómputo han conseguido una gran escalabilidad y flexibilidad bajo demanda, la infraestructura de red se ha convertido en un cuello de botella que frena el despliegue de servicios [3]. La necesidad de sincronizar la agilidad de la red con la del software es lo que ha propulsado la búsqueda de soluciones basadas en la programación.

Frente al modelo tradicional, el paradigma de las Redes Definidas por Software (SDN) ofrece ventajas competitivas fundamentales, como la abstracción del hardware, la centralización de la inteligencia

y la capacidad de innovar sin depender de actualizaciones físicas de los fabricantes. No obstante, esta transición no está exenta de compromisos técnicos. La centralización del control introduce un punto único de fallo potencial (Single Point of Failure) y añade una capa de complejidad en el desarrollo de software que no existe en el hardware autónomo. A diferencia del enfoque tradicional, donde el plano de control y el plano de datos se situaban en el mismo dispositivo, la SDN proporciona una separación funcional [4]. Esto permite que el despliegue de políticas evolucione desde una gestión basada en scripts específicos hacia un proceso de orquestación mediante código, tratando a la red como una entidad única y programable. Este enfoque ha sido validado por líderes del sector, un ejemplo es la red B4 de Google, que utiliza SDN para conectar sus centros de datos logrando una eficiencia cercana al 100 % [5].

Para materializar este cambio hacia la programabilidad, la industria ha adoptado una filosofía Net-DevOps y el concepto de infraestructura como Código (IaC). Esto permite tratar la configuración de los dispositivos con el mismo rigor que el desarrollo software, empleando sistemas de control de versiones y herramientas de automatización. En este contexto, soluciones como Ansible o lenguajes de Python se muestran como pilares para gestionar infraestructuras complejas y garantizar que el estado real de la red coincida con el diseño elaborado.

Finalmente, la necesidad de automatizar reside también en la búsqueda de una consistencia operativa. Como señala IBM [6], la automatización del despliegue no solo aporta rapidez, sino que también refuerza considerablemente la seguridad de la infraestructura. Eliminamos la variabilidad de la intervención humana asegurando que todas las políticas de seguridad y de segmentación de la red se apliquen de manera idéntica y sin omisiones en todos los elementos, facilitando el cumplimiento de normativas y permitiendo una recuperación ante desastres mucho más rápida, ya que la red puede ser reconstruida a partir de los archivos de configuración reduciendo los tiempos de recuperación de días o semanas a apenas unos minutos. No obstante, es preciso señalar que la automatización introduce nuevos desafíos, un error en la lógica del código o en los archivos de configuración puede propagarse de forma instantánea a toda la infraestructura, aumentando la magnitud de impacto de un fallo. Por ello, la transición hacia redes programables requiere de la implementación de mecanismos robustos de validación, pruebas en entornos de emulación y procesos de despliegue controlados para mitigar los riesgos propios de procesos automatizados.

1.2. Motivación

El presente Trabajo de Fin de Grado se centra en el diseño e implementación de un sistema capaz de desplegar y configurar infraestructuras de red mediante modelos declarativos de Infraestructura como Código (IaC). La principal motivación de este proyecto nace de la necesidad de unir la teoría de las redes basadas en software con su aplicación práctica en entornos reales ya que a pesar de los avances en SDN, la transición hacia estos modelos presenta una elevada curva de aprendizaje y una complejidad técnica inicial que, en muchos casos, dificulta su adopción en infraestructuras tradicionales. Como señala la industria, existe una desconexión crítica entre la disponibilidad de herramientas de automatización y la capacidad de la organización para integrarlas en flujos de trabajo seguros y unificados. La

mayoría de las herramientas actuales requieren un alto grado de especialización o están fragmentadas, lo que justifica la necesidad de desarrollar sistemas que actúen como una capa de abstracción [7], simplificando la interacción con el hardware y permitiendo una gestión unificada.

A nivel profesional, este proyecto está impulsado por la voluntad de dominar el paradigma Net-DevOps. La industria está dejando atrás al administrador de red que solo conoce la CLI y buscando perfiles que traten a la red como software [8]. Enfrentarse al reto de integrar lenguajes como Python y herramientas de automatización en un flujo de trabajo real con entornos de simulación supone una preparación para los desafíos tecnológicos actuales y permite explorar el cruce entre el desarrollo del software, la ingeniería de sistemas y las telecomunicaciones.

Por último, existe un interés en validar la eficiencia operativa que aporta la automatización. En entornos de red modernos, el tiempo destinado a configuraciones manuales y repetitivas representa una carga de trabajo que ralentiza el despliegue de servicios. El objetivo de este proyecto es mostrar cómo, mediante una aproximación programable, es posible reducir el tiempo de aprovisionamiento y minimizar los errores de configuración. Validar esta mejora en la agilidad mediante una prueba de concepto (PoC) es uno de los objetivos principales, permitiendo observar de forma práctica la diferencia entre la gestión tradicional y la orquestada.

1.3. Objetivos

El objetivo de este proyecto es diseñar e implementar una plataforma web para la orquestación y el despliegue automatizado de topologías de red virtualizadas basadas en contenedores (Docker). Mediante el uso del motor Containerlab, el sistema permite definir infraestructuras de red mediante archivos de configuración, facilitando el control operativo y la monitorización de los nodos desde un entorno centralizado. Este objetivo principal se desglosa en los siguientes objetivos secundarios:

Objetivos del Proyecto

- Desarrollar un flujo de automatización capaz de aprovisionar infraestructuras de red funcionales a partir de definiciones lógicas estructuradas.
- Habilitar un entorno de gestión que facilite el despliegue y la operación de la red, ofreciendo un asistente guiado que abstraiga la complejidad técnica para distintos perfiles de usuario.
- Implementar un modelo de persistencia híbrido basado en la Fuente Única de Verdad (SSOT).
- Mejorar la eficiencia en el aprovisionamiento de la infraestructura, reduciendo la intervención manual mediante la generación automática de configuraciones de red.
- Implementar un sistema de monitorización y gestión operativa centralizada para la visualización y control de los nodos.

Objetivos Personales

Dado el carácter académico de este Trabajo de Fin de Grado, se plantean una serie de metas orientadas al crecimiento profesional en el ámbito de la ingeniería de redes y el desarrollo software:

- Profundizar en el desarrollo de lógica de negocio orientada a redes, empleando lenguajes de alto nivel para el tratamiento de datos y la gestión automatizada de configuraciones.
- Adquirir experiencia práctica en herramientas de orquestación de infraestructuras para la gestión eficiente de inventarios y ejecución de tareas en entornos heterogéneos.
- Dominar el paradigma NetDevOps, integrando metodologías de desarrollo de software (control de versiones, CI/CD y lenguajes de programación) con la ingeniería de redes tradicional.
- Fortalecer la capacidad de planificación y gestión de proyectos tecnológicos, afrontando el ciclo de vida completo del desarrollo: desde el análisis de requisitos y el diseño arquitectónico hasta la implementación y las pruebas de validación finales.

1.4. Estructura de la memoria

Tras esta sección de introducción, el contenido de la memoria se estructura de la siguiente manera:

Capítulo 2. Estado de la cuestión: Se realiza una investigación técnica sobre la arquitectura y componentes de los sistemas SDN, su taxonomía y clasificación. Además, se presenta una comparativa de sistemas, herramientas de emulación de redes, lenguajes de automatización y plataformas de gestión de redes virtualizadas y se analizan diversos ejemplos de uso de las redes SDN.

Capítulo 3. Metodología y Planificación: Se detalla el marco de trabajo basado en el diseño iterativo e incremental junto con la organización de tareas mediante tableros Kanban integrados en Notion y la priorización de requisitos bajo la técnica MoSCoW. Se incluyen las herramientas utilizadas, la declaración sobre el uso de la Inteligencia Artificial y la planificación inicial, incluyendo el análisis de riesgos, la estimación de costes y el presupuesto real del proyecto. Por último, se incorpora la comparación del desarrollo final respecto a la planificación inicial y las lecciones aprendidas durante todo el proceso.

Capítulo 4. Análisis: Define el sistema mediante la especificación de requisitos funcionales (RF), no funcionales (RNF) y de información (RI). En este capítulo se detallan los Casos de Uso (CU) que rigen la interacción con el usuario y el Modelo de Dominio que estructura la lógica de las entidades de red. Además, se incorpora el análisis del comportamiento dinámico mediante Diagramas de Secuencia, que ilustran el intercambio de mensajes entre los componentes de la arquitectura para el cumplimiento de las funciones críticas del sistema.

Capítulo 5. Diseño del sistema: Se describe la arquitectura técnica, incluyendo el patrón MVVM y el diagrama de paquetes que define la organización lógica de sus componentes, la estructura de la base de datos, reglas de negocio y patrones de diseño. Se detalla la especificación del sistema mediante el diagrama de despliegue y se recogen las decisiones de diseño adoptadas, tales como el entorno de desarrollo, identidad visual, el control de acceso mediante Google Auth, el mecanismo de seguridad de confirmación por entrada de texto para la eliminación de topologías y la especificación de la interfaz de usuario.

Capítulo 6. Descripción de las iteraciones del sistema: Se detalla el proceso incremental de desarrollo. Se documentan los aspectos técnicos, la automatización mediante scripts en Python, la implementación de la persistencia de datos, el diseño de las topologías y la integración *full-stack*, documentando en cada etapa los objetivos técnicos, los contratiempos surgidos y las soluciones adoptadas para garantizar la correcta validación del producto final.

Capítulo 7. Pruebas y validación: Se describe la estrategia de pruebas empleadas para validar el correcto funcionamiento del sistema. Se detallan los escenarios de configuración, el manejo de errores y la evaluación de la usabilidad, garantizando así la fiabilidad, robustez y adaptabilidad de la herramienta frente a las expectativas del usuario.

Capítulo 8. Conclusiones: Se realiza un balance final del proyecto comparando los resultados obtenidos con los objetivos planteados, la valoración personal y se proponen futuras líneas de trabajo centradas en mejoras técnicas y de experiencia de usuario.

Bibliografía y Anexos: Se incluye el listado de referencias bibliográficas y documentación técnica adicional que abarca el manual de despliegue, el manual de uso de la aplicación y el acceso al repositorio del código fuente.

Capítulo 2

Estado de la cuestión

En esta sección se va a llevar a cabo un estudio para justificar la elección y fundamentar la base tecnológica de este sistema. Primero, se va a analizar la arquitectura y los componentes básicos que forman parte de las Redes Definidas por Software (SDN) y su funcionamiento. Posteriormente, se detallan distintos enfoques de automatización y por último, se van a citar distintos sistemas y herramientas similares para identificar las limitaciones de las soluciones actuales y posicionar al sistema desarrollado como una alternativa eficiente, escalable y adaptada a las necesidades de gestión de infraestructuras virtualizadas.

2.1. Arquitectura y componentes de los sistemas SDN

Típicamente, las arquitecturas tradicionales se componen de una serie de dispositivos que actúan como un equipo cerrado o caja negra conformando una integración vertical de la misma. En su interior, contienen tanto la lógica de enrutamiento como el hardware de reenvío que permite mover los paquetes desde la interfaz de entrada hasta la de salida [9]. Sin embargo, SDN rompe con esa filosofía arquitectónica e instaura un nuevo paradigma. Elimina la integración vertical, separa el plano de control del plano de datos creando un plano de gestión centralizado que permite a los administradores controlar y dirigir los dispositivos de una manera más fácil y eficiente con respecto a las redes tradicionales [10].

Una posible alteración en la topología de red producida por la adición de un nuevo dispositivo en una interfaz, conlleva un proceso de negociación entre los routers. En redes de gran escala, este proceso puede tardar segundos o minutos en realizarse y durante ese tiempo, se pueden producir bucles o pérdidas de paquetes. Si la alteración ha sido producida por una nueva política de seguridad, el administrador debe ir configurando la lista de acceso de forma manual en cada dispositivo.

En las redes SDN, sin embargo, este escenario se simplifica al disponer de un controlador de red que dispone de una visión global. Los cambios topológicos se gestionan de manera global y automáticamente suprimiendo las esperas por los procesos de negociación mencionados anteriormente. De igual manera, las políticas de seguridad se gestionan de forma uniforme en toda la infraestructura de

red, eliminando así la necesidad de intervenir en cada dispositivo y desplegando el cambio de forma inmediata [11].

No obstante, el concentrar la inteligencia de la red en un único punto genera una serie de inconvenientes, los cuales derivan de la principal y potencial virtud de este paradigma, la centralización. El controlador se convierte en un punto único de fallo (SPOF) generando una inadaptabilidad de la red frente a posibles cambios.

Consecuentemente, al residir la mayor vulnerabilidad en el controlador, este se convierte en objeto directo de ataques cibernéticos. En redes tradicionales, una intrusión a un router comprometería la disponibilidad de dicho dispositivo, dejando el resto de la red intacta. A diferencia de este enfoque, en las redes SDN, lograr vulnerar el controlador supone comprometer toda la infraestructura de red [12]. Además de los riesgos relacionados con la seguridad, surge un reto relacionado con el rendimiento, el controlador puede convertirse en un cuello de botella. Al abstraer la inteligencia de los dispositivos, estos carecen de autonomía y dependen totalmente del controlador, por lo tanto, cada vez que se produce un cambio en la topología o se recibe un flujo entrante desconocido, se debe pausar el reenvío de paquetes y mandar una solicitud al controlador para saber como procesar.

En las redes tradicionales, este fenómeno no ocurre debido a que cada dispositivo de red toma sus decisiones de reenvío de forma autónoma y distribuida. Al estar el plano de control integrado en cada nodo, la información de enrutamiento y las políticas se procesan localmente, eliminando la dependencia de un nodo central y evitando que el tráfico de control sature un único punto de la infraestructura.

Las arquitecturas de red se componen de tres planos: el plano de control, el plano de datos y el plano de gestión. A continuación, se detalla el propósito de cada uno:

- **Plano de Control (Control Plane):** El cerebro de la red, como se conoce informalmente, se encarga de transportar la información que se mueve a través de la red y de controlar el enrutamiento, el cómo se envían los paquetes de datos de un lugar a otro [13]. Sus funciones son las siguientes:
 - **Enrutamiento:** Ejecuta protocolos de enrutamiento para intercambiar información de enrutamiento entre los distintos dispositivos de red y determina la mejor ruta para que los paquetes viajen por la red basándose en métricas como: el estado del enlace, las condiciones del tráfico, la topología de red entre otras.
 - **Balanceo de carga:** Ejecuta un proceso denominado planificador que distribuye la carga de trabajo entre los distintos nodos que componen la red, aumentando la disponibilidad y la experiencia del usuario.
 - **Gestión del tráfico:** Aplica políticas de red definidas, como las listas de acceso (ACLs) o las reglas de calidad de servicio (QoS). Garantiza que el tráfico se encamine conforme a estas políticas para aumentar el rendimiento y la seguridad de la red.
 - **Mantenimiento de la topología de red:** Mantiene la topología de la red, un grafo que especifica la disposición de los dispositivos y el estados de los mismos. Se adapta a posibles cambios y gestiona las tablas de enrutamiento que especifican cómo deben transmitirse los datos a través de la red.

- **Plano de Datos (Forwarding Plane / Data Plane):** Conocido como el músculo de la red, comprende los componentes hardware y circuitos electrónicos que transportan los datos desde la interfaz de entrada a la de salida lo más rápido posible [14]. Entre sus funciones destacan las siguientes:
 - **Reenvío de paquetes:** Reenvía paquetes desde la interfaz de entrada a la de salida basándose en las decisiones de enrutamiento que ha recibido del plano de control.
 - **Procesamiento de paquetes:** Realiza ciertas tareas de procesamiento de paquetes, como pueden ser la puesta en cola, programación, filtrado y clasificación. Además puede implicar la encapsulación, desencapsulación y modificación del encabezado del paquete.

De forma análoga al plano de control, el plano de datos aplica reglas de calidad de servicio (QoS), balanceo de carga y políticas de seguridad. Estas acciones, que incluyen el filtrado mediante listas de acceso (ACLs) o reglas de firewall, se procesan analizando campos de la cabecera del paquete, como direcciones IP de origen y destino, números de puertos y protocolos, con el fin de optimizar el rendimiento y proporcionar seguridad a la red.

- **Plano de Gestión (Management Plane):** Se trata de la capa administrativa y se utiliza para configurar, monitorizar y gestionar el dispositivo a través de conexiones SSH, SNMP y APIs [15].

Una vez definido los planos lógicos sobre los que se sustenta el paradigma SDN y que justifican su eficiencia, se detallan los elementos arquitectónicos que permiten llevar este modelo a la práctica. Según el modelo de referencia propuesto por la ONF (Open Networking Foundation), las redes definidas por software se dividen en tres capas de componentes, las cuales se comunican por medio de interfaces de programación de aplicaciones (APIs): la capa de infraestructura, la capa de control y la capa de aplicación.[16]

- **Capa de infraestructura:** Constituye el fundamento físico de la red SDN. Esta compuesta por los dispositivos de red (físicos o virtuales, como *routers* o *switches*) denominados *forwarding devices*, ya que dichos dispositivos han sido despojados de su inteligencia y se han convertido en dispositivos de reenvío simple. Su única función es la de procesar y mover los datos a alta velocidad basándose en las tablas de flujo y las reglas que el controlador les ha proporcionado. En entornos modernos, estos dispositivos son sustituidos por nodos virtualizados o contenedores que se ejecutan sobre sistemas operativos de red.
- **Capa de control:** El controlador es el cerebro y el núcleo de la red. Se trata de una plataforma software que mantiene una visión global y en tiempo real de toda la topología y enlaces activos. Sus principales funciones son:
 - **Conversión de requisitos:** Convierte los requisitos de alto nivel definidos por las aplicaciones en reglas de bajo nivel para que los dispositivos las puedan entender.
 - **Gestión del estado:** Monitoriza la red para detectar posibles caídas de enlaces y recalcular las rutas de forma dinámica e inmediata.
 - **Abstracción:** Oculta la complejidad del hardware subyacente ofreciendo a los administradores una visión global y unificada de la red.

- **Capa de aplicación:** En la capa superior se encuentran las aplicaciones SDN. Son programas o scripts que definen la lógica de negocio y el comportamiento de la red. Los ingenieros, en lugar de utilizar comandos, desarrollan aplicaciones para tareas específicas, como balanceo de carga, cortafuegos de seguridad (firewalls), políticas de calidad de servicio (QoS), entre otras. Estas aplicaciones definen el que hacer mientras que el controlador define cómo se debe hacer.

Por último, para que estas tres capas puedan interactuar independientemente de un fabricante, se definen dos tipos de interfaces estandarizadas.

- **Southbound APIs (Interfaces en dirección Sur):** Conecta el controlador con los dispositivos de red. Es el protocolo que se utiliza para enviar las reglas a dichos dispositivos. Durante la historia de las redes SDN, han existido enfoques propietarios impulsados por fabricantes (como la extinta API *onePK* de Cisco); sin embargo, actualmente predominan estándares abiertos, tales como OpenFlow, protocolos modernos basados en modelos de datos YANG (NETCONF) o incluso conexiones automatizadas vía SSH.
- **Northbound APIs (Interfaces en dirección Norte):** Conecta las aplicaciones con el controlador. Normalmente se implementan con APIs RESTful lo que permite que un desarrollador software (usando Python, por ejemplo) pueda programar el comportamiento de la red a través del controlador sin la necesidad de conocer el lenguaje de bajo nivel de los dispositivos.

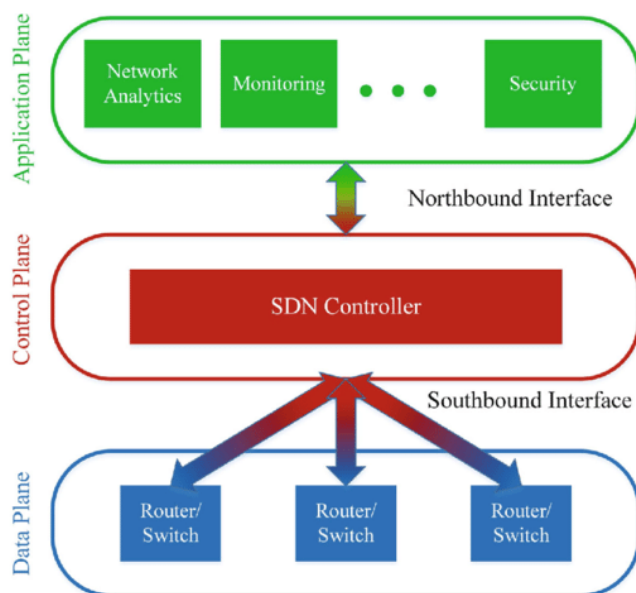


Figura 2.1: Arquitectura de referencia de los sistemas SDN. Fuente: adaptado de [17].

2.2. Taxonomía y clasificación de las redes SDN

En el ámbito de la ingeniería, este paradigma no se materializa en una única solución tecnológica, sino que abarca una amplia variedad de implementaciones. Para entender en profundidad las redes

SDN, es imprescindible establecer una taxonomía que clasifique estas redes atendiendo a diferentes dimensiones o criterios de diseño. A continuación, se examinarán los distintos enfoques de implementación organizados según la arquitectura del controlador, la gestión de los flujos, su topología lógica y el modelo tecnológico subyacente.

2.2.1. Según la arquitectura del controlador:

La organización y localización del plano de control es un factor determinante en el rendimiento y disponibilidad de una red SDN. Dependiendo de cómo se distribuya la lógica de control, se pueden distinguir tres modelos principales [18].

- **Centralizada:** Se trata de un modelo que dispone de un único controlador centralizado que se encarga de gestionar toda la red. Dispone de ciertas ventajas tales como que es más fácil de implementar, los costes de operación se reducen, ofrece una perspectiva completa del estado de la red y facilita su gestión al centralizar las tareas en un único punto. Sin embargo, al disponer de un único punto que gestiona la red, este no solo se establece como un punto único de fallo (SPOF) sino que, si la capacidad de procesamiento de dicho controlador no puede escalar en la misma medida que la red, acaba convirtiéndose en un cuello de botella.
- **Distribuida:** Al contrario que la centralizada, este modelo dispone de una serie de controladores distribuidos, cada uno de ellos encargándose de un segmento de la red y colaborando entre ellos para lograr una gestión global. Disponen de una mayor complejidad ya que hay que gestionar y coordinar múltiples controladores a cambio de una mayor flexibilidad (permite una gestión más granular con lo que se adapta mejor a topologías complejas), una mayor disponibilidad (si un controlador cae, los demás pueden seguir gestionando sus segmentos de red) y por último, una mayor escalabilidad ya que se distribuye la carga entre los distintos controladores.
- **Híbrida:** Se trata de un modelo que combina características de los anteriores para mitigar algunos de los inconvenientes que estos presentan. Está formada por un controlador en el que se concentran tareas de inteligencia estratégica y visión completa de la red, mientras que delega tareas rutinarias o de respuesta inmediata en controladores locales. Esta división permite mejorar la resiliencia y optimizar la escalabilidad, ya que garantiza que la red reaccione a eventos locales sin depender constantemente del controlador global.

2.2.2. Según el modelo tecnológico subyacente:

El protocolo de comunicación y la interfaz usada entre el plano de control y el plano de datos es otro de los criterios importantes a la hora de clasificar las redes. Dependiendo de si se emplean estándares abiertos o interfaces de programación específicas, podemos distinguir dos vertientes [19].

- **SDN Abierta:** Representa la implementación más pura de SDN. Se basa en el uso de protocolos de comunicación estandarizados y de código abierto, donde OpenFlow se ha consolidado como

el referente principal. En este tipo de arquitectura, el plano de control tiene la capacidad de programar directamente las tablas de flujos de los dispositivos favoreciendo la interoperabilidad y evitando el riesgo de dependencia de un fabricante (*vendor lock-in*). Los dispositivos físicos actúan como elementos de envío permitiendo a los administradores definir protocolos y políticas de red personalizadas desde el controlador. Convierte la infraestructura en un entorno maleable y adaptable a las necesidades al permitir que desarrolladores puedan acceder a las especificaciones del protocolo para optimizar el comportamiento de la red.

- **SDN Basada en APIs:** A diferencia del modelo anterior, este enfoque no busca la programación del hardware bit a bit, sino que interactúa con él, a través de interfaces de programación de aplicaciones (APIs) como REST, NETCONF o SNMP. Los dispositivos de red conservan parte de su inteligencia y el controlador utiliza estas APIs para invocar funciones específicas o solicitar cambios en la configuración del dispositivo. Es extremadamente popular entre los fabricantes de hardware tradicional ya que les permite usar parte de las capacidades avanzadas y rendimiento optimizado que viene integrado en los equipos de fábrica, haciendo que la transición desde redes tradicionales sea más gradual. Busca un equilibrio entre la automatización centralizada y el aprovechamiento del hardware propietario, permitiendo una orquestación ágil sin necesidad de que el controlador gestione cada entrada de la tabla de flujos en los dispositivos de red.

2.2.3. Según la gestión de los flujos:

Otro criterio importante en la taxonomía de las redes SDN es el mecanismo mediante el cual, el controlador inyecta las reglas en los dispositivos de red. Define como reacciona la infraestructura ante la llegada de tráfico nuevo e influye en la latencia y la carga de procesamiento del controlador [20].

- **Reactivo:** En este modelo de despliegue, el switch parte de un estado en blanco, es decir, que no dispone de tablas de flujo. El primer paquete que recibe dispara una llamada (se conoce como *Packet-In*) al controlador ya que el switch no sabe qué hacer con este paquete. El controlador calcula la ruta, se la envía al switch y el paquete es enviado. Las principales desventajas de este tipo de modelo son que el primer paquete de cada conexión dispone siempre de una latencia extra por la acción de preguntar al controlador y que en situaciones donde haya un aumento de tráfico nuevo, el controlador se satura de preguntas por parte de los switches pudiendo llegar a convertirse en un cuello de botella.
- **Proactivo:** En este modelo, a diferencia del anterior, el controlador antes de recibir cualquier tráfico de los usuarios, calcula las rutas e inyecta las reglas en los switches. De esta manera, cuando un paquete llega al switch, este sabe lo que tiene que hacer con él. Al no tener que realizar la consulta al controlador, los retrasos y la posibilidad de que el controlador se convierta en un cuello de botella desaparece. Este tipo de modelo funciona de forma similar a protocolos como OSPF. La desventaja es que consume demasiada memoria ya que cada switch tiene que tener las rutas precargadas.
- **Híbrido:** Este modelo combina los dos modelos explicados anteriormente. Surge para mitigar los inconvenientes de los otros modelos. Dispone de una parte reactiva para el tráfico de los usuarios

por su carácter imprevisible y de una parte proactiva para el tráfico crítico y previsible (por ejemplo, el controlador siempre inyecta reglas para el tráfico de gestión y para que los switches puedan verse entre sí).

2.2.4. Según la topología de red:

Las redes SDN se caracterizan por su capacidad de separar la infraestructura de la capa de servicios que ofrecen al usuario. Esta separación nos permite clasificar la topología en dos capas que van a actuar conjuntamente y de manera coordinada [17].

- **Superpuesta (Overlay):** Crea una red virtual sobre la infraestructura física ya existente. Se utiliza la programabilidad para crear túneles virtuales, mediante un hipervisor y puntos finales de túnel (VTEPs), el sistema es capaz de encapsular todo el tráfico en el borde de la red permitiendo que la información se envíe de forma transparente a través de los túneles virtuales que se han creado. Este enfoque permite al administrador gestionar los recursos de manera dinámica, asignar anchos de banda específicos a cada canal y crear redes aisladas y personalizadas para cada usuario mientras la red subyacente permanece intacta.
- **Subyacente (Underlay):** Representa la infraestructura física englobando dispositivos de hardware como conmutadores, enrutadores, cableado de fibra óptica y puntos de acceso. Se encarga de gestionar los protocolos de enrutamiento tradicionales y la topología física para garantizar la conectividad básica de la red y el transporte de los paquetes de datos de un punto a otro de la manera más eficiente y robusta. En el modelo SDN, actúa como la base sobre la que se van a construir las redes virtuales, centrándose exclusivamente en el rendimiento y fiabilidad del transporte físico.

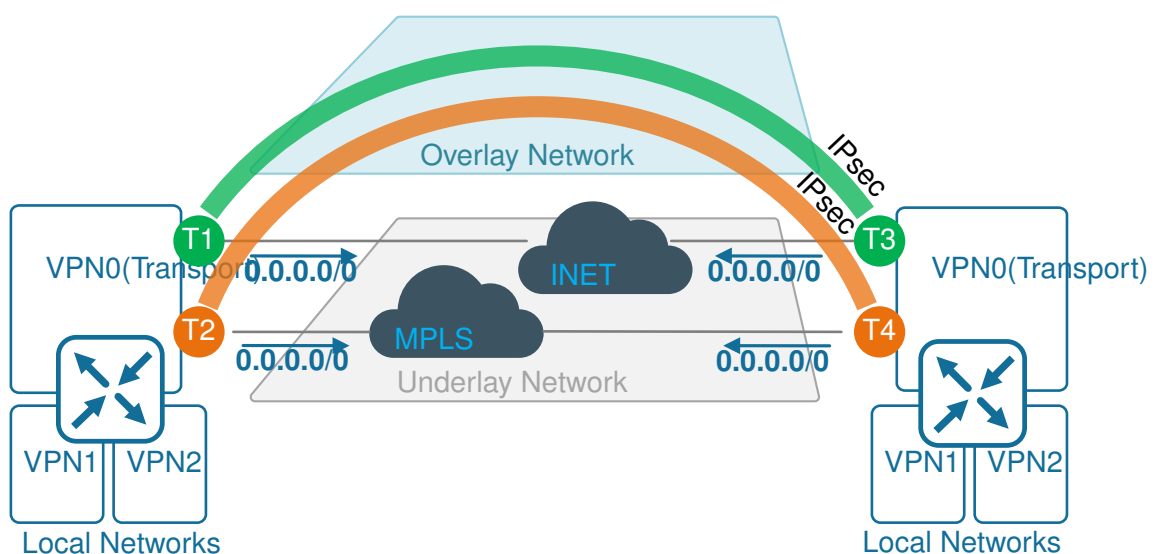


Figura 2.2: Representación del enrutamiento Underlay frente a Overlay. Fuente: basado en [21].

Estas cuatro clasificaciones no son excluyentes, sino que se combinan para definir la red SDN. La

elección de la arquitectura de control, el modelo de comunicación y la gestión de flujos va a determinar cómo de escalable es la red y el rendimiento de los recursos que operan sobre ella.

2.3. Ejemplos de uso de las redes SDN

En esta sección se van a exponer los principales escenarios de aplicación de las redes SDN, es decir, entornos donde la separación del plano de datos y el plano de control permiten una gestión más eficiente y una mayor adaptabilidad a la demanda del tráfico, analizando soluciones industriales que se han adoptado.

- **Centros de Datos y Cloud Computing:** La aplicación de esta solución en estos entornos se debe a la necesidad de eliminar la rigidez de la configuración manual, permitiendo que la red crezca al mismo ritmo que la demanda. Un caso crítico es el aprovisionamiento dinámico, donde el controlador SDN interactúa con el resto de orquestadores para configurar la conectividad y las reglas de seguridad en nuevas máquinas, sin necesidad de intervención humana. La creación de redes superpuestas (overlays) permite encapsular el tráfico de cada cliente en túneles independientes, lo que permite que varias empresas utilicen la misma IP dentro del mismo centro de datos sin que sus tráficos se crucen. Por último, la seguridad en redes SDN, se aplica en la interfaz de red virtual (vNIC) permitiendo que el tráfico se inspeccione paquete a paquete directamente en el hipervisor. Un ejemplo de este caso de uso es la implementación que hizo eBay, sustituyendo el modelo de red tradicional por una arquitectura basada en OpenStack [22].
- **SD-WAN:** La mayor innovación de SD-WAN es la gestión dinámica de las rutas basadas en aplicaciones. Los dispositivos de borde de las SD-WAN admiten varias conexiones mediante distintos medios de transporte (LTE, MPLS, 5G), esto permite que el sistema mediante la monitorización de ciertos parámetros como el jitter o la pérdida de paquetes en múltiples enlaces (MPLS, 5G, Fibra), redirija automáticamente el tráfico de esa aplicación por una ruta de mayor calidad, optimizando el ancho de banda y garantizando la continuidad del servicio. Un ejemplo de este caso de estudio es el caso de la tienda de ropa de Gap Inc que utilizó esta tecnología para interconectar sus tiendas globales, logrando niveles más altos de disponibilidad y resiliencia [23].
- **Redes de Campus y Acceso:** Se centra en resolver la complejidad de gestionar miles de dispositivos de diferente naturaleza. En redes tradicionales, el acceso se define mediante VLANs estáticas y direcciones IP ligadas a un puerto de acceso. Las redes SDN introducen el concepto de segmentación dinámica basada en políticas. El controlador identifica al usuario en el momento en el que se conecta a la red y le va a asignar una etiqueta de grupo escalable (SGT). Esta etiqueta define los privilegios de acceso de forma lógica, por ejemplo, un estudiante tendrá acceso a los recursos de la biblioteca y el controlador denegará cualquier intento de conexión a las cámaras de seguridad sin importar de dónde se encuentre físicamente. Un caso real que aplica estos principios es el del Deltion College. Aunque su enfoque inicial fue el de alta disponibilidad de servicios educativos, la implementación de una arquitectura SDN basada en OpenFlow también permitió una mejora en la seguridad perimetral y la protección contra ataques [24].

- **Interconexión de Centros de Datos:** La interconexión de los centros de datos mediante SDN permite gestionar múltiples ubicaciones geográficas como si fueran una única infraestructura lógica. En redes tradicionales, unir centros de datos requería configuraciones estáticas que desperdiciaban gran parte del ancho de banda disponible para evitar posibles picos de tráfico y que la red se congestionara. En la aplicación de SDN, el controlador tiene una visión global de los enlaces y calcula rutas óptimas en tiempo real. Esto permite la migración de máquinas virtuales en vivo entre ciudades sin interrumpir el servicio, clave para la recuperación ante desastres. Un caso de éxito es la red SWAN (Software-Defined WAN) de Microsoft quienes implementaron esta tecnología para conectar sus centros de datos de Azure a nivel global [25].

2.4. Tecnologías para la emulación de redes virtualizadas

Para el desarrollo de un sistema de automatización de la red, es necesario analizar las tecnologías actuales que permiten la emulación de topologías y la gestión de los flujos de datos. A continuación, se van a describir los principales paradigmas y herramientas disponibles.

2.4.1. Análisis de entornos de emulación de red

Podemos dividir las tecnologías de virtualización en dos grupos dependiendo de si estas se emulan directamente en el hardware del sistema o si comparten núcleo con el sistema operativo.

- **Virtualización completa (Basada en hipervisor):** Implica la creación de múltiples sistemas operativos que se van a ejecutar de manera independiente en un sistema operativo anfitrión. Cada sistema operativo invitado se comporta como una máquina independiente, lo que permite que se instalen y ejecuten aplicaciones y servicios de manera aislada. Para ello, se va a utilizar un hipervisor, que se encarga de crear y gestionar las máquinas virtuales. Con este paradigma se consigue una alta flexibilidad y aislamiento, ya que los sistemas operativos invitados no son conscientes de la existencia del hipervisor. De esta manera, cada uno de los sistemas operativos invitados puede utilizar su propio conjunto de recursos y configuración, permitiendo que se ejecuten distintos sistemas operativos en un mismo hardware. Sin embargo, este modelo de virtualización genera un mapeo constante entre los recursos físicos y virtuales, lo que implica una sobrecarga del sistema y afecta tanto al tiempo de respuesta como a la capacidad de procesamiento [26]. Algunas herramientas de emulación tradicionales que usan este modelo son GNS3 o EVE-NG.
- **Virtualización ligera (Basada en contenedores):** En este método de virtualización, se va a ejecutar una capa de virtualización sobre la que se van a ejecutar múltiples instancias aisladas que representan el espacio del usuario, denominadas contenedores. Estos contenedores contienen las librerías, marcos y otras dependencias necesarias para la ejecución de las aplicaciones que contienen. Este paradigma evita la sobrecarga característica del paradigma anterior debido a que todos los

contenedores de una máquina comparten el kernel del sistema operativo anfitrión. Al eliminar el hipervisor y el sistema operativo invitado, se suprime el complejo mapeo de recursos, permitiendo que las aplicaciones interactúen de manera casi directa con el hardware.

Sin embargo, el nivel de aislamiento es menor, dado que todos los contenedores comparten tanto el hardware como el sistema operativo anfitrión. Si un atacante consigue saltar la barrera del contenedor, obtendrá acceso al núcleo y por ende, acceso a los demás contenedores [27]. En este modelo, destacan herramientas como Containerlab o Mininet

2.4.2. Herramientas y lenguajes para la automatización del despliegue y gestión de redes virtualizadas

En el contexto de las Redes Definidas por Software, la configuración manual de los dispositivos se ha sustituido por el concepto de Infraestructura como Código (IaC) [28]. Para que un sistema de automatización del ciclo de vida de redes virtuales sea eficiente, necesitamos dos pilares fundamentales. Este proceso abarca desde la definición declarativa de la infraestructura hasta su despliegue y gestión operativa, delegando en el software la ejecución de las tareas repetitivas de aprovisionamiento y configuración:

- **Formatos de serialización de datos:** Para que un sistema de orquestación de infraestructura pueda interpretar la topología de red, es necesario que la estructura de la información sea legible tanto para los humanos como para las máquinas. Los lenguajes de marcado ligero (JSON y YAML) cumplen esta función mediante la serialización, un proceso que se basa en los siguientes principios [29]:
 - **Estructura de Pares Clave-Valor:** Consiste en asignar un identificador único (clave) a un dato específico (valor) permitiendo que el software de automatización localice información específica de la red de forma instantánea.
 - **Organización Jerárquica:** Estos lenguajes permiten anidar datos. Esto es crucial en redes para representar la dependencia física y lógica; por ejemplo, dentro de un objeto “Nodo”, podemos tener los objetos “Interfaces”, y dentro de cada interfaz, su “Dirección IP”.
 - **Abstracción del Hardware:** Usando estos formatos separamos la configuración del dispositivo de la sintaxis específica del fabricante. Los administradores definen qué debe tener la red y el software de automatización se encarga de traducir esa estructura a los comandos específicos que cada equipo necesita.
- **Lenguajes de programación:** Su función en este contexto, se centra en crear una capa de abstracción y actuar como una interfaz entre el ingeniero y la infraestructura [30].
 - **Librerías predefinidas:** La potencia de un lenguaje reside en la capacidad de manejar protocolos de comunicación (NETCONF, REST) de manera nativa con el uso de librerías predefinidas en él. Esto le permite entender al dispositivo y facilita las tareas como análisis de tráfico, modificación de rutas y recolección de estadísticas en tiempo real.

- **Capacidad de interoperabilidad:** El lenguaje permite conectar herramientas heterogéneas. Puede leer una topología definida en un formato de serialización, procesarla y transmitir la información a un orquestador de contenedores o a un controlador SDN.
- **Abstracción de la complejidad:** El lenguaje permite crear funciones genéricas que ocultan la complejidad del hardware. Esto hace que una misma lógica sea aplicable tanto a un pequeño laboratorio de pruebas como a una red extensa, permitiendo que el sistema sea escalable.

2.5. Plataformas y herramientas para el ciclo de vida de diseño y gestión de redes virtualizadas

El ciclo de vida de una infraestructura de red virtualizada abarca distintas fases críticas, desde la planificación y diseño inicial hasta el despliegue técnico y la monitorización operativa. Para estructurar este proceso de manera profesional, es común adoptar marcos de trabajo de referencia, como el modelo PPDIOO de Cisco [31], que permite desglosar las tareas de forma ordenada. A continuación, se describen las herramientas y plataformas existentes en el mercado para cubrir cada una de las etapas:

1. Etapa de Diseño y Validación de la Topología:

Esta etapa se centra en la recepción, comprobación de la integridad y procesamiento del escenario de red antes de ordenar su ejecución en el motor de contenedores. Para aportar un valor claro al flujo de trabajo, esta etapa se divide en dos subetapas complementarias:

- **Subetapa 1A: Modelado Topológico:** El diseño conceptual de una topología de red se traduce habitualmente en un modelo de datos estructurado, siendo el formato YAML el estándar predominante en los entornos de Infraestructura como Código (IaC). Este enfoque permite definir la intención de la red (nodos, enlaces y parámetros) de manera abstracta, independientemente del motor de ejecución subyacente. En la práctica, el modelado topológico requiere que el usuario gestione la creación y transferencia de estos archivos hacia el motor de emulación, un proceso que tradicionalmente implica una interacción intensiva con terminales de consola y una gestión manual de las rutas de configuración.
 - **Herramientas existentes:** **GNS3 (Graphical Network Simulator-3)** [32] y **EVE-NG (Emulated Virtual Environment Next Generation)** [33] son entornos de emulación gráfica tradicionales que permiten diseñar redes mediante un enfoque de “arrastrar y soltar” mientras que **Netlab** [34], herramienta de automatización basada en línea de comandos, utiliza un modelo de abstracción basado en YAML para definir qué protocolos (OSPF, BGP) deben configurarse.
 - **Limitaciones operativas:** Estas soluciones presentan una carga computacional elevada debido al uso de hipervisores pesados, y en el caso de las herramientas basadas en CLI, requieren un dominio avanzado de la sintaxis técnica, lo que incrementa la complejidad del flujo de trabajo y la curva de aprendizaje para usuarios no expertos.

2.5. Plataformas y herramientas para el ciclo de vida de diseño y gestión de redes virtualizadas

- **Subetapa 1B: Validación sintáctica y control de errores estáticos:** Antes de proceder al despliegue de los nodos y enlaces virtuales, es indispensable implementar una capa de control de calidad sobre el esquema de configuración. Este proceso consiste en analizar la estructura del archivo YAML para verificar que la sintaxis cumple con los requisitos del orquestador y que los parámetros definidos son correctos. Esta fase actúa como un filtro, asegurando que únicamente las configuraciones correctamente estructuradas sean procesadas por la base de datos y el motor de despliegue, evitando así fallos de ejecución.
 - **Herramientas existentes:** Netlab realiza comprobaciones de sintaxis directamente en la línea de comandos al intentar arrancar el laboratorio, notificando los fallos por consola. GNS3, al ser visual, restringe ciertos errores en el lienzo, pero requiere inicializar sus servicios para validar configuraciones avanzadas.
 - **Limitaciones operativas:** Ambas herramientas presentan una dependencia crítica del entorno de ejecución; la detección de errores no se realiza de forma estática (previa al despliegue), sino durante el arranque del laboratorio. Esta validación tardía impide un control de calidad temprano, lo que puede comprometer la estabilidad del servidor de emulación si se procesan topologías con parámetros inválidos.

2. Etapa de Despliegue y Orquestación:

Esta etapa consiste en la instanciación de los recursos de red (routers, switches o servidores virtuales) y en la configuración lógica de los enlaces de comunicación, los cuales emulan el cableado físico necesario para la interconexión entre nodos.

- **Herramientas existentes:** Dentro del paradigma de la Infraestructura como Código (IaC), **Terraform** [35] es el estándar de la industria para el despliegue automatizado de recursos (servidores, almacenamiento y base de datos) e infraestructura en la nube mediante archivos de configuración. Por otro lado, soluciones comerciales como **Cisco Modeling Labs (CML)** [36] y **EVE-NG** [33] dependen de hipervisores (KVM) para levantar máquinas virtuales y entornos de red complejos.
- **Limitaciones operativas:** La dependencia de hipervisores tradicionales conlleva una alta latencia en el despliegue y un consumo intensivo de recursos (especialmente memoria RAM). Este enfoque dificulta significativamente la escalabilidad de las topologías en estaciones de trabajo convencionales, limitando su uso a entornos de servidor con hardware dedicado.

3. Etapa de Configuración y Gestión:

Esta fase abarca el aprovisionamiento de parámetros de red y la configuración inicial de los nodos, incluyendo el esquema de direccionamiento IP, el encaminamiento y la parametrización de servicios básicos.

- **Herramientas existentes:** **Netlab** [34] destaca en esta etapa por su capacidad para automatizar la configuración de protocolos y servicios de red de forma declarativa, permitiendo definir el estado deseado de la infraestructura mediante modelos estructurados.
- **Limitaciones operativas:** La principal carencia de esta herramienta radica en la falta de un sistema de gestión de estado persistente; al no integrar una base de datos centralizada, el

sistema no mantiene un historial de despliegues ni una trazabilidad de las acciones realizadas por los distintos usuarios, lo que dificulta las tareas de auditoría y la gestión colaborativa en entornos de laboratorio.

4. Etapa de Monitorización y Operación:

Esta fase se centra en la observabilidad y el seguimiento continuo del estado de la red, permitiendo la visualización en tiempo real de la topología y el estado de los servicios desplegados.

- **Herramientas existentes:** **NextUI** [37] es una librería de JavaScript especializada en la renderización gráfica de diagramas de red interactivos. Por otro lado, **Meshery** [38] es una plataforma de gestión y visualización orientada al ecosistema de mallas de servicios (*Service Mesh*) en los clústeres empresariales de Kubernetes.
- **Limitaciones operativas:** Estas herramientas no están diseñadas para el despliegue de topologías de red virtualizada. Mientras que **NextUI** requiere un desarrollo a medida desde cero para implementar la lógica de red, **Meshery** presenta una complejidad técnica elevada y está sobredimensionada para escenarios de emulación de redes tradicionales, dificultando su adopción en entornos de experimentación rápida.

A partir del análisis descrito, se evidencia que, si bien existen herramientas capaces de cubrir fases individuales del ciclo de vida de una red virtualizada, carecen de una integración que simplifique la gestión integral para entornos de experimentación técnica. Actualmente, existe un vacío en la disponibilidad de plataformas que permitan al usuario cargar, almacenar y gestionar sus topologías de manera centralizada, realizar una validación estática previa al despliegue y automatizar la ejecución sobre motores ligeros de contenedores. El presente proyecto surge como respuesta a esta necesidad, proponiendo un sistema que centraliza estas etapas bajo una única interfaz web, optimizando tanto el flujo de trabajo del usuario como el consumo de recursos computacionales.

Capítulo 3

Metodología y Planificación

3.1. Metodología

Para la ejecución de este proyecto, se ha diseñado un marco de trabajo que combina los principios de metodologías ágiles con técnicas de planificación de proyectos de alto rendimiento. El enfoque principal es un diseño iterativo e incremental basado en la obtención de hitos funcionales que aseguran el progreso en la aplicación.

3.1.1. Fundamentos del diseño iterativo e incremental

El diseño iterativo e incremental es una metodología de desarrollo software o enfoque de planificación que se basa en dividir la evolución del proyecto en ciclos repetitivos llamados iteraciones. Sobre cada iteración se añade nueva funcionalidad o se mejora la ya existente. El objetivo de este tipo de enfoque es reducir la complejidad, hacer que el proceso de desarrollo sea más flexible frente a posibles cambios en requisitos o funcionalidades y obtener entregables funcionales desde las primeras etapas del desarrollo, a diferencia de lo que sucede en el enfoque de cascada, garantizando así que el proyecto cuente con una base que gana robustez y capacidades a medida que progresa el desarrollo y evitando que se llegue a las últimas etapas del mismo sin un producto tangible [39],[40].

Cada ciclo del diseño iterativo se compone de las fases de un ciclo de vida clásico: análisis, diseño, implementación, pruebas y validación. Una característica fundamental de este tipo de enfoque son los bucles de retroalimentación constantes, de manera que, tras la finalización de una iteración se analizan los resultados obtenidos para ajustar los objetivos del siguiente ciclo, permitiendo que el aprendizaje técnico adquirido se incorpore de forma inmediata al desarrollo.

3.1.2. Organización de tareas con Kanban

Kanban es un conjunto de prácticas y principios que ayuda a gestionar y mejorar el flujo de trabajo en el desarrollo de un producto [41]. Se basa en una filosofía de mejora continua donde las tareas se seleccionan de una lista de tareas pendientes (*backlog*) en función de la capacidad, manteniendo un flujo constante que evita la sobrecarga. Se implementa mediante unos tableros organizados por columnas, cada una de ellas representando una etapa del trabajo: *Por Hacer*, *En Progreso*, *Completado*.

Un elemento diferenciador de este enfoque es la limitación del *WIP* (*Work In Progress*), limitando la cantidad de tareas que se pueden realizar de manera simultánea para mantener un ritmo constante de trabajo y detectar posibles cuellos de botella sin llegar a sobrecargar al equipo. El funcionamiento del tablero consiste en un flujo dinámico donde las tareas se desplazan entre las distintas columnas dependiendo del estado en el que estas se encuentren. Al utilizar un sistema de arrastre, el inicio de las nuevas tareas no se produce hasta que las anteriores se han completado o liberado.

Este sistema de gestión destaca por aportar una transparencia total sobre el volumen de trabajo pendiente y completado. Una característica es la versatilidad, ya que a diferencia de otros marcos más rígidos como SCRUM que dependen de ciclos temporales cerrados (Sprints), permite reajustar prioridades en tiempo real y es fundamental en proyectos donde la curva de aprendizaje o la resolución de problemas técnicos requieran una distribución de los esfuerzos sin esperar a la finalización de un ciclo preestablecido.

3.1.3. Integración de Kanban en Notion

Notion es una herramienta de productividad y gestión de la información que permite crear espacios de trabajo personalizados. Una de las características por las que se ha seleccionado es su capacidad para integrar bases de datos relacionales facilitando la implementación de un tablero Kanban digital, flexible y adaptado a las necesidades de la automatización de las redes.

La integración de Kanban en Notion permite que cada tarea sea más que una simple etiqueta. Cada elemento de este tablero consiste en una página independiente que contiene información útil para su desarrollo.

Las tareas se desplazan de forma visual entre las distintas columnas definidas (Por Hacer, En Progreso, Completado) cumpliendo con el sistema de arrastre y limitación del trabajo en curso. Dentro de las mismas, se adjuntan bloques de código, notas de configuración, capturas de la topología de la red, asegurando que el conocimiento obtenido en cada iteración no se pierda en las siguientes.

Además, mediante el uso de etiquetas de metadatos, se asigna a cada tarea un nivel de prioridad, lo que permite decidir qué funcionalidades son críticas para el producto mínimo viable (MVP) y cuáles van a ser secundarias.

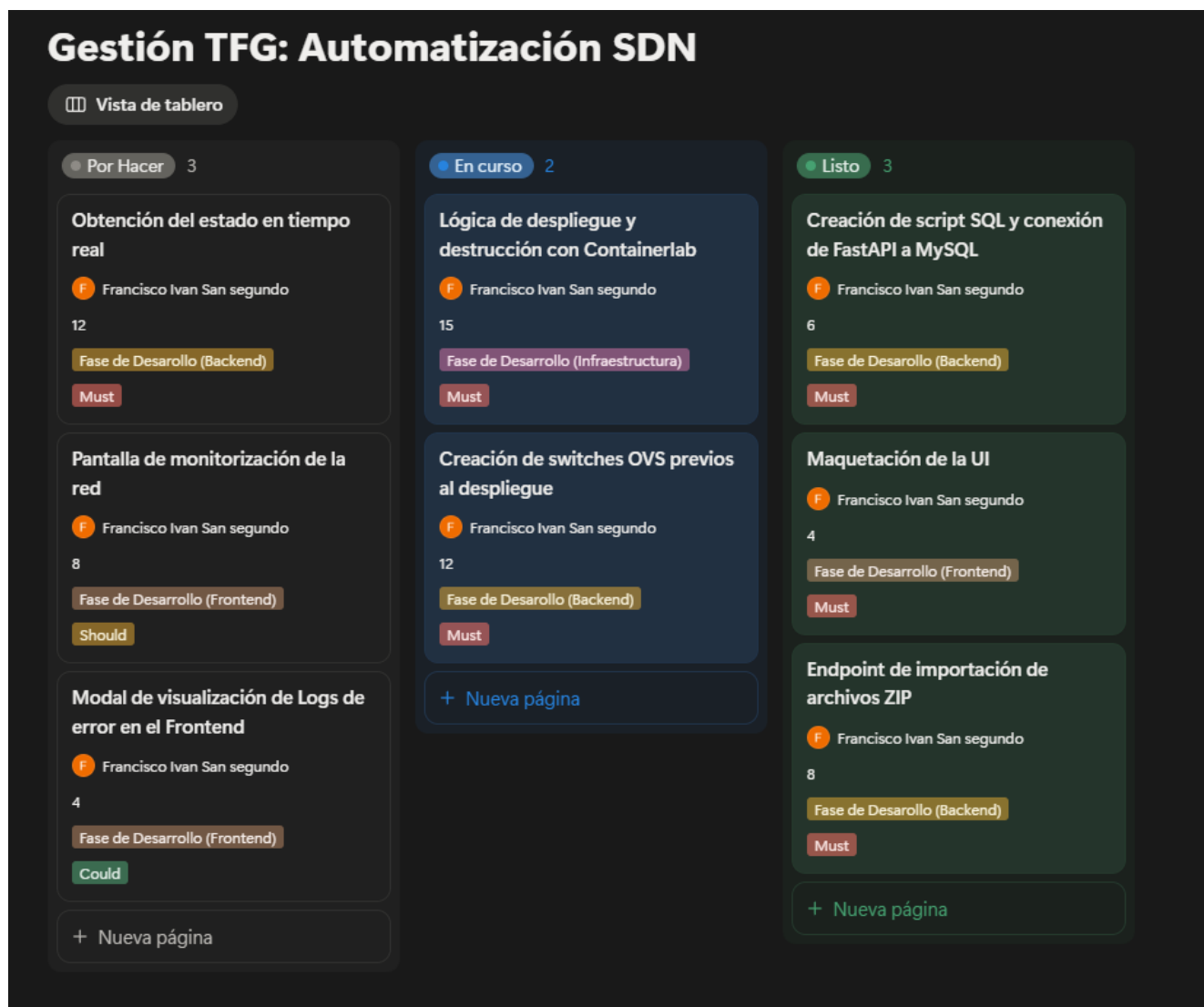


Figura 3.1: Tablero Kanban empleado durante el desarrollo del proyecto.

3.1.4. Priorización mediante la técnica MoSCoW

Para que el diseño iterativo funcione hay que decidir qué tareas son prioritarias en cada momento. Para ello, se ha utilizado la técnica MoSCoW, que permite clasificar los requisitos en cuatro niveles de importancia asegurando que, en caso de falta de tiempo o problemas técnicos, las funciones más críticas de la red estén terminadas primero.

Los niveles en los que se dividen las tareas son los siguientes:

Must Have (Debe tener): Son tareas obligatorias. Sin ellas, el proyecto no funciona o no cumple con los requisitos y objetivos básicos (ej. conectividad de red).

Should Have (Debería tener): Tareas que son importantes, pero no llegan a ser vitales para el proyecto. Se realizarán cuando las críticas se hayan completado (ej. documentación automática del inventario de la red).

Could Have (Podría tener): Funcionalidad que podría incluirse en caso de disponer de tiempo para ello (ej. interfaz gráfica para el script).

Won't Have (No tendrá): Tareas que se dejan fuera de esta etapa del proyecto pero que se plantean para posibles líneas de trabajo futuras (ej. alta disponibilidad con protocolos complejos) [42].

Esta técnica de priorización de tareas nos permite evitar la tendencia de agregar características no solicitadas o innecesarias a menudo a costa de funcionalidad real (*Gold-Plating*) [43].

3.1.5. Justificación y aplicación del enfoque

La elección de un diseño iterativo e incremental, apoyado en Kanban y MoSCoW, responde a la naturaleza del proyecto. La automatización de las redes implica una curva de aprendizaje importante y la integración de diferentes tecnologías que pueden presentar imprevistos. Al trabajar por ciclos, nos aseguramos de que la infraestructura de red sea sólida desde el principio, permitiendo incorporar funcionalidades de manera progresiva sin comprometer lo que ya funciona.

En la práctica, este enfoque se aplica dividiendo el desarrollo en etapas lógicas. En una primera iteración, el objetivo principal es establecer la conectividad básica y al acceso a dispositivos. Una vez que se ha validado este incremento, se desarrollan los scripts de configuración de protocolos. Este sistema de avance por pasos nos permite detectar errores en fases tempranas evitando que un fallo en la lógica de la automatización afecte a todo el despliegue en la fase final del proyecto.

Por último, el uso del tablero Kanban en Notion hace que la planificación sea flexible. La utilización de la técnica MoSCoW ayuda a decidir si es necesario dedicarle más tiempo por ser crítica a una tarea de configuración que se ha vuelto más compleja de lo previsto o si puede posponerse en favor de objetivos vitales. En conclusión, esta metodología garantiza que, ante cualquier dificultad o limitación de tiempo, se entregue una solución funcional que cumpla con los requisitos esenciales del TFG que se han planeado inicialmente.

3.2. Planificación inicial

3.2.1. Fase inicial

En esta sección, se presenta la planificación temporal del proyecto, estructurada mediante una metodología iterativa e incremental, organizada en seis fases principales que abarcan desde la definición de requisitos hasta la defensa del TFG. La duración total estimada es de 302 horas de trabajo.

Iteraciones del desarrollo del proyecto

Como se ha mencionado anteriormente, este proyecto se ha estructurado en seis iteraciones diferenciadas, diseñadas bajo una metodología iterativa e incremental. Cada una de estas etapas está orientada a la obtención de un incremento funcional tangible.

Este enfoque permite una mejora continua del sistema, donde la retroalimentación obtenida tras cada fase de evaluación es incorporada en el siguiente ciclo. Al inicio de cada etapa, se realiza una reunión con el tutor para revisar los avances previos y validar los resultados obtenidos.

Para ilustrar visualmente la temporalización y la distribución de la carga de trabajo estimada, a continuación se presenta el cronograma global del proyecto. Este diagrama detalla la secuencia de las seis iteraciones evidenciando el enfoque iterativo e incremental adoptado y marcando los hitos principales que componen las 302 horas de desarrolladas planificadas.

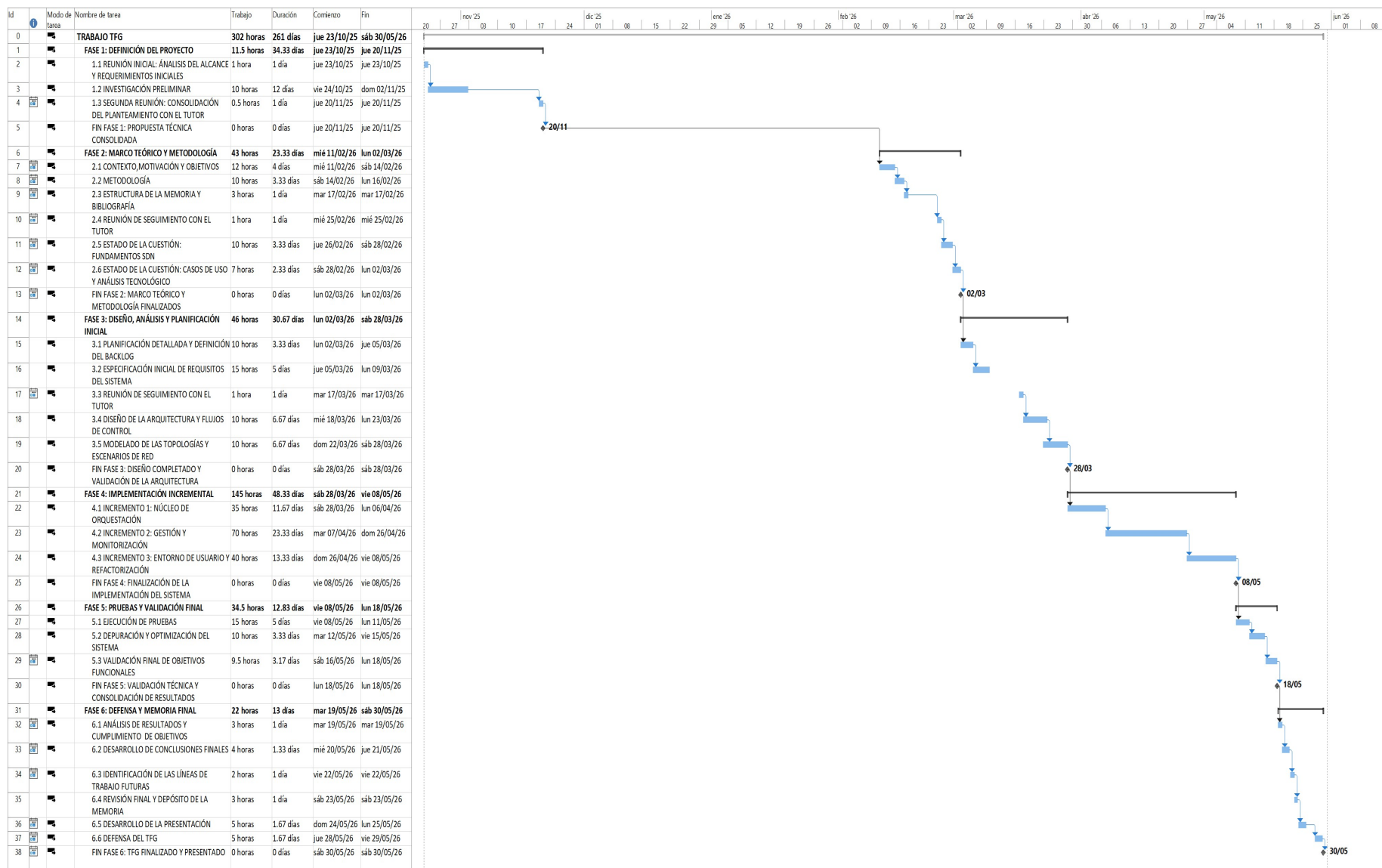


Figura 3.2: Diagrama de Gantt con la planificación global del proyecto

Primera iteración (23/10/2025 - 20/11/2025)

La primera iteración se denomina **Definición del proyecto**, su propósito es establecer las bases del proyecto mediante la definición del alcance, la identificación de los requerimientos iniciales y la consolidación del planteamiento técnico. Para ello, se parte de una reunión inicial con el tutor en la que se van a delimitar los objetivos generales del proyecto, seguida de una fase de investigación preliminar en la que se estudia el ámbito del proyecto (tecnologías candidatas, aproximaciones de implementación disponibles). Finalmente, una segunda reunión con el tutor permite validar las conclusiones obtenidas y acordar el enfoque del desarrollo.

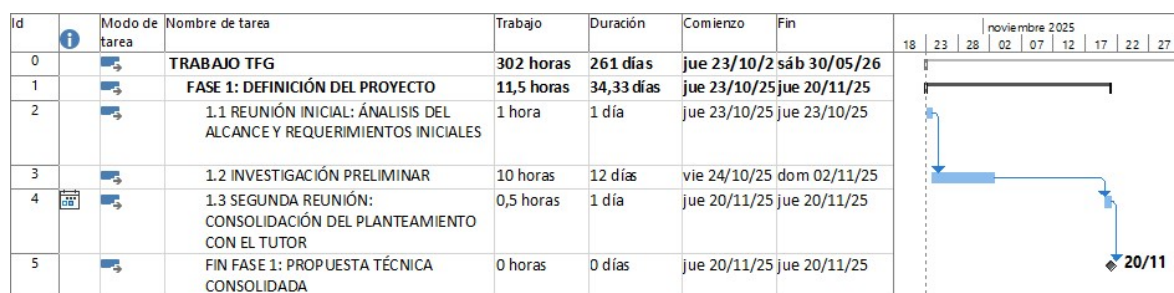


Figura 3.3: Diagrama de Gantt de la primera iteración

Segunda iteración (11/02/2026 - 02/03/2026)

La segunda iteración se denomina **Marco teórico y metodología**, su propósito es redactar el marco teórico y metodológico de la memoria, profundizando en el contexto, fundamentos técnicos y las soluciones existentes en el ámbito del proyecto. Para ello, se parte de la redacción del contexto, la motivación y los objetivos del TFG, seguida de la elaboración de la metodología y la estructura de la memoria. Después, se redacta el estado de la cuestión, abordando tanto los fundamentos técnicos de SDN como sus casos de uso en entornos reales y el análisis tecnológico. Una reunión con el tutor permite validar el avance y orientar las tareas restantes.

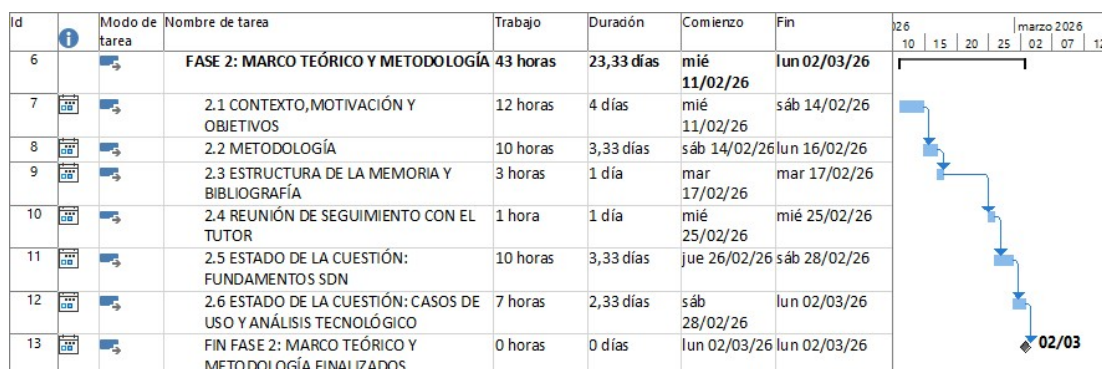


Figura 3.4: Diagrama de Gantt de la segunda iteración

Tercera iteración (02/03/2026 - 28/03/2026)

La tercera iteración se denomina **Diseño, análisis y planificación inicial**, su propósito es redactar el diseño, análisis y la planificación inicial del sistema, definiendo con detalle los requisitos, la arquitectura y los escenarios de red sobre los que se va a sustentar posteriormente el sistema. Para ello, se parte de una planificación detallada y la definición de las tareas del proyecto, seguida de la especificación de requisitos del sistema. Posteriormente, se redactan el diseño de la arquitectura y los flujos de control, así como el modelado de las topologías y los escenarios de red contemplados. Una reunión de seguimiento con el tutor permite validar el diseño propuesto antes de comenzar la fase de implementación.

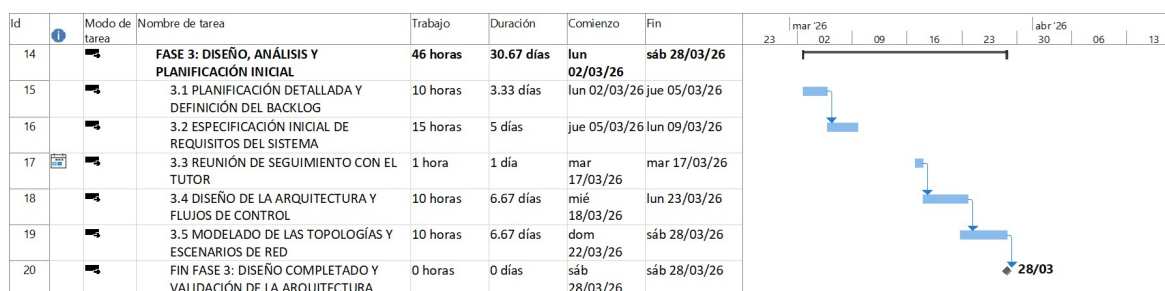


Figura 3.5: Diagrama de Gantt de la tercera iteración

Cuarta iteración (28/03/2026 - 08/05/2026)

La cuarta iteración se denomina **Implementación incremental**, su propósito es llevar a cabo la implementación incremental del sistema, desarrollando de forma progresiva las distintas capas y funcionalidades que lo componen. Para ello, el trabajo se organiza en tres incrementos: en primer lugar, se

implementa el núcleo de orquestación del sistema, sobre el que posteriormente se construye la capa de gestión y monitorización. Finalmente, se desarrolla el entorno de usuario y se realiza las tareas de refactorización necesarias para consolidar el conjunto del sistema.

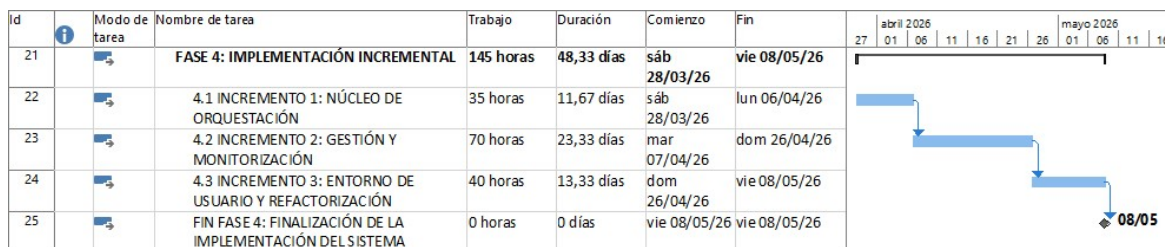


Figura 3.6: Diagrama de Gantt de la cuarta iteración

Quinta iteración (08/05/2026 - 18/05/2026)

La quinta iteración se denomina **Pruebas y validación final**, su propósito es verificar el correcto funcionamiento del sistema mediante la ejecución de pruebas de caja negra, depuración de los problemas encontrados y la validación final frente a los objetivos definidos al inicio del proyecto. Para ello, se parte de la ejecución de pruebas en la red, seguida de una fase de depuración y optimización del sistema. Finalmente, se lleva a cabo la validación final contra los objetivos funcionales definidos, comprobando que el producto desarrollado cumple con las funcionalidades especificadas.

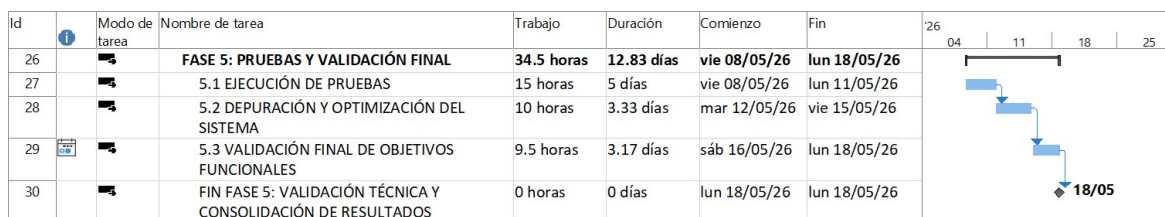


Figura 3.7: Diagrama de Gantt de la quinta iteración

Sexta iteración (19/05/2026 - 30/05/2026)

La sexta iteración se denomina **Defensa y memoria final**, su propósito es cerrar el proyecto mediante la redacción de las conclusiones, la preparación de la defensa y el depósito de la memoria. Para ello, se parte del análisis de los resultados obtenidos y la verificación del cumplimiento de los objetivos planteados inicialmente, seguido del desarrollo de las conclusiones finales y la identificación de las líneas de trabajo futuras. A continuación, se lleva a cabo la revisión final y el depósito de la memoria, el

desarrollo de la presentación y finalmente, la defensa del TFG.

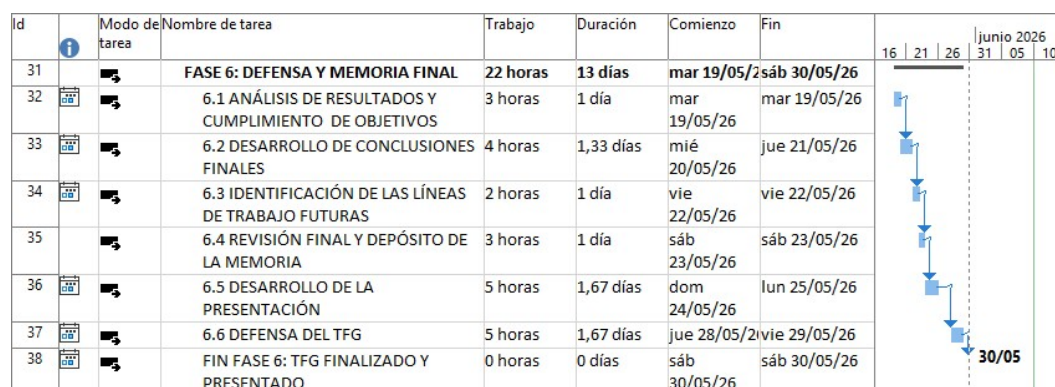


Figura 3.8: Diagrama de Gantt de la sexta iteración

3.2.2. Análisis de riesgos

La gestión de riesgos constituye un pilar estratégico en la planificación de un proyecto de software, cuyo propósito es transformar la incertidumbre en factores controlables. De acuerdo con el estándar PMBOK [44], un riesgo se define como un evento o condición incierta que, de producirse, tiene un efecto positivo o negativo en uno o más objetivos del proyecto como el alcance, el cronograma, el coste y la calidad.

Mientras que los riesgos negativos (amenazas) pueden comprometer la calidad, el presupuesto o el cronograma, los riesgos positivos (oportunidades) representan beneficios puntuales que una planificación proactiva debería potenciar. Es fundamental distinguir que esta disciplina se enfoca exclusivamente en eventos futuros posibles debido a que los problemas actuales se gestionan como incidencias y no forman parte del plan de riesgos inicial.

Una correcta planificación de los riesgos se divide en las siguientes fases o etapas:

- **Identificación de los riesgos:** Registro de todas las amenazas y oportunidades que podrían afectar al entorno de software (técnicas, externas, organizativas, etc).
- **Análisis cuantitativo y cualitativo de los riesgos:** Evaluación de la probabilidad de ocurrencia y el grado de impacto sobre el proyecto.
- **Planificación de los riesgos:** Definición de estrategias específicas (mitigar, evitar, transferir o aceptar).
- **Monitorización de los riesgos:** Seguimiento continuo para detectar la aparición de nuevos riesgos durante el desarrollo del proyecto.

Para cada riesgo identificado, se van a considerar cuatro elementos esenciales que permiten un análisis más profundo:

- **Probabilidad:** Hace referencia a la posibilidad de que un riesgo ocurra. Determinar la probabilidad de cada riesgo ayuda a priorizarlos, ya que aquellos con mayor probabilidad deben ser gestionados de manera más urgente.
- **Impacto:** Hace referencia a las consecuencias o efectos que puede tener un riesgo sobre el proyecto en caso de que llegase a ocurrir. De la misma manera que la probabilidad, conocer el impacto de los riesgos es primordial para priorizarlos. Un riesgo con alto impacto debe recibir atención aunque su probabilidad sea baja.
- **Plan de mitigación:** Comprende las acciones preventivas que se van a implementar para reducir la probabilidad de que el riesgo se dé o disminuir el impacto en caso de que ocurra. Siempre se implementan antes de que el riesgo suceda, anticipándose al problema.
- **Plan de contingencia:** Comprende las acciones correctivas para gestionar el riesgo una vez que se ha producido. Estas acciones buscan reducir los efectos del riesgo sobre el proyecto a diferencia de la mitigación que busca evitar el riesgo.

Existen principalmente dos técnicas a la hora de identificar los riesgos: las *checklists* y el *brainstorming*.

- **Checklists:** Recogen la experiencia de proyectos pasados e información de algunos riesgos recurrentes en proyectos de desarrollo de software.
- **Brainstorming:** Recogen las ideas y perspectivas del equipo de trabajo para identificar una amplia variedad de riesgos potenciales a través de la creatividad grupal.

3.2.3. Metodología de análisis y priorización

Una vez identificados los riesgos potenciales, es necesario evaluar cuáles tienen una mayor relevancia para el éxito del proyecto. Este proceso de jerarquización permite optimizar el uso de los recursos, enfocándolos en las amenazas que presentan una mayor criticidad.

Para llevar a cabo esta tarea, se utiliza la técnica de Exposición al Riesgo o Risk Exposure (RE) [45]. Este método consiste en asignar un valor numérico que cuantifica el impacto potencial del riesgo sobre el desarrollo del proyecto. El cálculo se realiza mediante la siguiente fórmula:

$$RE = probabilidad \times impacto$$

A cada una de las variables mencionadas en la fórmula se le asigna un valor numérico en una escala de 1 a 10.

Esta jerarquía se ve reflejada en la Tabla 3.1, donde los riesgos del proyecto se encuentran ordenados de mayor a menor criticidad.

Riesgo	Probabilidad	Impacto	Prioridad (RE)
1 Errores al estimar las duraciones de las tareas	8	7	56
2 Cambios en los requisitos durante el proyecto	7	8	56
3 Curva de aprendizaje tecnológica excesiva	6	8	48
4 Baja disponibilidad por otros compromisos	6	6	36
5 Gold-Plating	7	4	28
6 Inestabilidad en el entorno de desarrollo	4	7	28
7 Baja por enfermedad del desarrollador o familiares	3	9	27
8 Parada por análisis	4	5	20
9 Pérdida de información	2	10	20
10 Documentación técnica insuficiente	3	4	12

Tabla 3.1: Identificación de los principales riesgos del proyecto

En función de la probabilidad e impacto de los riesgos, se llevarán a cabo acciones preventivas o correctivas específicas. La prioridad de cada riesgo se determina mediante el cruce de ambos factores en la matriz de decisiones (ver Tabla 3.2).

Aunque se calcula el valor de Exposición al Riesgo ($RE = P \times I$) para permitir una ordenación numérica detallada en la fase de planificación, la categorización final (Bajo, Medio, Alto) se rige estrictamente por la zona de color resultante en dicha matriz. Esto garantiza, por ejemplo, que eventos con un impacto crítico sean gestionados con planes de mitigación aunque su probabilidad sea baja, independientemente de que su valor numérico de RE sea menor que otros riesgos de carácter moderado.

Prob. / Imp.	Bajo (1-3)	Medio (4-7)	Alto (8-10)
Bajo (1-3)	Monitorizar	Monitorizar	Aplicar mitigación
Medio (4-7)	Monitorizar	Aplicar mitigación	Mitigar y contingencia
Alto (8-10)	Aplicar mitigación	Mitigar y contingencia	Mitigar y contingencia

Tabla 3.2: Matriz de decisiones según Probabilidad e Impacto.

Una vez priorizados los riesgos de manera global, se procede al análisis detallado de cada uno de ellos. A continuación, se muestran las tablas individuales de los riesgos identificados durante la fase de planificación donde se especifican sus causas, el impacto concreto y las estrategias de respuesta diseñadas.

ID	RSK001
Nombre	Errores al estimar las duraciones de las tareas
Descripción	Desviación negativa entre el tiempo planeado y el tiempo real ejecutado.
Vulnerabilidad	Incumplimiento del cronograma y retraso en entregas.
Amenaza	Agotamiento de los márgenes de tiempo y reducción de la calidad.
Probabilidad	Alta (8)
Impacto	Medio (7)
Riesgo Total (RE)	Alto (56)
Acciones de mitigación	■ Desglosar las tareas en unidades de máximo 4 horas. ■ Añadir un factor de holgura del 15 % a cada tarea crítica. ■ Revisar el progreso semanalmente frente al plan original.
Acciones correctivas	■ Reequilibrar el esfuerzo eliminando requisitos no esenciales. ■ Intensificar las horas de trabajo en periodos de recuperación. ■ Ejecutar las tareas en paralelo si las dependencias técnicas lo permiten.

Tabla 3.3: Riesgo RSK001: Errores al estimar las duraciones de las tareas.

ID	RSK002
Nombre	Cambios en los requisitos durante el proyecto
Descripción	Modificaciones o adiciones al alcance inicialmente definido para el desarrollo del software.
Vulnerabilidad	Incremento de la complejidad técnica y retrasos en la fecha de finalización
Amenaza	Degradación del diseño técnico y posible abandono de funcionalidades críticas.
Probabilidad	Media (7)
Impacto	Alto (8)
Riesgo Total (RE)	Alto (56)
Acciones de mitigación	■ Definir y validar formalmente los requisitos iniciales con el tutor. ■ Detener el alcance una vez iniciada la fase de implementación. ■ Priorizar requisitos mediante el método MoSCoW.
Acciones correctivas	■ Analizar el impacto técnico y temporal de la nueva funcionalidad introducida. ■ Negociar el intercambio de requisitos (sustituir uno nuevo por uno previo). ■ Refactorizar componentes críticos para permitir cambios mínimos viables.

Tabla 3.4: Riesgo RSK002: Cambios en los requisitos durante el proyecto.

ID	RSK003
Nombre	Curva de aprendizaje tecnológica excesiva
Descripción	Dificultad mayor de la esperada para dominar las herramientas, lenguajes o frameworks seleccionados.
Vulnerabilidad	Bloqueo en el desarrollo y baja calidad en la implementación del código inicial.
Amenaza	Incapacidad de completar funcionalidades complejas y aumento de la deuda técnica.
Probabilidad	Media (6)
Impacto	Alto (8)
Riesgo Total (RE)	Alto (48)
Acciones de mitigación	■ Planificar una fase inicial exclusiva de formación y configuración del entorno. ■ Consultar documentación oficial y repositorios de referencia de código abierto. ■ Realizar prototipos rápidos para validar tecnologías rápidas.
Acciones correctivas	■ Simplificar la implementación buscando soluciones más estándar. ■ Solicitar apoyo puntual en foros especializados o comunidades de desarrolladores. ■ Derivar a bibliotecas con una curva de aprendizaje menor si el bloqueo persiste.

Tabla 3.5: Riesgo RSK003: Curva de aprendizaje tecnológica excesiva.

ID	RSK004
Nombre	Baja disponibilidad por otros compromisos
Descripción	Reducción del tiempo efectivo dedicado al desarrollo del proyecto debido a factores externos (trabajo, otras asignaturas).
Vulnerabilidad	Retraso en el cronograma y pérdida de ritmo en el desarrollo.
Amenaza	Incumplimiento de las fechas de entrega de los hitos y reducción del tiempo para pruebas y documentación.
Probabilidad	Media (6)
Impacto	Medio (6)
Riesgo Total (RE)	Medio (36)
Acciones de mitigación	■ Planificar el cronograma con periodos de baja intensidad coincidiendo con entregas de otras asignaturas. ■ Establecer objetivos semanales realistas basados en la carga de trabajo externa prevista. ■ Adelantar tareas críticas en periodos de alta disponibilidad (vacaciones, fines de semana).
Acciones correctivas	■ Aplicar técnicas de concentración para maximizar el rendimiento en horas disponibles. ■ Reajustar la planificación eliminando tareas de baja prioridad para compensar el tiempo perdido. ■ Intensificar la dedicación en las semanas posteriores al periodo de baja disponibilidad.

Tabla 3.6: Riesgo RSK004: Baja disponibilidad por otros compromisos.

ID	RSK005
Nombre	Gold-Plating
Descripción	Adición de funcionalidades o mejoras no previstas en el alcance inicial con el objetivo de perfeccionar el producto.
Vulnerabilidad	Consumo innecesario de recursos y tiempo en tareas que no aportan un valor crítico al proyecto.
Amenaza	Retraso en las funcionalidades principales y posible introducción de nuevos errores en el sistema.
Probabilidad	Media (7)
Impacto	Medio (4)
Riesgo Total (RE)	Medio (28)
Acciones de mitigación	■ Ceñirse estrictamente a las secciones de objetivos y requisitos del TFG. ■ Definir un criterio de “Finalizado” claro para cada tarea. ■ Definir las ideas de mejora en una sección de “Trabajos futuros” en lugar de implementarlas.
Acciones correctivas	■ Detener el desarrollo de cualquier funcionalidad que no sea esencial para el MVP. ■ Eliminar código o módulos no críticos que comprometan el cronograma. ■ Priorizar la finalización de la memoria técnica frente a los detalles estéticos del software.

Tabla 3.7: Riesgo RSK005: Gold-Plating.

ID	RSK006
Nombre	Inestabilidad en el entorno de desarrollo
Descripción	Problemas derivados de la configuración del software base, conflictos entre versiones de dependencias o averías físicas imprevistas en el hardware de trabajo (ej: rotura del ordenador de desarrollo).
Vulnerabilidad	Imposibilidad de ejecutar, compilar o acceder al código del proyecto correctamente, causando paradas totales en la producción.
Amenaza	Corrupción de la configuración local, pérdida de acceso a los entornos de emulación, averías en los componentes físicos y pérdida de tiempo en tareas de soporte o reparación.
Probabilidad	Baja (2)
Impacto	Medio (7)
Riesgo Total (RE)	Bajo (14)
Acciones de mitigación	■ Utilizar sistemas de gestión de versiones de lenguaje (como nvm, pyenv o rbenv). ■ Documentar los pasos de instalación y configuración en un archivo <i>README</i>. ■ Mantener copias de seguridad de los archivos de configuración (<i>.env</i>, <i>config</i>). ■ Alojarse el repositorio de código de forma remota y actualizada en plataformas en la nube (como GitHub o GitLab). ■ Sincronizar la documentación y la memoria del TFG en sistemas de almacenamiento en la nube.
Acciones correctivas	■ Restaurar el entorno a un estado estable previo mediante el control de versiones. ■ Reinstalar dependencias desde cero eliminando memorias caché y archivos temporales. ■ Evaluar el uso de contenedores (Docker) para aislar y estandarizar el entorno. ■ Clonar el repositorio remoto en un equipo secundario o de sustitución para reanudar el desarrollo inmediatamente en caso de fallo físico del hardware.

Tabla 3.8: Riesgo RSK006: Inestabilidad en el entorno de desarrollo.

ID	RSK007
Nombre	Baja por enfermedad del desarrollador o familiares
Descripción	Indisponibilidad temporal por motivos de salud propios o necesidad de cuidados a familiares directos.
Vulnerabilidad	Parada total de las actividades de desarrollo, pruebas y redacción.
Amenaza	Incumplimiento de la fecha de depósito y necesidad de solicitar una prórroga.
Probabilidad	Baja (3)
Impacto	Alto (9)
Riesgo Total (RE)	Medio (27)
Acciones de mitigación	■ Mantener un margen de seguridad de dos semanas en el cronograma final. ■ Documentar y subir el progreso diariamente al repositorio para facilitar una posible reanudación. ■ Finalizar el MVP lo antes posible para asegurar una base entregable en caso de cualquier imprevisto.
Acciones correctivas	■ Reducir el alcance del proyecto eliminando objetos secundarios para cumplir con el mínimo exigible. ■ Solicitar una ampliación del plazo o cambio de la convocatoria si la baja es prolongada. ■ Notificar a la coordinación del TFG aportando los justificantes médicos pertinentes.

Tabla 3.9: Riesgo RSK007: Baja por enfermedad del desarrollador o familiares.

ID	RSK008
Nombre	Parada por análisis
Descripción	Exceso de tiempo invertido en la fase de diseño o elección de alternativas, deteniendo la fase de implementación.
Vulnerabilidad	Estancamiento en fases iniciales y reducción del tiempo para pruebas y desarrollo.
Amenaza	Incapacidad de producir un producto mínimamente funcional dentro de los plazos establecidos.
Probabilidad	Media (4)
Impacto	Medio (5)
Riesgo Total (RE)	Medio (20)
Acciones de mitigación	■ Acotar temporalmente las fases de investigación y diseño de cada componente. ■ Adoptar un enfoque iterativo e incremental (focalizarse en que funcione antes de optimizar). ■ Consultar con el tutor ante dudas persistentes entre dos alternativas técnicas.
Acciones correctivas	■ Adoptar la solución más sencilla o estándar para debloquear el progreso inmediato. ■ Priorizar el desarrollo del núcleo del sistema (<i>Core</i>) posponiendo decisiones de diseño menores. ■ Establecer un plazo máximo de 24 horas para tomar cualquier decisión técnica pendiente.

Tabla 3.10: Riesgo RSK008: Parada por análisis.

ID	RSK009
Nombre	Pérdida de información
Descripción	Eliminación accidental, corrupción de archivos de configuración o fallo físico en las unidades de almacenamiento local que resulte en la pérdida del código o la memoria.
Vulnerabilidad	Necesidad de reescribir partes del proyecto, lo que implica un retraso crítico o insalvable.
Amenaza	Retraso crítico o imposibilidad de entregar el TFG en los plazos por pérdida de los activos digitales esenciales del sistema.
Probabilidad	Bajo (2)
Impacto	Alto (10)
Riesgo Total (RE)	Medio (20)
Acciones de mitigación	■ Uso obligatorio de control de versiones con Git y sincronización diaria con un repositorio remoto (<i>push</i> a GitLab). ■ Empleo de almacenamiento en la nube o discos externos para el respaldo continuo de todos los artefactos y activos digitales del proyecto. ■ Configuración de copias de seguridad automáticas en la nube (Google Drive, OneDrive o iCloud) para la documentación.
Acciones correctivas	■ Restaurar la versión estable más reciente desde el repositorio remoto. ■ Recuperar archivos de la memoria mediante el historial de versiones de las herramientas de edición en la nube. ■ Utilizar herramientas de recuperación de datos en caso de fallo físico de la unidad de almacenamiento local.

Tabla 3.11: Riesgo RSK009: Pérdida de información.

ID	RSK010
Nombre	Documentación técnica insuficiente
Descripción	Falta de registro detallado sobre el diseño, la arquitectura, las decisiones técnicas y el uso del sistema.
Vulnerabilidad	Dificultad para redactar la memoria final del TFG y problemas en el mantenimiento futuro del código.
Amenaza	Incoherencia entre el software desarrollado y la documentación entregada, y retrasos en la entrega final.
Probabilidad	Baja (3)
Impacto	Medio (4)
Riesgo Total (RE)	Bajo (12)
Acciones de mitigación	■ Adoptar una metodología “<i>Documentation as Code</i>” integrando comentarios en el código. ■ Reservar una sesión semanal dedicada exclusivamente a la actualización de la memoria técnica. ■ Generar diagramas de arquitectura y flujo en paralelo al desarrollo de los módulos.
Acciones correctivas	■ Realizar una auditoría del código para extraer y documentar la lógica implementada. ■ Utilizar herramientas de generación automática de documentación a partir de comentarios (JSDoc, Swagger, etc.). ■ Priorizar la redacción de los capítulos críticos de la memoria sobre nuevas funcionalidades.

Tabla 3.12: Riesgo RSK010: Documentación técnica insuficiente.

3.2.4. Estimación de costes

El análisis económico de este proyecto comprende la identificación y cuantificación de los recursos necesarios para su ejecución. Para ello, se han categorizado los costes en cuatro bloques principales: mano de obra, amortización de hardware, licencias de software y costes indirectos. Es fundamental recalcar que, dado el carácter académico de este Trabajo Final de Grado, las cifras presentadas constituyen una estimación teórica orientada a evaluar la viabilidad económica del proyecto en un entorno profesional real, y no implican un gasto real efectuado.

- El equipo de trabajo para el desarrollo de este proyecto está compuesto por un único integrante que asume los roles de arquitecto de red, desarrollador *full-stack* y administrador de sistemas. En España, a día 7 de mayo de 2026, el sueldo medio de un ingeniero DevOps con un perfil junior se sitúa en los 2.416,67 € brutos al mes (correspondiente a una media anual de 29.000 €) según el portal de búsqueda de empleo Glassdoor.es [46]. El desglose de este salario bruto, enfocado en las retenciones de IRPF y cuotas del trabajador, se puede observar en la Tabla 3.13

Concepto	Importe
Sueldo neto mensual (12 pagas)	1.832,32 €
Sueldo neto anual	21.987,84 €
Tipo de retención sobre la nómina (estimado)	17,83 %
Cuotas a la Seg. Social	1.841,50 €
Retenciones por IRPF (Anual)	5.170,70 €
Sueldo bruto anual	29.000,00 €

Tabla 3.13: Cálculo de retenciones y salario

Para calcular el coste del desarrollador durante la ejecución del proyecto, se parte de una jornada laboral de 8 horas diarias y un total de 253 días laborales según establece la página Workingdays [47], lo que supone un cómputo de 2.024 horas de trabajo al año. Tomando como referencia el sueldo bruto anual de 29.000 € detallado anteriormente, y añadiendo un coste aproximado del 33 % en concepto de Seguridad Social a cargo de la empresa, el coste anual de la entidad asciende a 38.570,00 €. Esto establece un coste por hora de trabajo de 19,13 €. Dado que el desarrollo del proyecto está estimado en 300 horas, lo que equivale a dos meses de dedicación a tiempo completo, el coste total de mano de obra para este TFG queda fijado en 5.739,00 €

- En cuanto al hardware empleado, el desarrollo principal se ha realizado en una estación de trabajo de sobremesa (torre y monitores) con un valor de 1.700,00 €. Considerando una vida útil de 5 años (60 meses) y un periodo de utilización de 5 meses (desde el inicio del proyecto en febrero de 2026 hasta su finalización estimada), el coste amortizado es de 141,67 €.

Asimismo, para la etapa de pruebas y la realización de la demostración técnica, se ha utilizado un ordenador portátil valorado en 500,00 €. Teniendo en cuenta la amortización correspondiente a un mes de uso para estas tareas finales, el coste asciende a 8,33 €. Sumando ambos conceptos, el coste total del hardware es de 150,00 €.

- En lo respectivo al software, para la realización de este proyecto se han empleado exclusivamente herramientas bajo licencias académicas o de uso gratuito (*Open Source*).

Para la elaboración de la documentación técnica y los diagramas de arquitectura se han utilizado plataformas web de diseño como draw.io, que permiten la creación de diagramas sin necesidad de un licencia.

La gestión de tutorías y comunicación se ha realizado mediante Microsoft Teams bajo la licencia institucional proporcionada por la Universidad, y la redacción de la memoria se ha realizado en la plataforma Overleaf. Dado que todas las herramientas utilizadas operan bajo modalidades gratuitas o licencias académicas, el coste total asignado al software se establece en 0,00 €.

- Para el correcto desarrollo del proyecto, es necesario contemplar los costes relacionados con la infraestructura y suministros del lugar de trabajo. Los gastos mensuales se han desglosado de la siguiente manera para reflejar de forma realista los recursos consumidos:

- **Suministro energético:** Teniendo en cuenta el uso intensivo de la estación de trabajo y los

periféricos asociados, se estima un coste de 85 € al mes, el cual cubre el consumo eléctrico y el mantenimiento térmico durante la jornada laboral.

- **Servicios de Internet:** Se ha asignado un importe mensual de 30 € para cubrir la conexión a Internet mediante fibra óptica de banda ancha.

En total, los gastos mensuales de infraestructura se sitúan en 115,00 €. Dado que la planificación y ejecución del proyecto abarca un periodo de 5 meses, el coste total asciende a 575 €.

El presupuesto base para el proyecto se establece en 6.464,00 €, calculado mediante la suma de los recursos humanos y materiales estrictamente necesarios para su ejecución. En la Tabla 3.14 se muestra el desglose consolidado con los costes estimados para cada una de las categorías analizadas.

Concepto	Importe
Mano de obra (Ingeniero DevOps Junior)	5.739,00 €
Amortización de Hardware	150,00 €
Licencias de Software	0,00 €
Costes Indirectos (Infraestructura)	575,00 €
PRESUPUESTO TOTAL	6.464,00 €

Tabla 3.14: Resumen del presupuesto total del proyecto

3.2.5. Presupuesto y coste real del proyecto

Dado el carácter y finalidad estrictamente académica de este Trabajo Final de Grado, es necesario diferenciar el presupuesto económico de un proyecto profesional del coste financiero real derivado de la ejecución del sistema. El equipo de desarrollo, compuesto por un único integrante, no percibe ninguna remuneración, por lo que el coste asociado a los recursos humanos se reduce a cero.

En lo que respecta al software, todas las herramientas empleadas (Containerlab, Overleaf, Docker, MySQL) operan bajo código abierto o licencias de uso gratuito, no generando gasto. Asimismo, al desarrollarse el proyecto íntegramente en el entorno doméstico del autor, los costes indirectos de infraestructura (suministro energético y conectividad a Internet) se consideran cubiertos por causas externas y quedan excluidos del balance financiero.

Por consiguiente, el único coste real asociado al desarrollo del sistema corresponde a la amortización del hardware empleado (la estación de sobremesa y el ordenador portátil de pruebas), sumando un total de 150,00 €. En la Tabla 3.15 se presenta el desglose de este presupuesto real.

Concepto	Importe
Mano de obra (Ingeniero DevOps Junior)	0,00 €
Amortización de Hardware	150,00 €
Licencias de Software	0,00 €
Costes Indirectos (Infraestructura)	0,00 €
PRESUPUESTO TOTAL	150,00 €

Tabla 3.15: Resumen del presupuesto real del proyecto

3.3. Comparación respecto a la planificación inicial

La siguiente sección recoge el seguimiento real del proyecto, comparando la planificación final con la inicial definida, identificando las desviaciones producidas y sus causas. Las primeras iteraciones transcurrieron con normalidad, siguiendo el calendario previsto sin desviaciones significativas. Sin embargo, la fase de diseño comenzó a acumular cierto retraso provocado por la materialización de tres de los riesgos identificados en el análisis de riesgos: la baja por enfermedad de un familiar directo (ver Tabla 3.9), cambios en los requisitos durante el proyecto (ver Tabla 3.4) y errores al estimar la duración de las tareas (ver Tabla 3.3). Ninguno de estos riesgos obligó a reducir las funcionalidades previstas, si bien provocaron un retraso prolongando la duración total del proyecto aproximadamente dos semanas con respecto a la planificación inicial. Por último, la fase de implementación requirió cuatro iteraciones (explicadas en el Capítulo 6) en lugar de las tres contempladas originalmente, dado que como se menciona en el Capítulo 8, el ámbito temático del TFG era desconocido para mí, lo que dificultó determinar con antelación como organizar e implementar correctamente el sistema.

Para plasmar estas desviaciones temporales y reflejar el impacto de dichas incidencias en el cómputo global de horas, en la Tabla 3.16 se presenta una comparativa entre el esfuerzo planificado inicialmente y el desarrollo real.

Fase / Iteración	Plan (h)	Real (h)	Desviación	Motivo principal
1. Definición del proyecto	30	30	0	Cumplimiento exacto del cronograma.
2. Marco teórico y metodología	45	45	0	Ejecución según lo previsto.
3. Diseño, análisis y planificación inicial	60	75	+15	Materialización de riesgos (RSK007, RSK002) y reestimación (RSK001).
4. Implementación incremental	70	100	+30	Desdoblamiento en 4 iteraciones por la curva de aprendizaje técnica.
5. Pruebas y validación final	65	65	0	Pruebas ejecutadas sin bloqueos críticos.
6. Defensa y memoria final	32	35	+3	Ajustes finales de formato y redacción.
TOTAL	302	350	+48	Retraso aprox. de dos semanas

Tabla 3.16: Comparativa de esfuerzo temporal: Planificación inicial vs. Desarrollo real.

3.4. Lecciones aprendidas de la planificación

La finalización del proyecto y el análisis de las desviaciones temporales proporcionan un conjunto de lecciones aprendidas de gran valor para la estimación y ejecución de futuros proyectos. La gestión de un proyecto de ingeniería de software no concluye con la entrega del producto, sino con la reflexión crítica sobre el proceso metodológico empleado.

De cara a futuras planificaciones en proyectos de características similares, se han identificado las siguientes consideraciones:

- **Desarrollo de prototipos técnicos tempranos:** Aunque la fase inicial del proyecto incluía una fase de investigación teórica previa a la definición de requisitos, los bloqueos principales se dieron durante la implementación debido a que las limitaciones prácticas y las verdaderas capacidades de la tecnología no se conocieron con exactitud hasta el momento del desarrollo del código. En futuros proyectos, la planificación inicial deberá contemplar el desarrollo de pequeños prototipos funcionales de las tecnologías críticas antes de cerrar los requisitos, asegurando la viabilidad técnica real de lo que se pretende construir.
- **Inclusión de márgenes de holgura en tareas críticas:** En la estimación original del proyecto no se contemplaron márgenes de holgura para las tareas que formaban parte del camino crítico. Al tratarse de un dominio tecnológico nuevo para el desarrollador, los obstáculos técnicos consumieron tiempo de forma directa, provocando retrasos en el cronograma. Se concluye que es imprescindible añadir un factor de holgura (por ejemplo, de un 20 % o 30 %) en las estimaciones

de este tipo de tareas para poder asumir imprevistos sin afectar a la fecha de entrega final.

- **Gestión de contingencias en proyectos individuales:** Al tratarse de un equipo compuesto por un único desarrollador, la materialización del riesgo de indisponibilidad personal (Riesgo 3.9) supuso un bloqueo temporal en el avance. Para futuros desarrollos bajo esta misma restricción de recursos humanos, la planificación debe priorizar más la consecución del Producto Mínimo Viable (MVP) en las primeras semanas garantizando que siempre exista una versión funcional y entregable ante cualquier causa de fuerza mayor.

En conclusión, aunque la planificación inicial sufrió un desajuste de aproximadamente dos semanas, la adopción de una metodología iterativa e incremental permitió absorber los retrasos, desdoblar la fase de implementación y alcanzar los requisitos funcionales sin necesidad de recortar el alcance técnico establecido inicialmente.

3.5. Tecnologías utilizadas

En este capítulo se describen las herramientas y tecnologías que se han utilizado para el desarrollo de este proyecto. Se clasifican las herramientas en función de su propósito: desde los recursos empleados para la comunicación y el seguimiento con el tutor, así como las utilizadas en la fase de análisis, diseño y generación de la documentación. Además, se detallan las herramientas utilizadas para la gestión y el control del proyecto. Finalmente, se profundiza en el stack tecnológico seleccionado para la implementación de la lógica de control, la orquestación de red y la interfaz de usuario.

3.5.1. Herramientas de comunicación

Microsoft Teams

Es una plataforma de comunicación y colaboración desarrollada por Microsoft en 2017 que ofrece mensajería instantánea, videollamadas y la integración de otras herramientas de Office 365 como Word, Excel o Power Point. En este proyecto, esta herramienta ha servido como canal principal de comunicación con el tutor, permitiendo la realización de reuniones periódicas mediante videollamadas para abordar el avance del proyecto y el intercambio ágil de dudas y documentación técnica. Se ha convertido en una opción popular tanto en entornos empresariales como académicos al permitir una gestión centralizada de los flujos de trabajo [48].

Outlook

Es una aplicación de calendario y correo electrónico dentro de la suite de Microsoft 365, utilizada como estándar de comunicación oficial dentro de los entornos académicos [49]. En este proyecto, Outlook

desempeñó un papel fundamental en las primeras fases, permitiendo establecer los primeros contactos y coordinar la propuesta del proyecto antes de establecer una dinámica de seguimiento constante mediante videollamadas [50].

3.5.2. Herramientas para el análisis, diseño y documentación

Overleaf

Overleaf es un editor de $\text{\LaTeX}$ colaborativo basado en la nube que permite la redacción técnica y un diseño profesional propio de redacciones científicas y académicas. Se ha seleccionado esta herramienta por su capacidad para gestionar de forma automática el formato, índice de figuras y la bibliografía garantizando que la memoria cumpla con unos estándares de calidad. Además, la naturaleza del lenguaje $\text{\LaTeX}$ permite la inserción de bloques de código de manera estructurada, fundamental en un proyecto de automatización y despliegue de infraestructuras [51].

3.5.3. Herramientas para la gestión y el control del proyecto

Git

Es un sistema de control de versiones distribuido de código abierto que permite realizar una trazabilidad en los cambios que se han producido en el software. Aunque está diseñado para el trabajo en equipo y es ampliamente utilizado en entornos profesionales, también es útil para proyectos individuales por su capacidad para la creación de puntos de restauración (commits), facilitando la reversión de cambios ante posibles errores y garantizando la integridad del desarrollo, permitiendo documentar de forma cronológica la evolución del sistema tanto en repositorios remotos como locales [52]. En este proyecto, Git se ha utilizado para gestionar de manera eficiente las distintas versiones del controlador desarrollado en Python, los archivos de configuración de red (YAML) y los componentes de la interfaz de usuario.

GitLab

GitLab es una plataforma web de control de versiones y desarrollo de software colaborativo basado en el sistema Git [53]. Se ha elegido como repositorio remoto principal debido a su facilidad para el alojamiento y trazabilidad del código y por su gestión eficiente del historial técnico. Su adopción responde a la necesidad de asegurar la accesibilidad por parte del equipo docente y el alojamiento directo en la infraestructura del centro, garantizando la integridad de los datos ante cualquier fallo en el entorno local.

Alternativas:

- **GitHub**: Es una plataforma de alojamiento en la nube ampliamente utilizada y estandarizada en la comunidad de código abierto [54]. Aunque ofrece una interfaz ágil y una integración directa con distintos entornos de desarrollo, se descartó en favor de GitLab para priorizar la compatibilidad con las cuentas corporativas del profesorado y por el uso de un entorno de gestión más alineado con las directrices de la institución.
- **BitBucket**: Se trata de otra plataforma basada en Git, especialmente fuerte en entornos corporativos que utilizan el ecosistema de Atlassian (como Jira o Confluence) [55]. Se descartó debido a que su enfoque está más orientado a repositorios privados y flujos de trabajo cerrados, mientras que GitHub fomenta un perfil más abierto, ideal para la publicación y visibilidad de proyectos académicos.

3.5.4. Herramientas para el desarrollo del proyecto

Ubuntu (Linux)

Es un sistema operativo de código abierto basado en el núcleo Linux [56] que sirve como entorno de desarrollo principal para este proyecto. Se ha seleccionado la distribución Ubuntu debido a su estabilidad, amplia comunidad y su compatibilidad nativa con el motor Docker y la herramienta Containerlab [57]. Por otra parte, su gestión mediante la interfaz de línea de comandos (CLI) permite un control preciso sobre los recursos del sistema.

Alternativas:

- **Windows / macOS**: Son sistemas operativos comerciales orientados al usuario final y entornos corporativos que utilizan núcleos distintos a Linux. Se descartaron por la necesidad de utilizar máquinas virtuales intermedias (WSL o HyperKit) [58, 59] para ejecutar contenedores, lo cual añade una capa de abstracción que impide a Containerlab manipular la red de forma nativa y simular enlaces físicos directos.

Containerlab

Es un orquestador de topologías de red de código abierto basado en contenedores (Docker) que permite desplegar infraestructuras virtuales de manera rápida y escalable. A diferencia de simuladores tradicionales, Containerlab adopta el paradigma de Infraestructura como código (IaC) [28], donde la topología se define mediante un archivo de configuración estructurado en formato YAML. En este proyecto, esta herramienta actúa como el núcleo de la emulación, permitiendo levantar y conectar nodos de la red [60]. Se ha optado por esta herramienta debido a la ligereza que proporciona para garantizar el rendimiento del sistema en el entorno de ejecución local, permitiendo al controlador de Python gestionar el ciclo de vida de la red de forma automatizada.

Alternativas:

- **OpenShift / Kubernetes:** Son plataformas de orquestación de contenedores de nivel empresarial diseñadas para el despliegue, escalado y gestión de aplicaciones en la nube. Se descartaron debido a que no permiten la emulación de enlaces físicos a bajo nivel [61, 62]. Su arquitectura y modelo de red están diseñados para la orquestación y despliegue de aplicaciones en contenedores y no en la simulación de topologías de infraestructuras de red.
- **Kathará:** Es una herramienta de emulación de redes basada en contenedores Docker, diseñado principalmente para entornos académicos y de experimentación. Se descartó debido a que utiliza una sintaxis propia de configuración (lab.conf) [63] alejada del estándar industrial YAML para la infraestructura como código (IaC).

Docker

Es una plataforma de código abierto basada en la tecnología de contenedores que permite distribuir, empaquetar y ejecutar aplicaciones en entornos aislados. A diferencia de la virtualización tradicional, Docker se ejecuta en el sistema operativo anfitrión, permitiendo a los nodos levantarse en cuestión de segundos con un consumo mínimo de recursos. En este proyecto, Docker actúa como el motor de la ejecución alojando las imágenes de los distintos dispositivos de red [64]. Debido a su arquitectura ligera, es posible desplegar topologías de red complejas de forma dinámica, garantizando que cada componente del sistema funcione de manera independiente y reproducible en cualquier entorno Linux.

Alternativas:

- **Podman:** Es un motor de contenedores de código abierto que destaca por operar sin necesidad de un demonio central y por no requerir privilegios de raíz [65]. Se descartó porque Containerlab requiere interactuar a bajo nivel con el kernel de Linux para manipular los namespaces de red y crear los enlaces físicos virtuales. Este proceso requiere privilegios elevados y una integración con la API del motor, características que Docker ofrece de manera nativa y estable a través de su daemon.

YAML y JSON

Son formatos de serialización de datos legibles por los humanos, estándares en el desarrollo de software y en la automatización de las redes. Ambos desempeñan un papel crítico en la comunicación entre las distintas capas del sistema.

- **YAML (*YAML Ain't Markup Language*):** Es el formato definido por Containerlab para definir las topologías de red [66]. El controlador de Python genera automáticamente archivos con esta sintaxis, especificando los nodos, enlaces virtuales e imágenes de Docker para desplegar la infraestructura de red de manera automática.
- **JSON (*JavaScript Object Notation*):** Se utiliza como estándar de intercambio de información entre la interfaz de usuario realizada en React y el backend en Python [67]. Su estructura ligera

permite enviar las configuraciones introducidas por el usuario de forma eficiente mediante peticiones API.

Alternativas:

- **XML (eXtensible Markup Language)**: Es un lenguaje de marcado basado en etiquetas diseñado para almacenar y transportar datos de forma estructurada. Se descartó debido a su excesiva complejidad sintáctica. A diferencia de YAML o JSON, XML requiere de etiquetas de apertura y cierre para cada dato, aumentando considerablemente el tamaño de los ficheros de configuración y la carga de procesamiento en el parsing [68] priorizando en el sistema formatos más simples y ligeros que faciliten la depuración y el envío de los datos a través de la API.
- **TOML (Tom's Obvious Minimal Language)**: Es un formato de archivo de configuración diseñado para ser fácil de leer y escribir gracias a su semántica minimalista [69]. Pese a ser un formato excelente para archivos de configuración estáticos y su creciente popularidad en el ecosistema de Python, se descartó por una limitación en la compatibilidad técnica. Containerlab, el motor central de la infraestructura de red de este proyecto utiliza YAML como estándar nativo para la definición de topologías. Por esta razón, haber optado por TOML hubiera requerido de traducciones intermedias innecesarias, aumentando la complejidad sin aportar beneficios significativos.

MySQL

Es un sistema de gestión de base de datos relacional basado en el lenguaje de consulta SQL [70]. En este proyecto, MySQL actúa como el componente de persistencia de datos, permitiendo almacenar de forma segura y organizada toda la información necesaria y crítica del sistema. Su función principal es gestionar los requisitos de información 4.3, tales como el registro de los usuarios, las credenciales de acceso o las configuraciones de las topologías de red personalizadas. Gracias a su arquitectura cliente-servidor y a su compatibilidad con Python [71] garantiza que la información no se pierda al cerrar la aplicación, una gestión de estados robusta y una recuperación eficiente de los datos en cada sesión de trabajo.

Alternativas:

- **PostgreSQL**: Es un sistema de gestión de base de datos relacional orientado a objetos, conocido por su robustez y cumplimiento de estándares SQL. Se descartó debido a que MySQL ofrece un rendimiento ligeramente superior en operaciones de lectura simple y una curva de aprendizaje y configuración más ágil [72], lo cual se alinea mejor con los requisitos de velocidad y simplicidad de un orquestador de red local.
- **MongoDB**: Es una base de datos NoSQL orientada a documentos que almacena la información en estructuras similares a JSON en lugar de tablas [73]. Se analizó su uso, sin embargo, se descartó debido a que la estructura del sistema es altamente relacional. La elección de una base de datos no relacional hubiera complicado la integridad referencial de los datos, además, MySQL

garantiza mediante el uso de clave foráneas, que no existan nodos huérfanos en la base de datos si se elimina un nodo, siendo vital para la estabilidad del sistema.

- **SQLite:** Es un motor de base de datos SQL ligero y sin servidor que almacena toda la estructura en un único archivo de disco. Se descartó porque no permite la concurrencia de múltiples usuarios de forma eficiente ni la robustez de un servidor independiente [74].

Python

Es un lenguaje de programación de alto nivel, interpretado y multiparadigma, consolidado como el estándar de la industria en la automatización de infraestructura de red (NetDevOps) [75]. En este proyecto, Python actúa como el cerebro del controlador, integrando todas las capas del sistema [76]. Debido a su versatilidad, permite procesar las peticiones de la interfaz de usuarios, gestionar la persistencia de los datos en MySQL y automatizar el despliegue de la topología en Containerlab. Se ha seleccionado este lenguaje por la cantidad de librerías especializadas y su capacidad para manejar estructuras de datos complejas, facilitando la creación de un software escalable y modular.

Alternativas:

- **Go (Golang):** Es un lenguaje de programación compilado y de código abierto diseñado por Google para ofrecer alta eficiencia y gestión nativa de la concurrencia. Aunque es conocido por su excelente rendimiento y gestión de la concurrencia [77], se descartó debido a que el lenguaje Python ofrece una velocidad de desarrollo mayor y un extenso ecosistema de librerías de red más sencillo de integrar para un proyecto de estas características.
- **Bash:** Es un lenguaje de comandos e intérprete de shell estándar en sistemas Unix utilizado para la ejecución de scripts y automatización de tareas del sistema operativo [78]. Se descartó por su falta de escalabilidad y la dificultad para gestionar la comunicación con la base de datos o la creación de una API REST. Para un sistema como el propuesto en este TFG, era necesario la utilización de un lenguaje de propósito general y de alto nivel que permitiera una programación orientada a objetos.
- **Node.js (JavaScript):** Es un entorno de ejecución para JavaScript construido con el motor V8 de Chrome que permite ejecutar código en el servidor de forma asíncrona [79]. Se analizó el uso de esta tecnología para unificar el lenguaje de programación tanto en el frontend (React) como en el backend. Se descartó porque, pese a que es excelente para aplicaciones asíncronas, carece del soporte y la especialización que tiene Python en el ámbito de las redes.

React

Es una biblioteca de JavaScript de código abierto utilizada para la creación de interfaces de usuario modernas y dinámicas [80]. A diferencia de frameworks tradicionales, React se basa en una arquitectura basada en componentes reutilizables [81], permitiendo desarrollar una plataforma web modular donde

cada elemento de la red se gestiona de forma independiente. Se ha elegido esta tecnología por su eficiencia en el manejo del estado de la aplicación, facilitando una interacción visual con el controlador en Python y garantizando una experiencia de usuario fluida durante la orquestación de la infraestructura de red.

Alternativas:

- **Angular:** Es una plataforma y framework de desarrollo basado en TypeScript para construir aplicaciones web de una sola página (SPA) con una estructura altamente jerarquizada [82]. Se descartó por su elevada curva de aprendizaje y su arquitectura más rígida y pesada. React, al ser una biblioteca centrada en la interfaz de usuario, permite una mayor flexibilidad y ligereza en el desarrollo, facilitando la integración con Python de una forma más modular y ágil.
- **Vue.js:** Es un framework de JavaScript progresivo y versátil utilizado para construir interfaces de usuario mediante un modelo de datos reactivo [83]. Se decidió utilizar React por disponer de un ecosistema más amplio en librerías de terceros y componentes de visualización disponibles. La inmensa comunidad de React aseguró encontrar soluciones probadas para cada reto de la interfaz en un sistema donde se quiere mostrar la topología de red.
- **Desarrollo con JavaScript nativo (Vanilla JS):** Se refiere al uso del lenguaje JavaScript pero sin la utilización de librerías o frameworks externos. Se descartó por la complejidad de gestionar manualmente el DOM (Document Object Model) [84] en una aplicación dinámica. En un orquestador de red donde los elementos cambian constantemente según la respuesta del controlador, React permite actualizar solo las partes necesarias de la pantalla de forma automática, garantizando un rendimiento mayor y un código más mantenible.

Visual Studio Code

Es un editor de código fuente ligero, potente y de código abierto desarrollado por Microsoft [85]. Se ha seleccionado como el entorno de desarrollo principal (IDE) debido a su versatilidad y sus extensiones, que permiten la integración con todas las tecnologías del proyecto. Gracias a sus herramientas de depuración y su terminal integrada, permite gestionar de forma simultánea el controlador en Python, la creación de interfaces en React y la ejecución de comandos en Docker y Containerlab. Además, su capacidad para trabajar sistemas de control de versiones como Git garantiza un flujo de trabajo ágil y permite organizar y editar los archivos de configuración en YAML y JSON con una sintaxis clara y estructurada.

Alternativas:

- **PyCharm / Eclipse:** Son entornos de desarrollo integrados (IDE) muy potentes que ofrecen herramientas avanzadas de análisis de código, depuración y gestión de proyectos, especializados en Python y Java respectivamente. Se descartaron por su alto consumo de memoria RAM [86]. Para un proyecto que ya requiere ejecutar contenedores Docker y un entorno React en paralelo,

se prefirió la ligereza de VS Code, que ofrece una experiencia similar a través de extensiones, sin penalizar el sistema [87, 88].

- **Sublime Text / Atom:** Son editores de texto avanzados, ligeros y altamente personalizables con plugins [89, 90]. Se descartaron porque carecen de una integración tan profunda y nativa con herramientas de depuración y terminales de contenedores como las que ofrece VS Code. La facilidad que ofrece VS Code para gestionar extensiones de YAML, Git, Docker en un solo entorno lo hace superior para este proyecto.
- **Vim / Nano:** Son editores de texto basados en la terminal que permiten la edición de archivos directamente desde la línea de comandos sin necesidad de interfaz gráfica [91, 92]. Aunque son herramientas indispensables para ediciones rápidas directamente en el servidor o dentro de los nodos de red, se descartaron como entorno de desarrollo personal. Carecen de interfaz gráfica que facilite la gestión de múltiples archivos, autocompletado inteligente de código y la visualización de la estructura del proyecto en React y Python de forma simultánea.

3.5.5. Declaración sobre el uso de la Inteligencia Artificial

En concordancia con las directrices actualizadas de la ANECA (Agencia Nacional de Evaluación de la Calidad y Acreditación) [93], se declara que se ha utilizado herramientas de Inteligencia Artificial (en concreto Gemini [94] y Claude Code [95]) como un asistente de apoyo durante el desarrollo del proyecto.

El uso de estas herramientas se ha limitado exclusivamente a tareas de soporte técnico y optimización, que se resumen en los siguientes puntos:

- **Optimización, revisión de código y maquetación visual:** Se ha usado como asistencia en la identificación de redundancia en los componentes de React y depuración de errores puntuales. Ha sido fundamental para la implementación de efectos visuales avanzados en el frontend; la IA permitió integrar animaciones complejas y resolver desafíos geométricos en el posicionamiento dinámico de los nodos y enlaces del canvas SVG (ej: diagrama de red en la parte de monitorización) que, por el tiempo de aprendizaje técnico necesario, habrían resultado inviables dentro del cronograma. Asimismo, se ha empleado para generar estilos en Tailwind CSS y la estructura vectorial de los elementos gráficos.
- **Soporte en la redacción y corrección de la memoria:** Se han utilizado modelos de lenguaje para la revisión ortográfica, sugerencia de sinónimos académicos para evitar redundancias en la redacción de la presente memoria y generación de código técnico propio de LaTeX, específicamente en el diseño y configuración de tablas complejas y elementos flotantes. Asimismo, se ha utilizado como soporte para la normalización de las citas bibliográficas, facilitando la generación de las entradas en formato BibTeX bajo el estándar IEEE.
- **Generación de estructura base en código:** Se ha usado para la aceleración de escritura de código repetitivo o plantillas iniciales, proporcionando una base sólida sobre la cual construir la

lógica del proyecto. Esto incluye la implementación de las plantillas base para la concurrencia asíncrona mediante WebSockets en FastAPI, facilitando la gestión de flujos de logs en tiempo real. Asimismo, se utilizó para estructurar la lógica inicial de cálculo de posiciones de los nodos en el grafo de la topología, creando el algoritmo para la normalización de coordenadas que permitió transformar los datos brutos del servidor en una representación visual coherente en el frontend.

- **Asistencia en la investigación y resolución de dudas técnicas:** Dado que la temática del presente TFG ha requerido la exploración de un ámbito tecnológico que no se encontraba dentro de los conocimientos adquiridos durante el periodo académico, la IA ha servido como un recurso de investigación preliminar. Se ha usado para la búsqueda de información técnica, asimilación de conceptos complejos y la resolución de dudas sobre la integración de componentes de red en entornos virtualizados. Este apoyo ha sido fundamental para aclarar las posibilidades de desarrollo, evaluar la viabilidad técnica de las distintas soluciones planteadas y acelerar la curva de aprendizaje ante la falta de conocimientos previos en el tema.

Es obligatorio destacar que el uso de la IA ha actuado bajo un enfoque de programación en pareja, donde el desarrollo intelectual, la toma de decisiones arquitectónicas, el diseño conceptual del orquestador, el modelado de la base de datos MySQL, la lógica de integración con Containerlab y la validación final del software han sido ideados, supervisados y ejecutados íntegramente por mi persona.

Capítulo 4

Análisis

El objetivo principal de este capítulo es la identificación y definición formal de las necesidades que debe satisfacer el sistema desarrollado. Para garantizar una herramienta que responda eficazmente a las demandas del entorno profesional, el análisis se ha centrado en un perfil de usuario especializado: ingenieros de infraestructura y operadores de redes. Estos perfiles demandan soluciones de orquestación que reduzcan la complejidad operativa y permitan la experimentación ágil.

4.1. Requisitos del sistema

El presente capítulo tiene como objetivo determinar los requisitos funcionales, no funcionales y de información que condicionan el desarrollo del proyecto. A través de este análisis, se identifican las necesidades del usuario y las restricciones tecnológicas del entorno de red virtualizado, sirviendo como guía para las posteriores fases de arquitectura e implementación del sistema.

4.1.1. Requisitos funcionales

Los requisitos funcionales comprenden las acciones que debe llevar a cabo el sistema para cumplir con las expectativas del usuario, detallando funcionalidades y características necesarias para que la interacción del usuario sea completa y eficiente.

Los requisitos funcionales detallados a continuación se corresponden con el *Producto Mínimo Viable (MVP)*. Se han priorizado aquellas funcionalidades que permiten el despliegue y la orquestación base de la red virtualizada. Sin embargo, el diseño del sistema se ha concebido con un enfoque modular y escalable, posibilitando que el sistema evolucione e integre nuevas funcionalidades en líneas de trabajo futuras.

Tabla 4.1: Requisitos funcionales del sistema

Id	Nombre	Descripción	Prioridad
RF01	Gestión de cuenta de usuario (CRUD)	El sistema debe permitir el registro de nuevos usuarios, consulta de datos y preferencias, edición de la información personal y eliminación definitiva de la cuenta.	Alta
RF02	Gestión de archivos de configuración	El sistema debe permitir la carga de archivos de topología, verificando la sintaxis YAML para asegurar que el archivo cumple con el esquema requerido por el motor de orquestación.	Alta
RF03	Plantillas de topología predefinidas	El sistema debe disponer de un conjunto de topologías de red predefinidas. En esta versión, los usuarios pueden consultar e importar estas plantillas a su espacio de trabajo personal; una vez importadas, el usuario tendrá autonomía para modificar, desplegar o eliminar dicha configuración como si fuera una propia, garantizando que el catálogo original permanezca inalterado para el resto de usuarios.	Media
RF04	Clonación de escenarios propios	El sistema debe permitir duplicar una configuración existente para crear una nueva variante sin modificar la original.	Baja
RF05	Exportación de topologías	El sistema debe permitir la descarga completa de la configuración, incluyendo tanto el archivo de topología (YAML) como los archivos de configuración de los nodos. La exportación se realizará en un formato comprimido para garantizar la portabilidad y facilitar la edición externa.	Media
RF06	Histórico de topologías	El sistema debe permitir la consulta de un historial de configuraciones creadas por cada usuario.	Media

Análisis

Id	Nombre	Descripción	Prioridad
RF07	Despliegue de la topología	El sistema debe ser capaz de ejecutar el despliegue automatizado de la topología a partir de configuraciones válidas. Se considerará una configuración válida aquel archivo YAML cuya sintaxis, imágenes declaradas y estructura lógica hayan sido verificadas previamente por el sistema sin registrar errores.	Alta
RF08	Visualización de logs de despliegue	La interfaz web debe permitir la consulta de los registros (logs) generados por el proceso de despliegue una vez finalizado el mismo.	Alta
RF09	Mensajes de error	La interfaz web debe notificar al usuario de fallos en el despliegue mediante mensajes informativos.	Alta
RF10	Monitorización de nodos	El sistema deberá proporcionar un panel visual donde se muestre el estado de cada nodo desplegado.	Alta
RF11	Inspección de nodos	El sistema debe permitir visualizar detalles técnicos de un nodo activo (dirección IP, estado de las interfaces).	Media
RF12	Modificación de la topología	El sistema debe permitir la modificación de parámetros de red en configuraciones que se encuentran actualmente desplegadas.	Baja
RF13	Reinicio de nodos	El sistema debe permitir el reinicio de un nodo específico sin desplegar toda la topología de nuevo.	Media
RF14	Parada de la topología	El sistema debe permitir detener todos los contenedores y servicios de una topología desplegada, conservando los archivos de configuración en el host para permitir un futuro despliegue.	Alta
RF15	Eliminación completa de la topología	El sistema debe permitir la destrucción completa de una topología. Esta acción conlleva el borrado de sus registros en la base de datos.	Alta

4.1.2. Requisitos no funcionales

Los requisitos no funcionales son aquellos que definen cómo debe comportarse el sistema en términos de rendimiento, seguridad, escalabilidad, accesibilidad y usabilidad. Son fundamentales para garantizar que el sistema satisfaga las expectativas del usuario en cuanto a calidad y experiencia de uso.

Tabla 4.2: Requisitos no funcionales del sistema

Id	Nombre	Descripción	Prioridad
RNF01	Rendimiento	El sistema deberá ser capaz de garantizar el despliegue de cinco nodos de red en menos de 60 segundos. ¹	Media
RNF02	Estabilidad y aislamiento	El sistema debe garantizar el aislamiento de recursos y redes entre topologías, asegurando que un fallo o el uso intensivo de recursos por una topología no interrumpa la estabilidad del sistema anfitrión ni de otras sesiones.	Alta
RNF03	Seguridad	El sistema debe delegar la autenticación de usuario mediante un proveedor externo, asegurando que solo usuarios autenticados puedan acceder a sus respectivas topologías.	Alta
RNF04	Disponibilidad web	La interfaz de usuario debe estar disponible de manera continua para la gestión y consulta de los nodos.	Alta
RNF05	Disponibilidad de red	El sistema debe garantizar que los nodos desplegados permanezcan activos independientemente del estado de la interfaz o la pérdida de conexión de la sesión web.	Alta
RNF06	Usabilidad	La interfaz debe ofrecer un diseño intuitivo y responsivo con una curva de aprendizaje reducida y unos tiempos de carga inferiores a tres segundos. ²	Media

¹Este tiempo de 60 segundos se ha establecido como umbral máximo para garantizar una experiencia de usuario fluida. Superar este umbral penalizaría la agilidad operativa, contrastando negativamente con la inmediatez que ofrece la orquestación mediante contenedores frente a la virtualización tradicional.

²Este umbral de tres segundos se considera el punto de inflexión donde la paciencia del usuario disminuye significativamente, comprometiendo la percepción de eficiencia de la herramienta.

4.1.3. Requisitos de información

Los requisitos de información comprenden los datos y activos digitales que el sistema debe procesar y almacenar para garantizar su correcto funcionamiento y cumplir con las necesidades del usuario. Estos requisitos aseguran que la información crítica, como los perfiles de usuario, los registros de actividad, las configuraciones de red predefinidas, se mantengan íntegras y accesibles.

Tabla 4.3: Requisitos de información del sistema

Id	Nombre	Descripción	Prioridad
RI01	Gestión de cuentas	El sistema deberá almacenar información correspondiente a los usuarios, en concreto: Nombre, email y fecha de registro.	Alta
RI02	Repositorio de topologías	El sistema debe almacenar los datos estructurales y descriptivos de cada diseño de red. Esto incluye los siguientes campos de información: ■ Metadatos: Identificador único (id), nombre de la topología, descripción, fecha de creación y el id del usuario creador. ■ Definición técnica: Archivos de configuración en formato YAML que contienen la declaración de los nodos, imágenes de contenedor, recursos de hardware asignados y la matriz de adyacencia. ■ Configuraciones de red: Archivos auxiliares de protocolos (archivos .conf o .txt) vinculados a nodos específicos. ■ Estado: Indicador del estado actual (Almacenada / Desplegada / Error).	Alta
RI03	Histórico de logs	El sistema debe almacenar el registro de logs y eventos generados por el motor de orquestación en cada proceso de despliegue.	Media

Id	Nombre	Descripción	Prioridad
RI04	Estado operativo de los nodos	El sistema debe proporcionar la información de disponibilidad de los nodos activos. Esta información incluye el estado de ejecución (Activo / Inactivo) y métricas básicas de consumo de recursos. Para optimizar los recursos del host, estos datos se consultarán de forma asíncrona mediante la API, visualizándose en la interfaz bajo demanda del usuario.	Alta

4.2. Criterios de aceptación

Para validar el cumplimiento de los requisitos definidos anteriormente, se establecen los siguientes criterios de aceptación que van a servir de base para la fase de pruebas del sistema.

Tabla 4.4: Criterios de aceptación del sistema

Id	Requisitos	Criterios de aceptación
CA01	RF01, RI01	El usuario puede registrarse, iniciar sesión y editar su perfil, quedando los datos (email, nombre) almacenados en la base de datos.
CA02	RF02, RI02	Al subir un archivo YAML, el sistema valida la sintaxis y lo almacena en el repositorio. El sistema confirma la correcta importación sin iniciar el despliegue automático.
CA03	RF08, RNF01, RF07	Al solicitar el despliegue, el sistema orquestador instancia los nodos y enlaces. Se considera exitoso si se despliegan 5 nodos en menos de 60 segundos dejando los servicios listos para ser monitorizados.
CA04	RF09, RF10, RF11, RI03, RI04, RNF05	Con la red en ejecución, el panel visual muestra el estado y métricas de los nodos y de la red, proporcionando una consola integrada para la visualización de los logs generados por el proceso de despliegue. Se garantiza que los nodos permanezcan activos independientemente del estado de la sesión web del usuario.

Id	Requisitos	Criterios de aceptación
CA05	RF12, RF13, RF14, RF15	El sistema permite modificar parámetros de los nodos desplegados, reiniciar nodos individualmente, detener la topología y realizar la eliminación definitiva de recursos.
CA06	RF03, RF04, RF06	El usuario puede importar plantillas predefinidas a su espacio, clonar sus propios escenarios y consultar el historial de topologías creadas.
CA07	RF05	El sistema permite la descarga completa de la topología (YAML y archivos de configuración asociados) en un archivo comprimido.
CA08	RNF02, RNF03, RNF04, RNF06	El sistema mantiene el aislamiento de la red, garantiza la disponibilidad continua de la interfaz web, la cual dispone de unos tiempos de carga menores de tres segundos y permite el despliegue de la red en un máximo de tres pasos.

4.3. Casos de uso

Los casos de uso se corresponden con el conjunto de interacciones que los actores (usuarios o sistemas externos) realizan con la aplicación para lograr objetivos concretos. Actúan como el puente entre los requisitos funcionales descritos anteriormente y la implementación técnica, permitiendo visualizar cómo el actor y el sistema van a interactuar para cumplir los requisitos. En esta plataforma de despliegue, se ha definido un actor principal denominado “Usuario”, que engloba los perfiles de ingeniero de infraestructura y operador de red definido anteriormente. Al observar que las funcionalidades de gestión de perfil, despliegue y carga de configuraciones son comunes para cualquier persona que acceda a esta herramienta, se utiliza este actor general para simplificar el análisis y evitar redundancias en los diagramas.

A continuación, se detallan los casos de uso a desarrollar a lo largo del proyecto. En la Figura 4.1 se ilustra de manera global la interacción del actor con las diferentes funcionalidades del sistema, las cuales se especifican en las tablas posteriores.

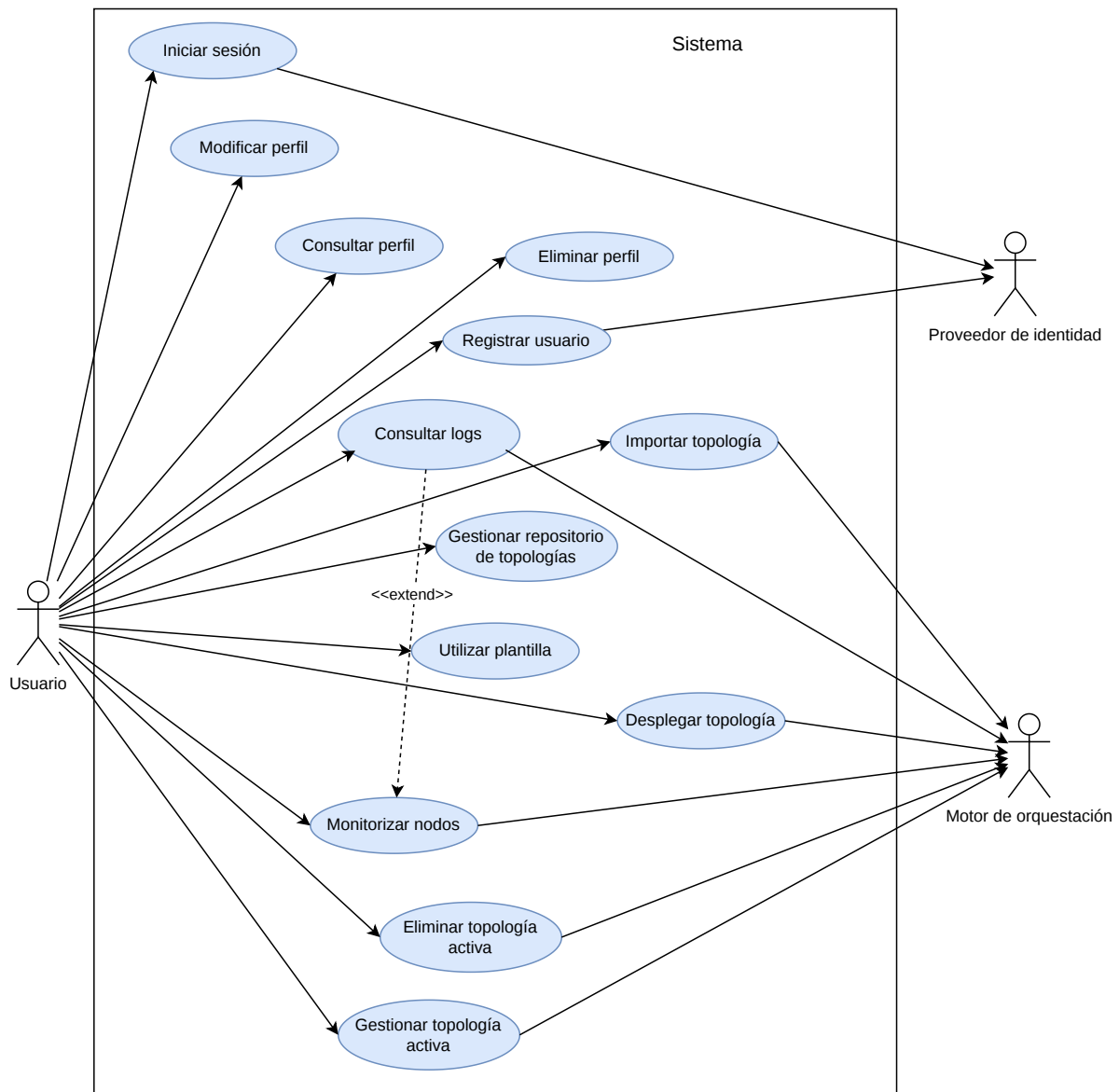


Figura 4.1: Diagrama de CCUU del sistema

ID	CU01
Nombre	Iniciar sesión
Descripción	El usuario inicia sesión en el sistema para acceder a su espacio de trabajo privado.
Precondición	El usuario debe disponer de una cuenta previa en el sistema.
Secuencia normal	1. El usuario solicita el acceso al sistema mediante el proveedor de identidad. 2. El sistema redirige la solicitud al proveedor para la validación de la identidad. 3. El proveedor confirma la identidad y devuelve un token de acceso válido al sistema. 4. El sistema autoriza la sesión y habilita las funcionalidades del usuario.
Postcondición	El usuario queda autenticado y tiene acceso a su espacio de trabajo.
Excepciones	2.a. Si el usuario cancela la autenticación en el proveedor externo o la identidad es denegada, el sistema restringe el acceso y muestra un mensaje de error.

Tabla 4.5: Especificación del Caso de Uso: Iniciar sesión

ID	CU02
Nombre	Registrar usuario
Descripción	El sistema crea un nuevo perfil para un nuevo usuario tras una autenticación exitosa mediante el proveedor externo.
Precondición	El usuario no debe estar registrado previamente en el sistema.
Secuencia normal	1. El usuario inicia el proceso de autenticación con el proveedor externo. 2. El sistema recibe la confirmación de la identidad y los datos básicos (email, nombre, ...). 3. El sistema verifica que la identidad no existe en la base de datos. 4. El sistema crea una nueva instancia de usuario en la base de datos local. 5. El sistema confirma el alta en el sistema y permite el acceso al espacio de trabajo.
Postcondición	Existe un nuevo registro de usuario vinculado a la identidad externa.
Excepciones	4.a. Si se produce algún error en la creación del perfil, el sistema notifica del fallo y se deniega el acceso.

Tabla 4.6: Especificación del Caso de Uso: Registrar usuario

ID	CU03
Nombre	Modificar perfil
Descripción	El usuario actualiza sus datos personales almacenados en el sistema.
Precondición	El usuario debe estar autenticado en el sistema.
Secuencia normal	1. El usuario solicita el acceso a sus datos personales. 2. El sistema recupera y presenta la información actual del usuario. 3. El usuario realiza las modificaciones que desea en los campos permitidos. 4. El sistema valida la integridad de los nuevos datos proporcionados. 5. El sistema actualiza la persistencia en la base de datos y confirma la operación.
Postcondición	Los datos del usuario quedan actualizados en el sistema.
Excepciones	4.a. Si los datos introducidos por el usuario no cumplen con el formato o requisitos del sistema, el sistema notifica al usuario del error y se vuelve al paso 3. 5.a. Si se produce un error al actualizar la base de datos, el sistema informa al usuario del error y se vuelve al paso 5.

Tabla 4.7: Especificación del Caso de Uso: Modificar perfil

ID	CU04
Nombre	Consultar perfil
Descripción	El usuario visualiza su información personal almacenada en el sistema.
Precondición	El usuario debe estar autenticado en el sistema.
Secuencia normal	1. El usuario solicita el acceso a sus datos personales. 2. El sistema recupera los datos asociados a la identidad del usuario. 3. El sistema presenta la información detallada.
Postcondición	El usuario conoce el estado y los datos de su cuenta.
Excepciones	2.a. Si se produce un error en la recuperación de los datos, el sistema muestra el error al usuario y se vuelve al paso 2.

Tabla 4.8: Especificación del Caso de Uso: Consultar perfil

ID	CU05
Nombre	Eliminar perfil
Descripción	El usuario solicita la baja definitiva del sistema y la eliminación de todos los datos asociados.
Precondición	El usuario debe estar autenticado en el sistema.
Secuencia normal	1. El usuario solicita la eliminación de su cuenta personal. 2. El sistema elimina los registros asociados al usuario (perfil, topologías,...). 3. El sistema finaliza la sesión y notifica la ejecución de la baja.
Postcondición	La identidad del usuario y los recursos asociados dejan de existir en el sistema.
Excepciones	2.a. Si se produce un error crítico durante la eliminación de los datos, el sistema informa de los problemas técnicos y mantiene la cuenta activa.

Tabla 4.9: Especificación del Caso de Uso: Eliminar perfil

ID	CU06
Nombre	Importar topología
Descripción	El usuario incorpora archivos de definición de red y configuraciones adicionales a su repositorio personal en el sistema.
Precondición	El usuario debe estar autenticado.
Secuencia normal	1. El usuario suministra los archivos de configuración desde su almacenamiento local. 2. El sistema analiza la estructura y valida la sintaxis de las definiciones proporcionadas. 3. El sistema vincula la nueva configuración a la identidad del usuario en la persistencia. 4. El sistema confirma la correcta integración de la topología en el repositorio.
Postcondición	La topología queda guardada y disponible para su gestión.
Excepciones	2.a. Si el archivo no cumple con el esquema YAML o contiene parámetros no soportados, el sistema notifica al usuario del error y vuelve al paso 1. 3.a. Si se produce un error durante la escritura de los datos, el sistema informa del error al usuario y permite reintentar la operación.

Tabla 4.10: Especificación del Caso de Uso: Importar topología

ID	CU07
Nombre	Gestionar repositorio de topologías
Descripción	El usuario administra su colección de topologías permitiendo la duplicación, eliminación o extracción de los recursos almacenados.
Precondición	El usuario debe estar autenticado y disponer de topologías guardadas.
Secuencia normal	1. El usuario solicita las topologías asociadas a su cuenta. 2. El sistema recupera y presenta los metadatos de las configuraciones almacenadas. 3. El usuario selecciona una entrada y solicita una operación de gestión (Clonar, Exportar o Eliminar). 4. El sistema procesa la solicitud: genera una nueva instancia idéntica, empaqueta la configuración para su transferencia externa o revoca el recurso de manera persistente. 5. El sistema confirma la finalización de la tarea administrativa.
Postcondición	El repositorio de topologías se actualiza según la acción ejecutada.
Excepciones	2.a. Si el historial está vacío, muestra un mensaje avisando de la ausencia de datos. 3.a. Si se produce un error en el procesamiento de la solicitud, el sistema informa del error y mantiene el estado previo.

Tabla 4.11: Especificación del Caso de Uso: Gestionar repositorio de topologías

ID	CU08
Nombre	Utilizar plantilla
Descripción	El usuario selecciona una topología preconfigurada proporcionada por el sistema para añadirlo a su repositorio y utilizarlo como base.
Precondición	El usuario debe estar autenticado.
Secuencia normal	1. El usuario solicita el listado de plantillas preconfiguradas disponibles. 2. El sistema presenta los modelos de red de referencia. 3. El usuario selecciona un modelo y solicita su incorporación a su espacio de trabajo. 4. El sistema duplica la configuración de la plantilla y vincula la nueva instancia a la identidad del usuario. 5. El sistema confirma la disponibilidad del nuevo recurso en el repositorio del usuario.
Postcondición	Se crea una copia de la plantilla en el espacio del trabajo del usuario.
Excepciones	4.a. Si se produce un error en la duplicación de los archivos, el sistema muestra un mensaje al usuario y cancela la operación.

Tabla 4.12: Especificación del Caso de Uso: Utilizar plantilla

ID	CU09
Nombre	Desplegar topología
Descripción	El usuario solicita la ejecución y puesta en marcha de una configuración de red previamente almacenada.
Precondición	El usuario debe estar autenticado y disponer de una topología válida en su repositorio.
Secuencia normal	1. El usuario selecciona una topología y solicita su despliegue en la infraestructura. 2. El sistema valida la integridad de la configuración. 3. El sistema inicia el flujo de aprovisionamiento mediante el motor de orquestación. 4. El sistema actualiza el estado del recurso a “En despliegue” y proporciona retroalimentación del proceso en tiempo real. 5. El motor de orquestación confirma la creación de nodos y enlaces virtuales. 6. El sistema notifica la finalización exitosa del proceso.
Postcondición	La topología cambia su estado a “Activa” y se habilitan los servicios de monitorización y gestión de los nodos.
Excepciones	1.a. Si el usuario tiene activa una topología, el sistema restringe la posibilidad de nuevos despliegues concurrentes. 6.a. Si ocurre un fallo en el motor de orquestación, el sistema aborta el despliegue, deshace los cambios y el estado de la topología pasa a “Error”.

Tabla 4.13: Especificación del Caso de Uso: Desplegar topología

ID	CU10
Nombre	Monitorizar nodos
Descripción	El usuario supervisa el rendimiento y los parámetros técnicos de uno de los nodos que componen la topología activa.
Precondición	El usuario debe estar autenticado y debe disponer de una topología en estado “Activa”.
Secuencia normal	1. El usuario solicita el estado de los elementos de la topología activa. 2. El sistema recupera y presenta los indicadores de disponibilidad de cada nodo. 3. El usuario selecciona un nodo específico para auditar sus métricas de rendimiento. 4. El sistema obtiene y suministra telemetría en tiempo real (direccionamiento IP, consumo de CPU y RAM y estado operacional de las interfaces).
Postcondición	El usuario visualiza la telemetría de los recursos sin alterar la topología.
Excepciones	4.a. Si el nodo deja de responder durante la consulta de telemetría, el sistema lo representa visualmente en color rojo dentro del esquema de la topología.

Tabla 4.14: Especificación del Caso de Uso: Monitorizar nodos

ID	CU11
Nombre	Gestionar topología activa
Descripción	El usuario realiza modificaciones o acciones específicas sobre nodos concretos o sobre el despliegue mientras la topología se encuentra en ejecución.
Precondición	El usuario debe estar autenticado y disponer de una topología en estado “Activa”.
Secuencia normal	1. El usuario solicita una instrucción de control (reinicio de nodo, parada de emergencia o modificación de parámetros de red). 2. El sistema traduce la solicitud en directivas compatibles con el motor de orquestación. 3. El motor de orquestación aplica los cambios de manera síncrona sobre los recursos virtuales. 4. El sistema monitoriza el resultado de la operación y actualiza el estado de la topología. 5. El sistema confirma la ejecución y el nuevo estado operacional del recurso.
Postcondición	El estado de la topología o del nodo se actualiza según la acción solicitada.
Excepciones	3.a. Si se produce un error de comunicación con el motor de orquestación, el sistema notifica de la imposibilidad de aplicar el cambio y mantiene el estado previo.

Tabla 4.15: Especificación del Caso de Uso: Gestionar topología activa

ID	CU12
Nombre	Eliminar topología activa
Descripción	El usuario ordena la destrucción de los recursos desplegados en el host para liberar memoria y CPU, manteniendo la configuración en su repositorio.
Precondición	El usuario debe estar autenticado y existe una topología desplegada en el sistema.
Secuencia normal	1. El usuario solicita la detención y destrucción del despliegue activo. 2. El sistema requiere una confirmación de la acción. 3. El usuario confirma la voluntad de continuar con la eliminación. 4. El sistema transmite las directivas de finalización al motor de orquestación. 5. El sistema confirma la eliminación de los nodos y enlaces virtuales liberando los recursos del host. 6. El sistema actualiza el estado de la topología a “Almacenada”.
Postcondición	Los recursos del host quedan liberados y la infraestructura virtual deja de existir.
Excepciones	4.a. Si se produce un error en el motor de orquestación durante la destrucción, el sistema notifica la anomalía y ofrece la posibilidad de reintentar la operación para garantizar la liberación de recursos en el host.

Tabla 4.16: Especificación del Caso de Uso: Eliminar topología activa

ID	CU13
Nombre	Consultar logs de actividad
Descripción	El usuario accede al historial de eventos y salida de consola de un nodo específico para depurar fallos de configuración o protocolos.
Precondición	El usuario debe estar autenticado y el nodo seleccionado debe pertenecer a una topología en estado "Activa".
Secuencia normal	1. El usuario solicita el acceso a los logs de actividad de un nodo en concreto. 2. El sistema gestiona la petición de datos ante el motor de orquestación. 3. El motor de orquestación recupera los flujos de salida asociados al proceso del nodo. 4. El sistema canaliza y presenta la información de la actividad registrada.
Postcondición	El usuario visualiza la actividad interna del nodo sin interrumpir su ejecución.
Excepciones	3.a. Si el nodo no ha generado logs todavía, el sistema notifica de la ausencia de registros y permanece a la espera de nuevos flujos de datos. 3.b. Si se produce un error de comunicación con el motor de orquestación, el sistema informa de la imposibilidad de recuperar los logs en ese momento.

Tabla 4.17: Especificación del Caso de Uso: Consultar logs

4.4. Modelo de dominio

El modelo de dominio proporciona una representación que permite visualizar las entidades conceptuales que rigen la lógica del sistema y las relaciones que existen entre ellas abstrayéndose de detalles técnicos específicos del lenguaje de programación o de la base de datos. Su objetivo es establecer un vocabulario común y una estructura lógica que soporte los requisitos previamente definidos.

En la Figura 4.2, se observa cómo el modelo estructura la gestión de las topologías por parte de los usuarios. Se detalla la descomposición de estas redes en sus elementos fundamentales (nodos, interfaces y enlaces) y se define el mecanismo de trazabilidad mediante un sistema de logs. Esta jerarquía asegura que la lógica de negocio se mantenga coherente independientemente de la infraestructura subyacente.

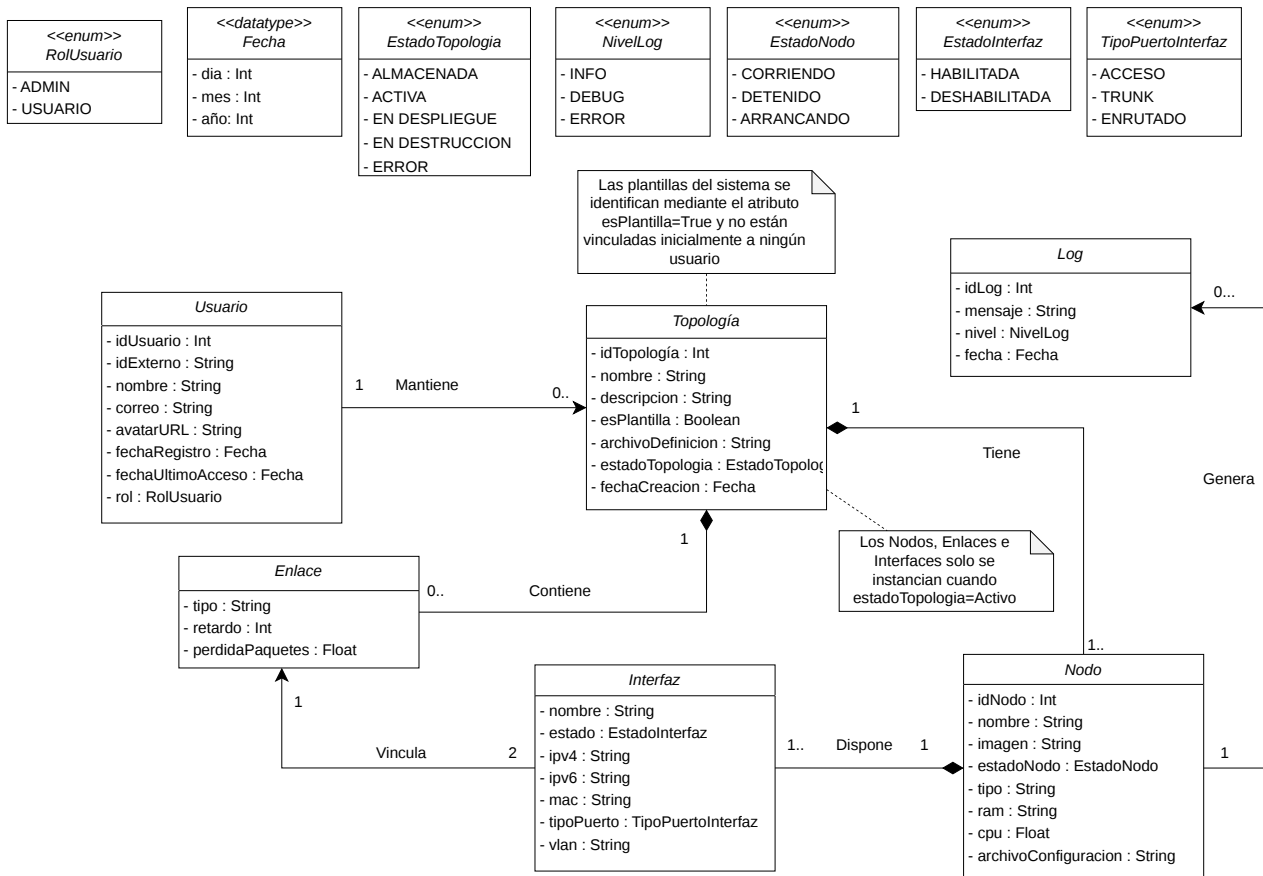


Figura 4.2: Modelo de dominio del sistema

En este modelo se conceptualizan las siguientes entidades:

- **Usuario:** Representa a los actores del sistema. Cada usuario se identifica mediante un correo y un *idExterno*, lo que sugiere una integración con sistemas de autenticación externos. Su rol principal es el de propietario y gestor de sus propias configuraciones.
- **Topología:** Es la unidad principal de trabajo y representa una infraestructura de red completa. Puede encontrarse en tres estados: almacenada, desplegada o en error; y se diferencia entre plantillas (modelos de red base proporcionados por el sistema) y topologías de usuario mediante un atributo booleano. Cada topología contiene un *archivoDefinicion*, el cual almacena el código fuente (YAML) que define el estado deseado de la red materializado por el orquestador.
- **Nodo:** Representa un dispositivo de red individual (router, switch o controlador SDN) dentro de la infraestructura. Además de sus recursos hardware (CPU o RAM), cada nodo cuenta con un *archivoConfiguracion* donde reside la lógica del plano de control (protocolos de routing, políticas de seguridad). Su estado es dinámico y cambia según la ejecución del controlador.
- **Interfaz:** Define los puntos de conexión lógica de un nodo. Es una entidad versátil que permite modelar tanto puertos de Capa 2 (switches) como de Capa 3 (routers) mediante el uso de atributos como *mac*, *vlan*, *ipv4* o *ipv6*, garantizando de esta manera la compatibilidad con arquitecturas de

red modernas. Su comportamiento se rige por una enumeración que distingue entre modos de *Acceso*, *Trunk* o *Enrutado*.

- **Enlace:** Representa la conexión física o virtual entre dos interfaces. Es la entidad encargada de definir la topología de interconexión (quién se conecta con quién) y permite abstraer la red como un grafo de nodos y aristas.
- **Tipos de datos y enumeraciones:** Con el objetivo de garantizar la integridad de los datos y la robustez del sistema, se han aplicado los siguientes elementos de modelado:
 - **Enumeraciones (Estados y Modos):** Restringen los estados de topologías y nodos a valores válidos, así como los modos de operación de las interfaces, asegurando un comportamiento predecible por el motor de orquestación.
 - **Datatypes (Tipos de datos personalizados):** Se ha definido un tipo de dato específico para la gestión temporal (Hora/Timestamp). Esto asegura que los registros de creación de topologías, de creación de usuarios y marcas de tiempo de los logs sigan un formato homogéneo y preciso, facilitando la correlación de eventos durante la depuración.
- **Identificadores:** Finalmente, todas las clases disponen de un identificador único que permite su persistencia en la base de datos, facilitando las acciones de consulta, modificación o borrado de los recursos a través de la API del sistema.

Estas entidades conforman una estructura jerárquica donde el Usuario es el propietario de múltiples Topologías, las cuales actúan como contenedores lógicos de Nodos e Interfaces, interconectados entre sí por Enlaces.

4.5. Diagramas de secuencia

Para facilitar la comprensión técnica del sistema, se han modelado las interacciones más significativas mediante diagramas de secuencia. A diferencia de la vista estática proporcionada por los casos de uso, los diagramas de secuencia muestran de forma dinámica las llamadas a la API, las validaciones de seguridad y las operaciones de persistencia. Se detallan a continuación los escenarios de autenticación de usuarios, el proceso de orquestación y despliegue de la topología, la monitorización de los nodos y la consulta de sus logs de actividad para la depuración del sistema.

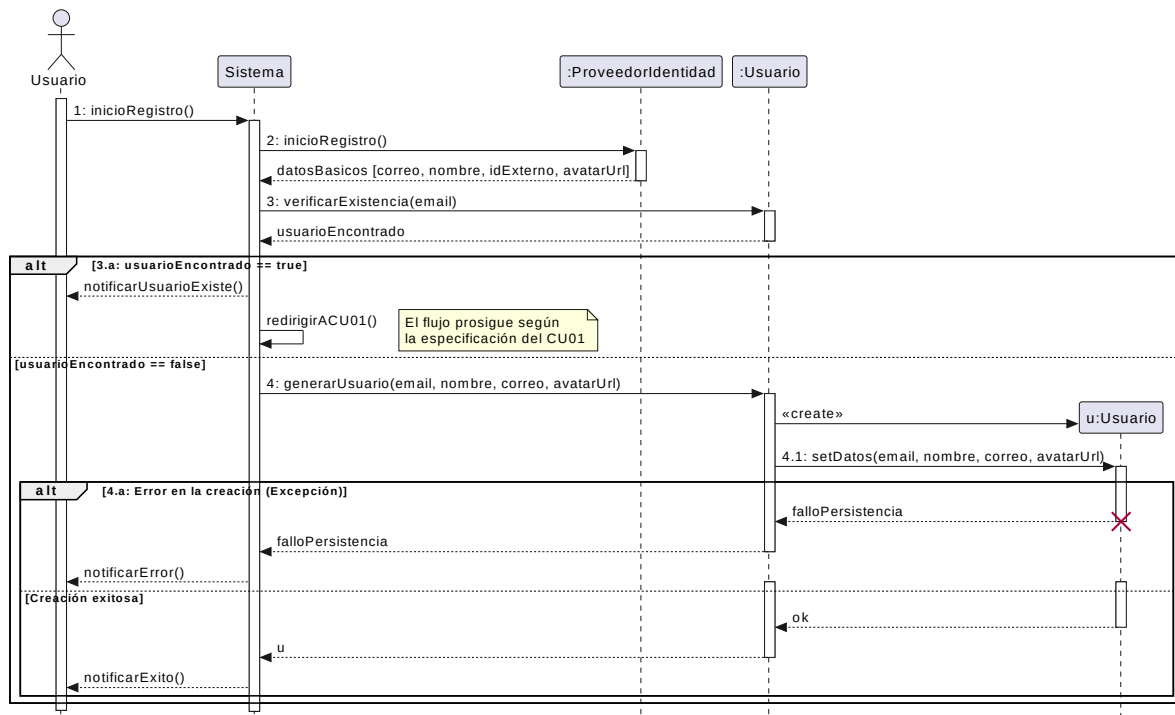


Figura 4.3: Diagrama de secuencia del CU02: Registrar Usuario

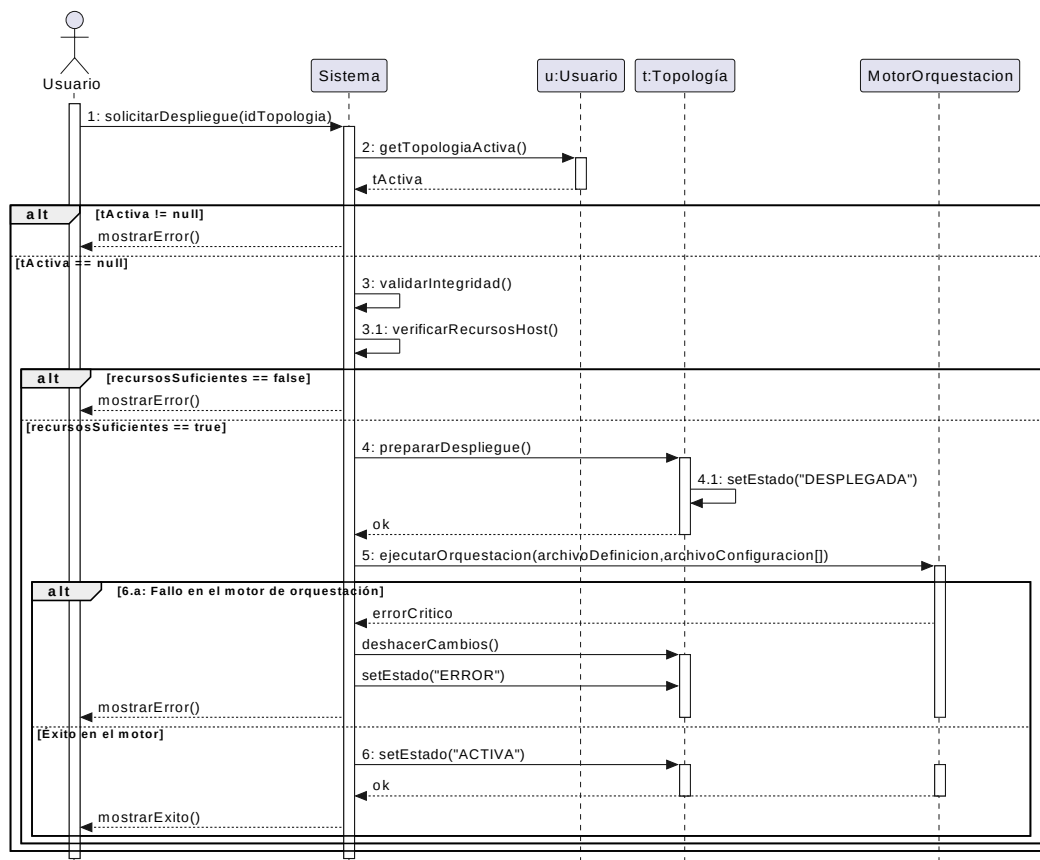


Figura 4.4: Diagrama de secuencia del CU09: Desplegar Topología

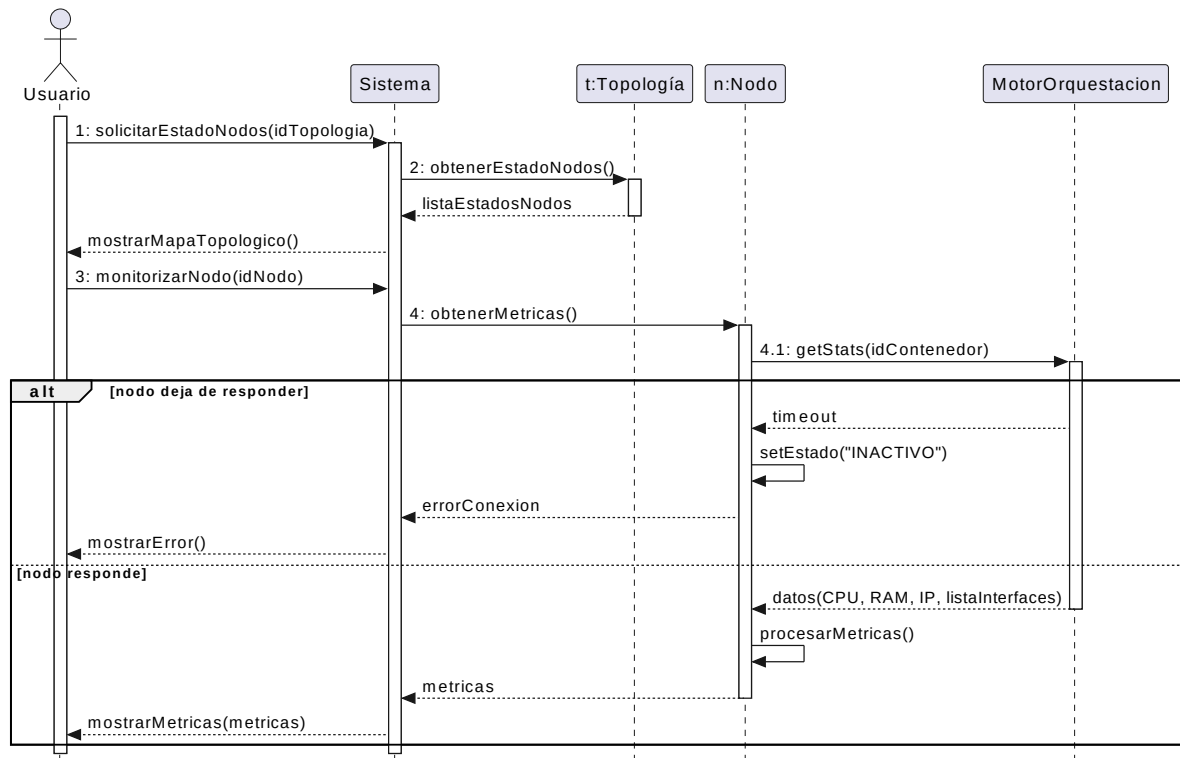


Figura 4.5: Diagrama de secuencia del CU10: Monitorizar Nodos

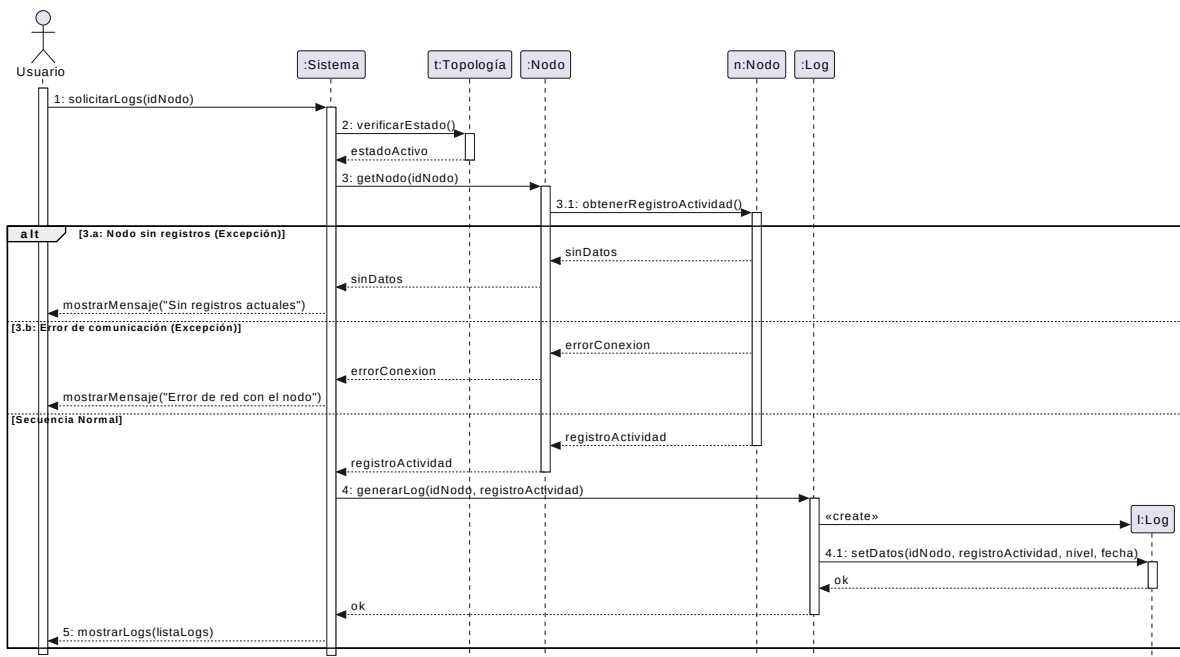


Figura 4.6: Diagrama de secuencia del CU13: Consultar logs de actividad

Capítulo 5

Diseño del sistema

En este capítulo se detalla el proceso de diseño técnico y conceptual del sistema desarrollado, cuyo principal objetivo es trasladar los requisitos analizados previamente en una arquitectura de software escalable, sólida y eficiente. El capítulo se estructura jerárquicamente, comenzando por la arquitectura general del sistema que define la organización modular y los patrones de diseño aplicados para la separación de responsabilidades. Posteriormente, se detalla el modelado de la estructura de la base de datos orientada a la persistencia. A continuación, se profundiza en los patrones y decisiones de diseño clave que guían la lógica del sistema, para concluir con la especificación de la interfaz de usuario y la estrategia de despliegue planteada. Este conjunto de especificaciones servirá como marco de referencia para la fase de implementación.

5.1. Arquitectura del Sistema

La arquitectura del sistema ha sido diseñada siguiendo un enfoque modular, orientada a garantizar la escalabilidad, la mantenibilidad y la separación clara de responsabilidades. Esta estructura se fundamenta en una organización por dominios funcionales, donde cada componente posee una finalidad definida, permitiendo una gestión eficiente de las dependencias. A continuación, se detalla la organización de los componentes del sistema mediante un diagrama de paquetes, que define las responsabilidades de las capas de presentación, aplicación y persistencia. Asimismo, se expone el patrón arquitectónico MVVM adoptado para el frontend, cuya implementación asegura una separación clara entre la interfaz de usuario y la lógica de negocio.

5.1.1. Diagrama de paquetes

En la Figura 5.1 se presenta el diagrama de paquetes del sistema, el cual formaliza la organización modular y las dependencias entre los componentes de software desarrollados. Con el objetivo de garantizar un diseño escalable y facilitar el mantenimiento, la arquitectura se estructura siguiendo una separación lógica de responsabilidades, agrupando el código en dominios de funcionalidad claramente

definidos:

- **Capa de Presentación (ui):** Representa la interfaz con la que interactúa el usuario. Agrupa los componentes visuales de la aplicación, desarrollados en React, responsables de renderizar las vistas y gestionar la experiencia del usuario. Incluye la lógica de la autenticación en el lado del cliente, apoyada en el proveedor de identidad de Google. Esta capa no contiene la lógica de negocio y se comunica con la capa de aplicación exclusivamente mediante peticiones a la API HTTP.
- **Capa de Aplicación (api - servicios):** Constituye el núcleo lógico del sistema, implementado en Python mediante el framework *FastAPI*. Expone los endpoints que consume la capa de presentación y centraliza el procesamiento de las solicitudes. Dentro de esta capa reside también el módulo de gestión de topologías, responsable de coordinar el ciclo de vida de las topologías: creación, despliegue, monitorización y eliminación. Este módulo se encarga de gestionar los logs de despliegue, determinando su nomenclatura y persistencia en el sistema de archivos a partir de la salida generada por *Containerlab*.
- **Capa de Persistencia (datos - almacenamiento):** Agrupa todos los mecanismos de almacenamiento del sistema. Comprende la base de datos relacional MySQL, que consolida de forma persistente los registros de usuarios y el estado de las topologías; las plantillas predefinidas, que proporcionan las topologías disponibles para el usuario; los ficheros de topología, que contienen las definiciones YAML de cada topología y existen con independencia de si la topología ha sido desplegada o no; los avatares de usuario, almacenados como ficheros en el sistema de archivos; y los logs de despliegue, generados y gestionados por el orquestador durante el despliegue de cada topología.

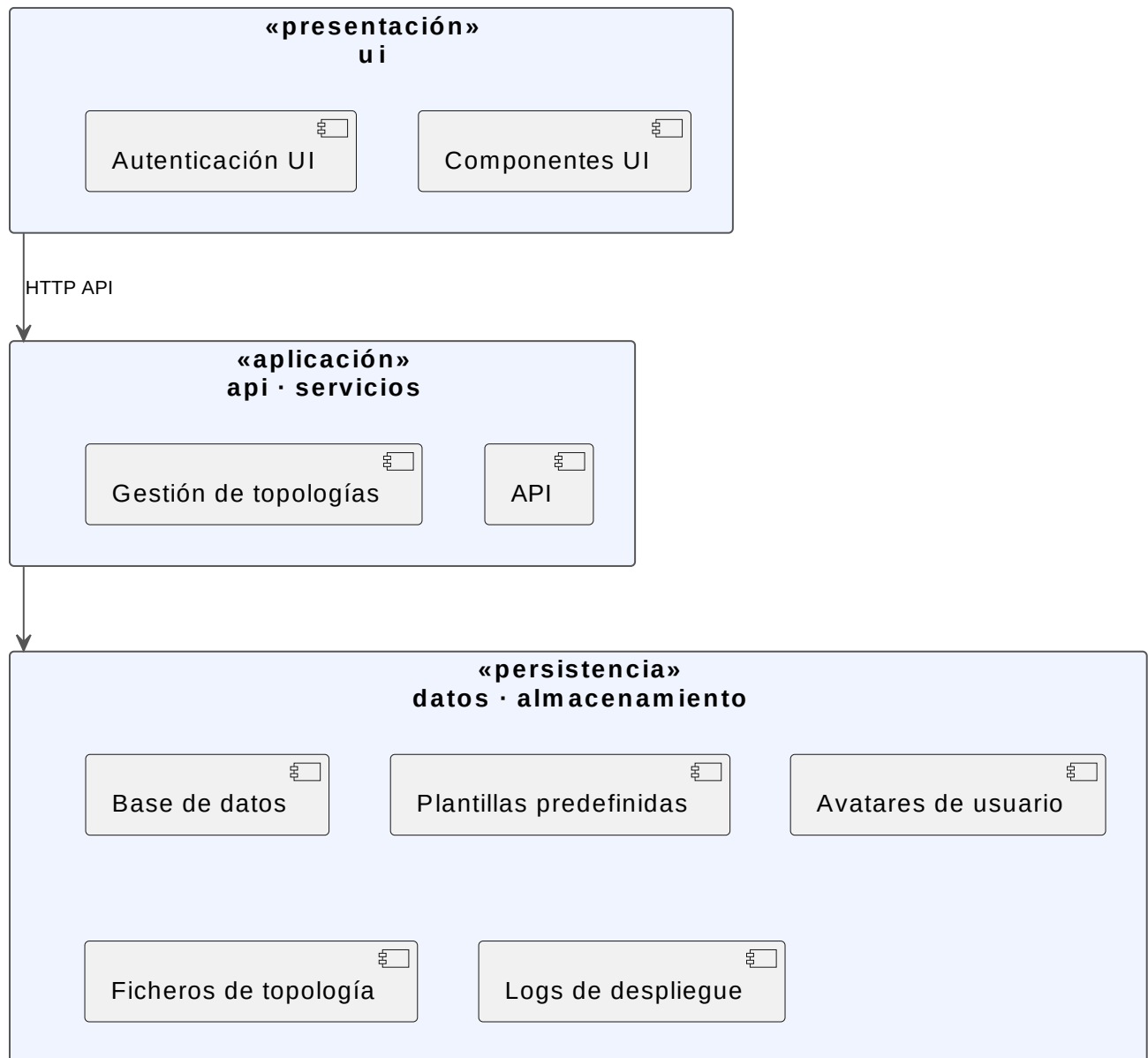


Figura 5.1: Diagrama de paquetes del sistema

5.1.2. Patrón arquitectónico MVVM

Para la estructuración del frontend, se ha adoptado el patrón arquitectónico MVVM (Model-View-ViewModel). Este patrón permite una separación clara entre la interfaz de usuario y la lógica de negocio, facilitando el desarrollo paralelo y la reutilización de componentes [96].

La implementación se divide en tres capas fundamentales:

- **Modelo (Model)**: Representa la capa de datos y la lógica de negocio. En este proyecto, el modelo está construido por las estructuras de datos que definen las topologías, los estados de los nodos y la información del usuario obtenida a través de la API.
- **Vista (View)**: Es la interfaz gráfica con la que interactúa el usuario. Se ha desarrollado mediante

componentes reactivos que observan los cambios en el ViewModel para actualizarse automáticamente sin necesidad de recargar la página.

- **Vista-Modelo (View-Model):** Actúa como el intermediario entre la Vista y el Modelo. Su función principal es gestionar el estado final de la aplicación (mediante manejadores de eventos y *Hooks* de React como *useState* y *useEffect*), transformando la información del backend en datos listos para ser visualizados.

La principal ventaja de este patrón en el proyecto es la vinculación de datos. Gracias a esto, cuando el orquestador actualiza el estado de un contenedor Docker en el servidor, el cambio fluye a través del Modelo hacia el ViewModel, y la Vista se actualiza de forma reactiva, permitiendo que el usuario vea el progreso del despliegue en tiempo real de manera permanente.

Aunque el patrón Model-View-Controller (MVC) ha sido el estándar tradicional para el desarrollo web, para la creación de la interfaz de usuario de este proyecto se ha optado por el patrón Model-View-ViewModel (MVVM). A continuación, se detallan los motivos de esta elección:

La elección de MVVM frente a patrones tradicionales como MVC se fundamenta en los requisitos técnicos específicos de esta aplicación:

- **Reactividad:** El requisito RF09 exige que la interfaz se actualice constantemente con los logs del orquestador. MVVM, con la vinculación de datos, permite que la Vista reaccione a los cambios de estado (ViewModel) sin intervención manual o sin necesidad de recargar la página.
- **Complejidad del estado del frontend:** Al gestionar topologías de red dinámicas, el estado de la interfaz es complejo (nodos activos, conexiones, errores). El ViewModel permite gestionar esta lógica de forma aislada, manteniendo la Vista limpia y centrada únicamente en la representación visual.
- **Experiencia de usuario:** MVVM permite una navegación fluida y rápida, propia de una *Single Page Application (SPA)*. En un entorno técnico de orquestación, es vital que la interfaz responda con agilidad a las acciones del usuario sobre los contenedores Docker.

Por tanto, mientras que MVC hubiera resultado en una arquitectura más rígida y dependiente de peticiones constantes al servidor, MVVM proporciona la flexibilidad necesaria para una herramienta de monitorización y gestión de infraestructura virtualizada moderna.

5.2. Estructura de la base de datos

Se ha implementado una base de datos relacional, cuyo objetivo es almacenar de forma estructurada las topologías de red, la configuración de los nodos y los registros de actividad.

A continuación, se detalla la estructura lógica del modelo relacional, que puede verse gráficamente en la Figura 5.2:

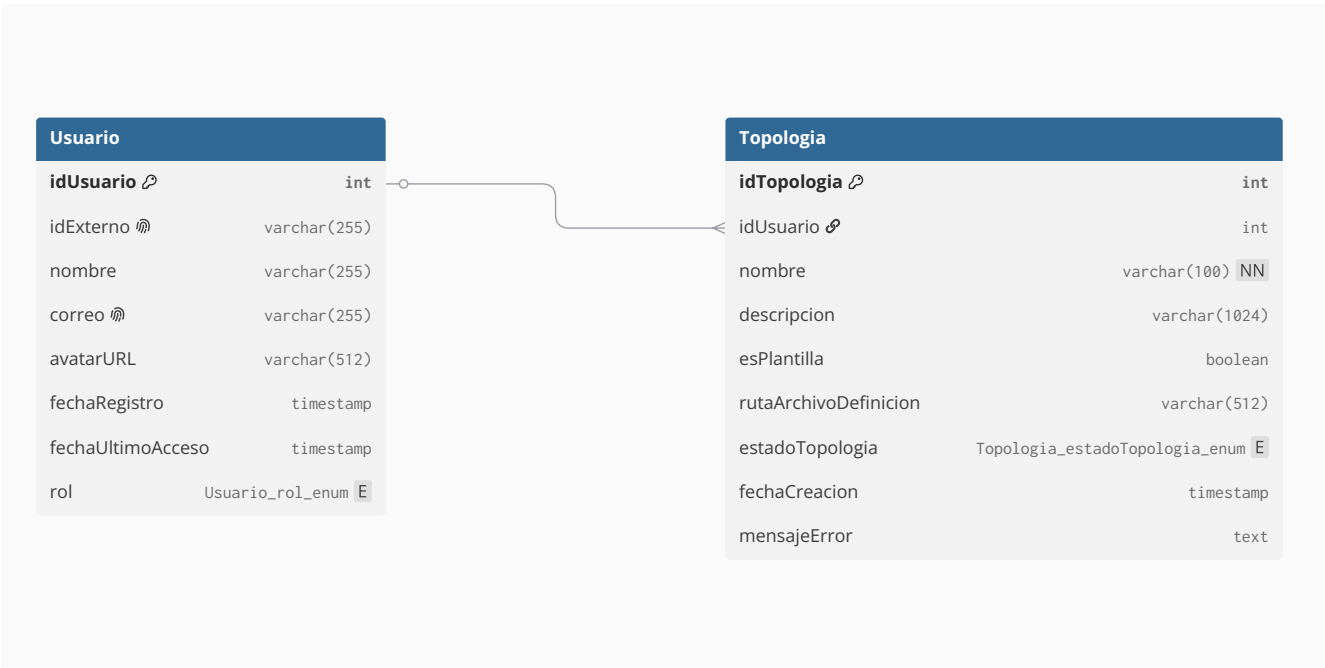


Figura 5.2: Estructura de la base de datos

- **Gestión de Identidad (*Usuario*):** Esta tabla almacena la información esencial de los usuarios del sistema, incluyendo sus identificadores externos (*idExterno*), datos de perfil (*nombre*, *correo*, *avatarURL*), fechas de registro y acceso, así como el *rol* asignado (mediante un tipo enumerado), lo cual permite un control de acceso centralizado.
- **Definición de Topologías (*Topología*):** Representa el núcleo del sistema. Vincula cada diseño de red con un usuario específico (*idUsuario*) y almacena metadatos críticos como el nombre, descripción, una bandera booleana para diferenciar entre plantillas y redes del usuario (*esPlantilla*), la *rutaArchivoDefinicion* y el estado actual del despliegue (*estadoTopología*). Asimismo, incluye un campo para capturar mensajes de error en casos de fallo durante el despliegue de la red.

5.3. Integridad y Restricciones de Sistema

Para asegurar la consistencia del sistema en escenarios complejos de orquestación, se han definido las siguientes restricciones en el esquema físico:

- **Gestión del ciclo de vida:** Se ha configurado la propiedad *ON DELETE CASCADE* en las relaciones de dependencia. De esta forma, si una topología es eliminada, todos los recursos asociados se borran automáticamente para evitar la existencia de datos huérfanos en el sistema.
- **Unicidad y Seguridad:** Atributos clave como el correo electrónico del usuario están protegidos mediante restricciones de unicidad (*unique*), asegurando la integridad del sistema y evitando colisiones lógicas en el acceso administrativo.

- **Persistencia y abstracción:** El sistema hace uso de MySQL como motor de persistencia debido a su fiabilidad en la gestión de relaciones complejas. La integración con el backend se realiza a través de una capa de abstracción de datos, que permite realizar operaciones de consulta y actualización de forma estructurada, garantizando la integridad de la información en todo momento.

5.4. Patrones de diseño

Los patrones de diseño representan soluciones estandarizadas y reutilizables a problemas recurrentes que surgen durante la fase de diseño y desarrollo de software. Estos patrones funcionan como planos preestablecidos que se pueden personalizar para resolver problemas de diseño específicos en el código, permitiendo que el software sea más flexible y reutilizable [97]. Su aplicación en este proyecto busca garantizar que el sistema mantenga una estructura robusta y profesional.

Al adoptar estos patrones, se establece un lenguaje común que optimiza la legibilidad y la eficiencia. La implementación de patrones de diseño es una práctica esencial para abordar la complejidad de los sistemas modernos. Se destaca la importancia de la separación de preocupaciones entre las capas de presentación y la lógica de negocio para mejorar la mantenibilidad y escalabilidad del software [98]. A continuación, se detallan los patrones específicos que conforman la arquitectura lógica del sistema y su implementación en el proyecto.

5.4.1. Patrón de Inyección de Dependencias

El patrón de inyección de dependencias es un patrón de diseño que permite la creación de sistemas débilmente acoplados. En lugar de que un componente instancie internamente sus propias dependencias (como conexiones a bases de datos o servicios de validación), estas le son provistas desde el exterior. Esto facilita el mantenimiento, la modularidad y la realización de pruebas unitarias [99]. En este proyecto, la inyección de dependencias constituye el núcleo de la arquitectura del backend y se implementa de forma nativa a través del framework FastAPI. Se aplica principalmente en dos flujos críticos del sistema:

- **Gestión de la base de datos:** A través de la dependencia *get_db()*, el sistema inyecta la conexión a MySQL directamente en los endpoints que la requieren. Esto delega en el framework la responsabilidad de iniciar la conexión y, lo que es más importante, garantiza su cierre seguro al finalizar la petición, evitando fugas de memoria.
- **Seguridad y Autenticación:** La validación de las identidades se realiza inyectando la función *verificar_token()* en las rutas protegidas. De esta forma, el código de cada endpoint se mantiene limpio y centrado exclusivamente en su lógica de negocio, asumiendo que si la ejecución ha llegado a ese punto, la dependencia ya ha validado la firma del token JWT de manera exitosa.

La adopción de este patrón elimina la necesidad de variables globales y de código repetitivo de inicialización, resultando en una API robusta, escalable y con una separación de responsabilidades estricta.

5.4.2. Patrón DTO

El patrón DTO (Data Transfer Object) consiste en la creación de objetos cuya finalidad es el transporte de datos entre procesos, habitualmente entre la interfaz de usuario (frontend) y el servidor (backend). El uso de DTOs permite agrupar múltiples datos en una sola estructura, reduciendo el número de llamadas remotas y optimizando la comunicación en sistemas distribuidos [100]. En este proyecto, el patrón DTO se aplica para resolver dos cuestiones fundamentales:

- **Seguridad y privacidad de los datos:** Al interactuar con la base de datos, el sistema obtiene entidades completas que pueden contener información sensible o innecesaria para el cliente (como identificadores internos o metadatos técnicos). Mediante el uso de DTOs, se realiza un filtrado, enviando a la interfaz únicamente los atributos requeridos por el usuario, como el nombre de la topología o el estado de los nodos.
- **Desacoplamiento de las capas:** El DTO se establece como una capa de abstracción. Si la estructura de la base de datos cambia, el DTO puede mantenerse inalterado, evitando que los cambios en el backend afecten directamente al frontend y viceversa. Esto proporciona una gran flexibilidad durante el desarrollo y facilita el mantenimiento a largo plazo.

Un ejemplo claro de su uso en el sistema es la gestión de los usuarios tras el proceso de autenticación. En lugar de manejar el objeto completo devuelto por el proveedor de identidad (el cual contiene metadatos internos, configuraciones del proveedor y campos irrelevantes para la interfaz), el backend genera un DTO simplificado que incluye únicamente el nombre, el correo electrónico y el token de sesión, optimizando de esta manera la información que consume la aplicación cliente.

5.4.3. Patrón Observer

El patrón Observer es un patrón de diseño de comportamiento que define una dependencia de uno a muchos entre objetos, de forma que cuando un objeto (sujeto) cambia su estado, todos sus dependientes (observadores) son notificados y actualizados automáticamente [101]. Este patrón es esencial en aplicaciones dirigidas por eventos y sistemas que requieran una sincronización constante entre la lógica de negocio y la interfaz de usuario.

En el contexto de este proyecto, la implementación del patrón Observer es fundamental para la gestión de la consola de eventos. El proceso de despliegue de una topología de red es una tarea asíncrona que genera un flujo constante de información (logs de Docker, estados de las interfaces, errores de red).

La arquitectura se articula de la siguiente manera:

- **Sujeto (Observable):** El motor de orquestación actúa como el sujeto. Este componente supervisa el ciclo de vida de los contenedores y los eventos del sistema.
- **Observador:** La interfaz web (frontend) actúa como el observador. Mantiene una conexión activa con los canales de eventos del servidor, lo que garantiza que cualquier cambio en el estado de la orquestación se refleje de manera reactiva en la interfaz de usuario de manera instantánea.

Este patrón permite que, en lugar de que la interfaz web realice peticiones constantes al servidor para preguntar por el estado del despliegue, sea el propio motor de orquestación quien envíe la información hacia el usuario en el momento en el que ocurra. Esto optimiza el uso de los recursos de red y garantiza una experiencia de usuario fluida, permitiendo visualizar el proceso de despliegue de los nodos de forma instantánea.

5.5. Decisiones de diseño

5.5.1. Entorno de desarrollo: React y Vite

Para el desarrollo de la interfaz de usuario se ha seleccionado *React*¹, una biblioteca de JavaScript de código abierto orientada a la creación de interfaces de usuario en su versión más reciente, utilizando Vite (un entorno de desarrollo y herramienta que optimiza tiempos de compilación) como herramienta de construcción. Esta elección se fundamenta en la necesidad de un alto rendimiento y una recarga modular rápida. A diferencia de las estructuras monolíticas, se ha optado por un diseño basado en componentes funcionales y *Hooks* (funciones especiales de React que permiten enlazar el estado global y los ciclos de vida de los componentes sin necesidad de instanciar clases), permitiendo una gestión eficiente, centralizada y reactiva del estado dinámico de los contenedores Docker y nodos de red dentro del frontend.

5.5.2. Tailwind CSS

En lugar de bibliotecas de componentes predefinidos rígidos, se ha decidido utilizar Tailwind CSS, un framework de CSS orientado a utilidades. Esta elección permite un control total sobre el diseño sin salir del archivo HTML/JSX, lo que resulta en un desarrollo más ágil y un código final más ligero. Las ventajas clave para el orquestador son:

- **Diseño centrado en datos:** Permite crear interfaces densas en información (tablas de nodos, estados de la API) de forma clara y organizada.
- **Rendimiento optimizado:** Al generar solo el CSS que utiliza, la carga de la plataforma web es casi instantánea, algo crítico en herramientas de gestión de infraestructura.
- **Estética “Dark Mode” nativa:** Facilita la implementación de una interfaz oscura, la cual es ampliamente preferida en entornos profesionales de redes y monitorización. Este enfoque reduce la luminosidad y el deslumbramiento en condiciones de baja luminosidad, alineándose con las preferencias del usuario del sector técnico.
- **Total adaptabilidad:** El diseño es completamente *responsive*, asegurando que en futuras iteraciones, el orquestador pueda ser visualizado desde dispositivos móviles sin necesidad de reescritura de código, facilitando la monitorización remota en caso de emergencia.

¹<https://react.dev/>

5.5.3. Identidad Visual: Nomenclatura, Logo, Tipografía y Paleta de Colores

Antes de definir los componentes visuales de la interfaz, es necesario justificar la nomenclatura de la plataforma. El nombre “S2 Orchestrator” surge a partir de la combinación de las iniciales estilizadas del autor (“S2”, en referencia al apellido San Segundo) junto con el término “Orchestrator”, elegido debido a la naturaleza y propósito general del sistema: actuar como un motor centralizado de orquestación, automatización y gestión de la infraestructura de red.

- **Logo:** Como se puede ver en la Figura 5.3, el logotipo de S2 Orchestrator presenta una estructura geométrica basada en cubos interconectados mediante líneas de red. Esta composición simboliza la naturaleza del proyecto: la orquestación de infraestructuras (representadas por los cubos o contenedores) y la gestión de sus conexiones lógicas. El uso de las líneas cinéticas y los puntos de conexión refuerza la idea de una red dinámica, escalable y centralizada.
- **Tipografía:** Para la interfaz del orquestador se ha implementado una combinación estratégica de tipografías con el objetivo de diferenciar visualmente la navegación de los datos técnicos. Como fuente principal se utiliza Lato, una tipografía Sans-Serif disponible a través de Google Fonts, reconocida por su claridad y accesibilidad, lo que facilita la lectura en dispositivos de todos los tamaños. Por otro lado, para la representación de información crítica de red, como direcciones IP, comandos de consola, registros del sistema y los identificadores de los nodos, se utiliza una familia tipográfica monoespaciada (combinando Courier New y fuentes nativas del sistema). Esta elección combinada añade un toque moderno a la interfaz y asegura la alineación perfecta de caracteres, garantizando una legibilidad óptima y evitando errores de transcripción en entornos de monitorización industrial.
- **Paleta de Colores:** La identidad visual se apoya en una combinación de alto contraste, diseñada específicamente para entornos de ingeniería y gestión de redes. El objetivo es proporcionar una interfaz clara que minimice la fatiga visual durante periodos prologados de monitorización.
 - **Azul Marino Oscuro (#0D2333):** Se utiliza como color de fondo principal. Este tono proporciona una estética sobria y profesional, propia de software de infraestructura.
 - **Azul Cian Eléctrico (#22B5E2):** Es el color de contraste principal. Se emplea para resaltar nodos, iconos y estados activos, facilitando la identificación rápida de elementos operativos.
 - **Blanco Puro (#FFFFFF):** Reservado para la tipografía principal y títulos, garantizando la máxima legibilidad sobre el fondo oscuro.
 - **Gris (#4B5563):** Utilizado en textos secundarios, leyendas y bordes de componentes para jerarquizar la información de manera sutil.

En la Figura 5.4 se muestran concretamente los tonos que se van a utilizar.



Figura 5.3: Logo de la interfaz de usuario

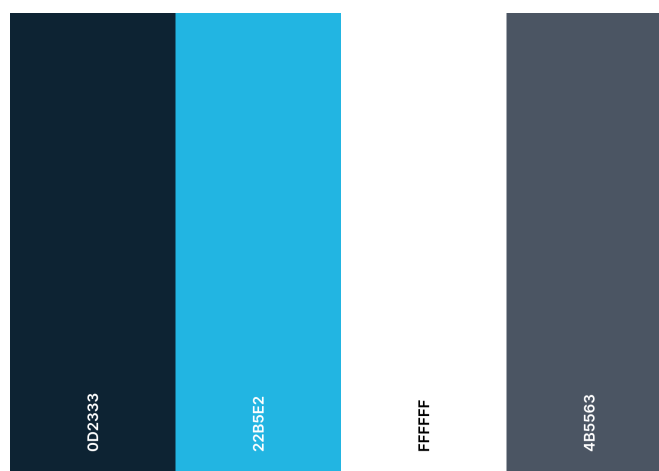


Figura 5.4: Paleta de colores

5.5.4. Control de acceso y gestión de identidades: Google Auth

Para el control de acceso y gestión de los usuarios, se ha tomado la decisión de implementar un modelo de autenticación basado en el protocolo OAuth 2.0 a través del proveedor de identidad Google Auth.

La selección de Google Auth como proveedor de identidad frente a alternativas robustas como Github o Microsoft Entra ID o soluciones autoalojadas se debe a un criterio de facilidad de despliegue y máxima cobertura. Github es una opción excelente para entornos de desarrollo, sin embargo, una gran parte del público objetivo no dispone de un perfil activo en dicha plataforma, lo que hubiera requerido registros previos. Por otro lado, aunque Microsoft cuenta con una gran presencia institucional, la complejidad de su API [102] y las restricciones de algunas de sus políticas de privacidad a cuentas educativas suelen dificultar las integraciones de proyectos independientes. Finalmente, descartar soluciones autoalojadas evitó añadir la carga administrativa y el consumo de recursos en el servidor Linux que supondría mantener una base de datos y un servicio de identidad adicionales operativos las 24 horas.

En lugar de construir un sistema local tradicional que requiere de un almacenamiento y cifrado de contraseñas en la base de datos MySQL, esta elección de diseño nos aporta las siguientes ventajas

arquitectónicas:

- **Mitigación de ataques cibernéticos:** Al delegar la gestión de las identidades a la infraestructura global de Google, el orquestador reduce los riesgos asociados a la filtración de las contraseñas o ataques de fuerza bruta en el servidor del proyecto.
- **Mecanismos de seguridad avanzados heredados:** La integración permite que el sistema se beneficie de factores de protección avanzados gestionados por el proveedor, como la autenticación en dos pasos (2FA), la verificación biométrica y la detección de inicios de sesión sospechosos, cuya implementación propia habría aumentado los tiempos de desarrollo del proyecto.

El frontend en React actúa como el iniciador del flujo de autenticación, delegando al backend el intercambio seguro del código de autorización con el proveedor por el token de identidad y la posterior validación del mismo. Una vez validado, se transfiere al backend en FastAPI, el cual verifica su identidad, registra o localiza al usuario en la base de datos y emite un token de sesión seguro en formato JWT (JSON Web Token) almacenado en las cookies del navegador.

5.5.5. Mecanismo de seguridad en la eliminación: Confirmación por entrada de texto

Para garantizar la integridad del sistema y prevenir la eliminación accidental de topologías, se ha optado por implementar un mecanismo de confirmación mediante entrada de texto, sustituyendo a los modelos tradicionales de confirmación binaria (ventanas modales con botones "Sí/No").

La decisión de abandonar el diálogo modal se basa en que estos elementos, aunque comunes, son susceptibles de ser aceptados de forma automática e irreflexiva por parte del usuario, lo que aumenta el riesgo de ejecuciones erróneas ante un clic impulsivo. En su lugar, el sistema obliga al usuario a escribir manualmente el nombre exacto de la topología que desea borrar para habilitar el botón de confirmación de la acción.

Esta implementación aporta las siguientes ventajas:

- **Freno cognitivo:** La acción de escribir el nombre, requiere una pausa, lo que fuerza al usuario a leer y confirmar que está actuando sobre el recurso correcto antes de proceder.
- **Verificación de identidad:** Al requerir una coincidencia exacta con el nombre del recurso, se elimina la posibilidad de eliminar una topología distinta a la que el usuario tenía en mente, aumentando la precisión en entornos de trabajo con múltiples topologías.
- **Seguridad operativa:** Esta capa de validación refuerza los controles de seguridad del *backend*, garantizando que el borrado de una topología sea siempre un proceso intencionado y validado por el usuario.

5.6. Especificación de la interfaz de usuario

5.6.1. Criterios de diseño y usabilidad

En lugar de basar el diseño en un sistema cerrado de terceros, se ha optado por un enfoque centrado en la usabilidad lógica y la experiencia del usuario (UX). El objetivo principal ha sido crear una interfaz funcional, intuitiva y coherente fundamentada en los siguientes principios:

- **Simplicidad y minimalismo:** Se ha seguido la premisa de "menos es más"[103]. La interfaz evita elementos innecesarios para que el usuario pueda enfocarse exclusivamente en las tareas principales, reduciendo de esta manera la carga cognitiva.
- **Convenciones de usuario:** Aunque no se ha seguido una guía de estilo específica, se han respetado los patrones de diseño estándar de la industria. Esto incluye el uso de iconos universales (como el engranaje para ajustes o la basura para eliminar) y la ubicación lógica de menús para que la aplicación resulte familiar desde el primer contacto.
- **Jerarquía visual:** Se ha trabajado con el contraste de tamaños y pesos tipográficos para guiar al usuario hacia las acciones más importantes, asegurando que los botones principales sean fácilmente identificables.

5.6.2. Prototipado y definición de la interfaz

Para la construcción de la interfaz se ha seguido un modelo de diseño iterativo e incremental. Esto ha permitido realizar ajustes sobre la marcha basándose en la funcionalidad necesaria de cada módulo del sistema.

- **Esquemas iniciales:** Se definieron los flujos de navegación para asegurar que el número de pasos para realizar una acción sea el mínimo posible.
- **Interfaz final:** El resultado es una estética limpia que prioriza la legibilidad. Se ha optado por una paleta de colores representada en la Fig. 5.4 que garantiza un buen contraste y una buena visibilidad en diferentes condiciones de luminosidad.

5.6.3. Diseño de vistas y catálogo de pantallas

En esta sección se presentan los bocetos finales de la interfaz, los cuales son el resultado de aplicar los criterios de usabilidad mencionados anteriormente.

Bloque de acceso y gestión de cuenta

Este primer bloque comprende la pantalla de acceso a la plataforma y la administración básica del perfil del usuario:

- **Panel de inicio de sesión:** En la Figura 5.5 se ilustra la pantalla de acceso, la cual cuenta con indicadores en tiempo real del estado de la *API* y del motor de ejecución. Esta interfaz implementa un esquema centralizado que delega la seguridad en el botón de autenticación federada de *Google Auth*.
- **Gestión de perfil:** Una vez dentro del sistema, la Figura 5.6 muestra el panel de configuración del perfil del usuario. Esta vista proporciona una interfaz interactiva que permite al usuario modificar su nombre completo y actualizar su avatar para personalizar su identidad en el sistema. Asimismo, se muestra de forma informativa el correo electrónico vinculado y el rol operativo asignado de manera estática por el administrador del sistema.

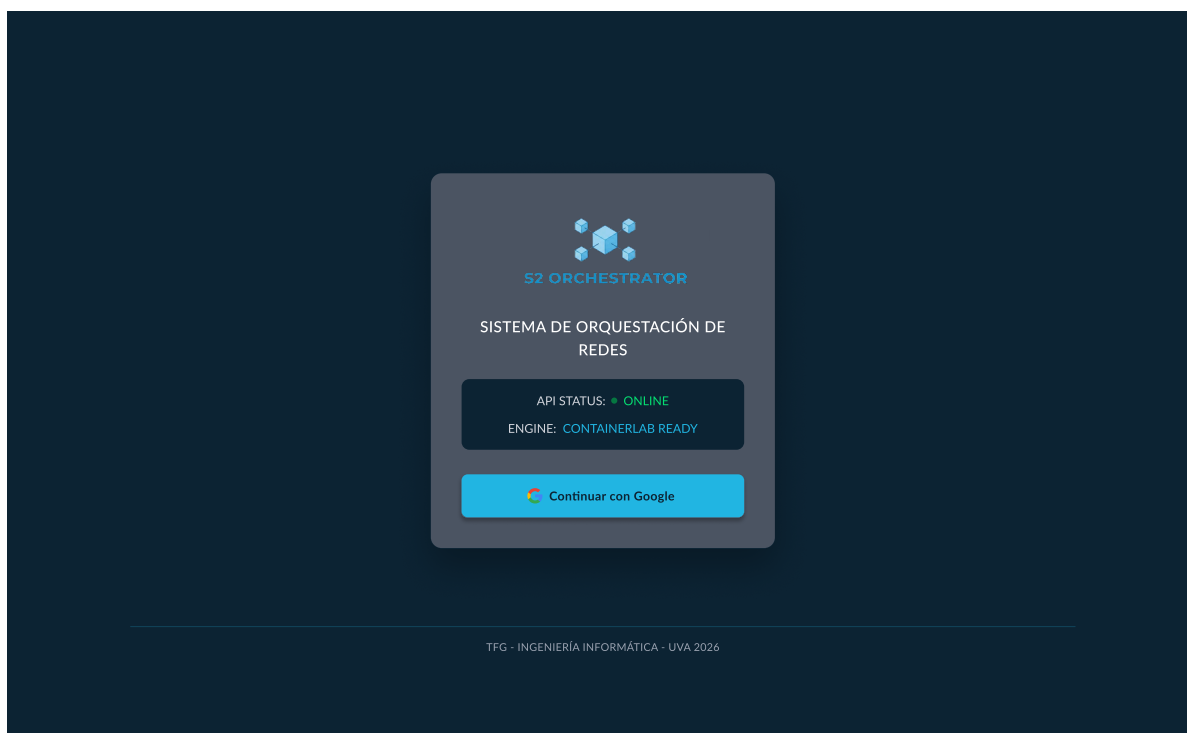


Figura 5.5: Vista de inicio de sesión

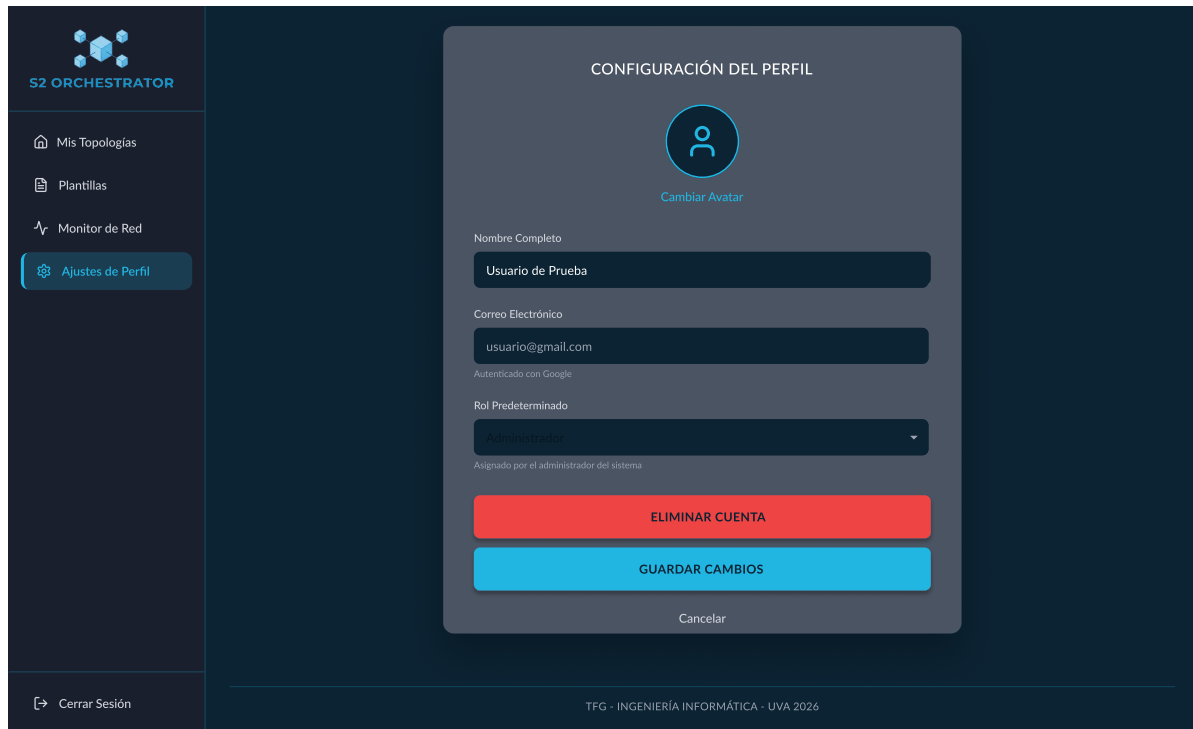


Figura 5.6: Vista de gestión del perfil

Bloque de administración de topologías de red

Este bloque constituye el núcleo de gestión documental del sistema, permitiendo administrar los escenarios virtuales tras su despliegue:

- **Biblioteca de plantillas:** Reflejada en la Figura 5.7, ofrece un catálogo con escenarios preconfigurados comunes para facilitar un despliegue rápido. Cada plantilla de la biblioteca integra un botón operativo que permite al usuario seleccionar dicho diseño e importarlo de forma automática a su espacio de trabajo personal.
- **Panel de topologías del usuario:** La Figura 5.8 expone el listado de proyectos propios guardados por el usuario acompañados de etiquetas del estado operativo (*Activo*, *Almacenado*, *En Despliegue*, *En Destrucción* o *Error*). Para optimizar la experiencia del usuario y facilitar la localización del proyecto en escenarios masivos, la vista incorpora una barra de búsqueda dinámica para filtrar topologías concretas por su nombre. Asimismo, incluye un componente de ordenación que permite organizar el listado tanto por orden alfabético como por fecha de creación, soportando criterios tanto ascendentes como descendentes. Por último, la vista incorpora un botón que, al ser pulsado, despliega la vista para importar una nueva topología en el sistema.
- **Formulario de creación:** En la Figura 5.9 se detalla la interfaz para cargar nuevos escenarios mediante la subida de archivos de configuración en formato YAML.
- **Confirmación de borrado:** Como mecanismo de seguridad ante acciones críticas e irreversibles, la Figura 5.10 muestra la vista que obliga al usuario a escribir explícitamente el nombre de la

topología para autorizar su eliminación completa.

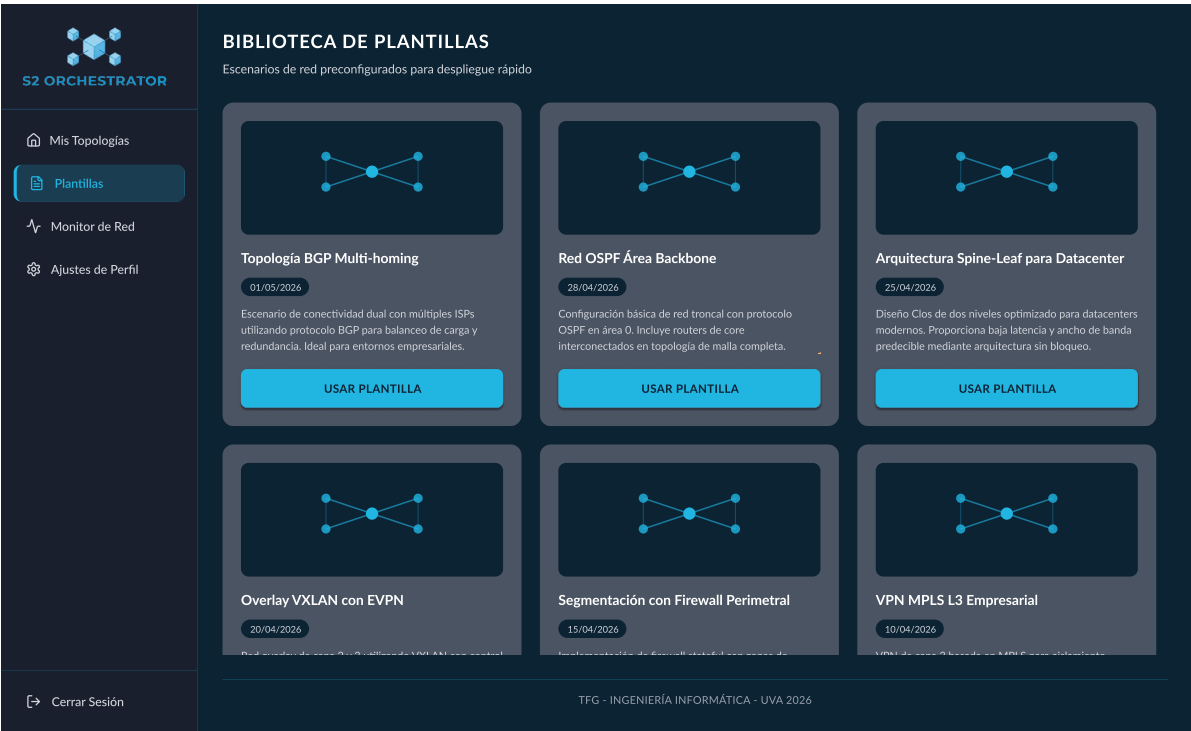


Figura 5.7: Vista de la selección de plantillas

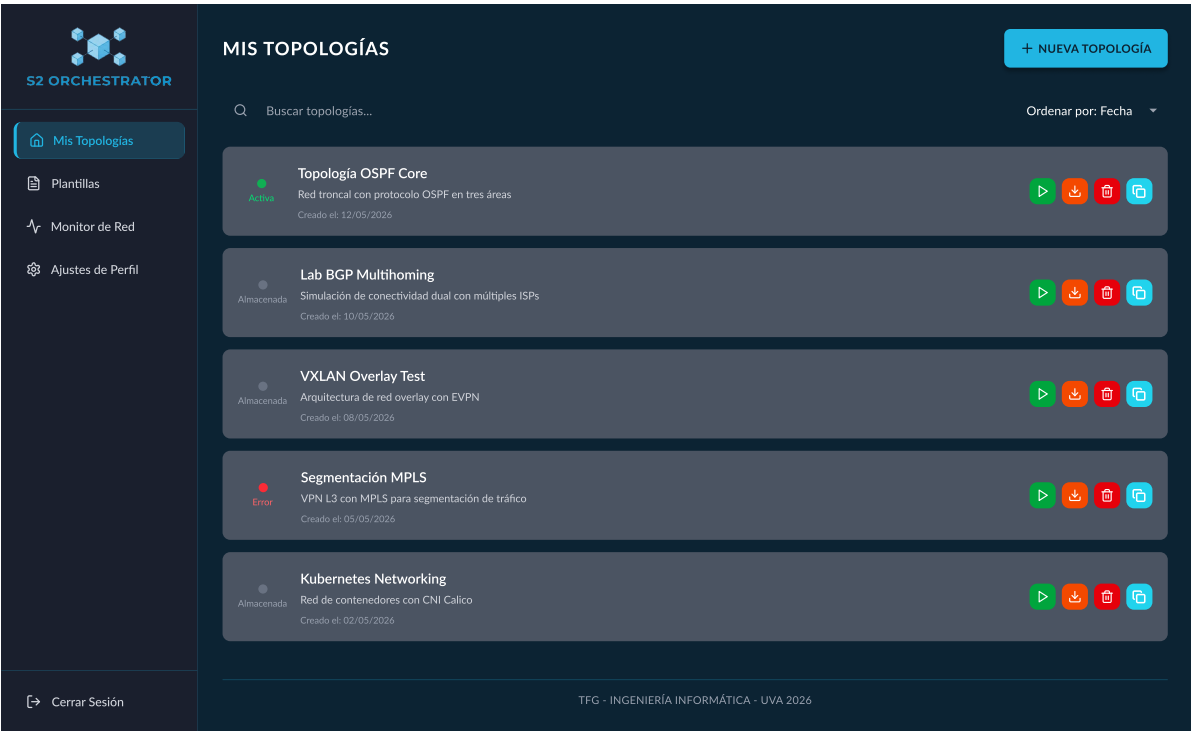


Figura 5.8: Vista de las topologías del usuario

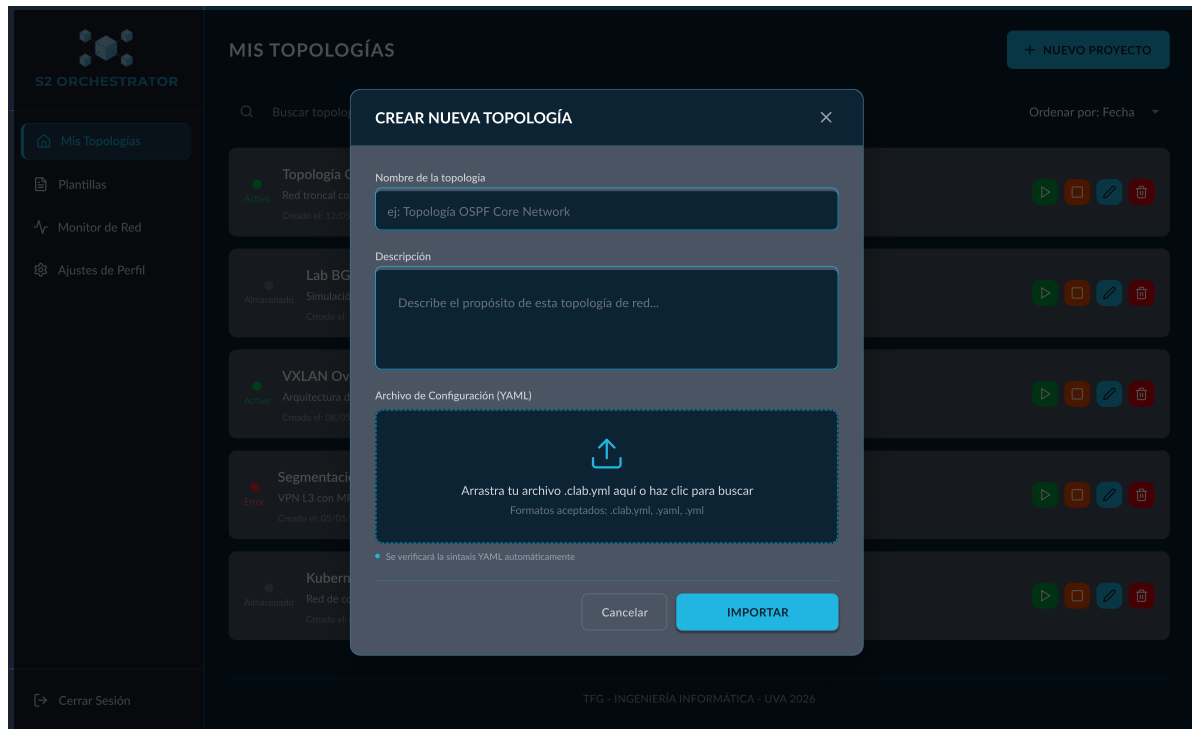


Figura 5.9: Vista de creación de una topología

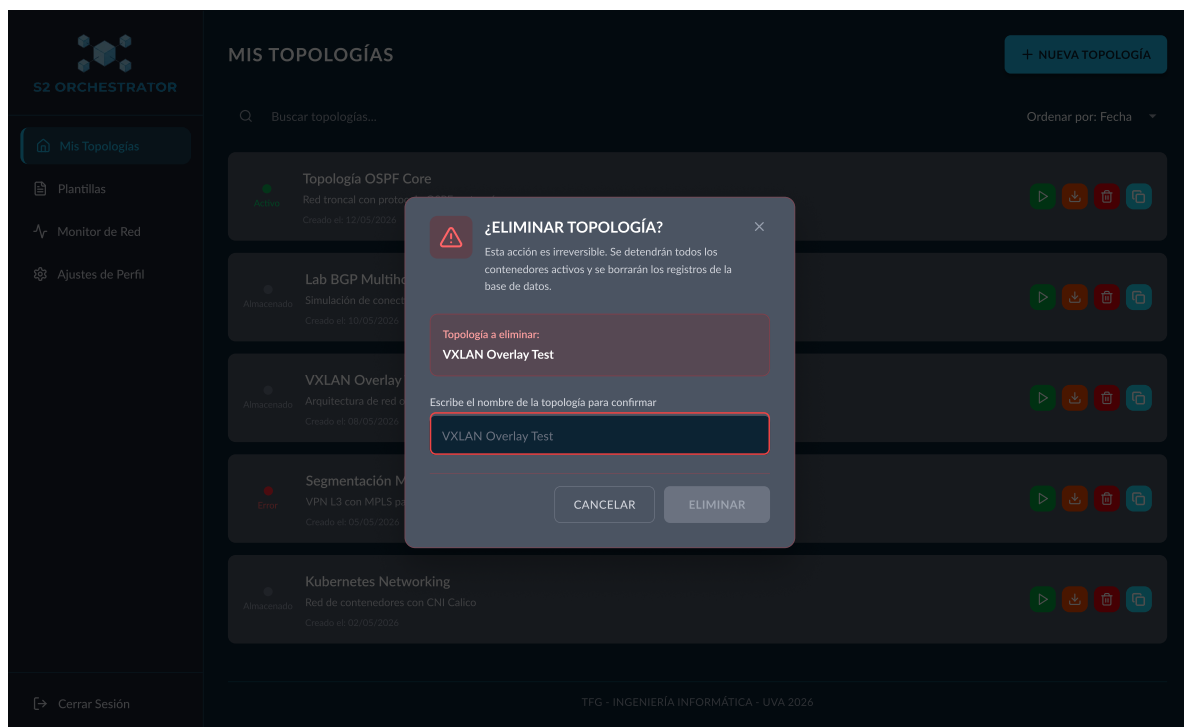


Figura 5.10: Vista de confirmación de eliminación de una topología

Bloque de monitorización operativa e interacción con nodos

Este último bloque abarca las herramientas en tiempo real diseñadas para el control de la infraestructura desplegada mediante Containerlab:

- **Monitorización interactiva:** Las Figuras 5.11 y 5.12 muestra el panel de control global. Ofrecen métricas de salud (nodos activos, enlaces y latencia media) y un flujo dinámico de eventos que puede alternarse con la terminal interactiva de la topología.
- **Métricas del nodo:** Para un análisis detallado, la Figura 5.13 expone el panel lateral con las métricas de consumo en tiempo real de *CPU* y *RAM* de un contenedor específico, permitiendo aplicar configuraciones lógicas e *IP* en caliente.
- **Consola dedicada:** Finalmente, la Figura 5.14 detalla el visor de *logs* avanzado y la terminal de línea de comandos con exclusividad al nodo seleccionado para labores de administración profundas.

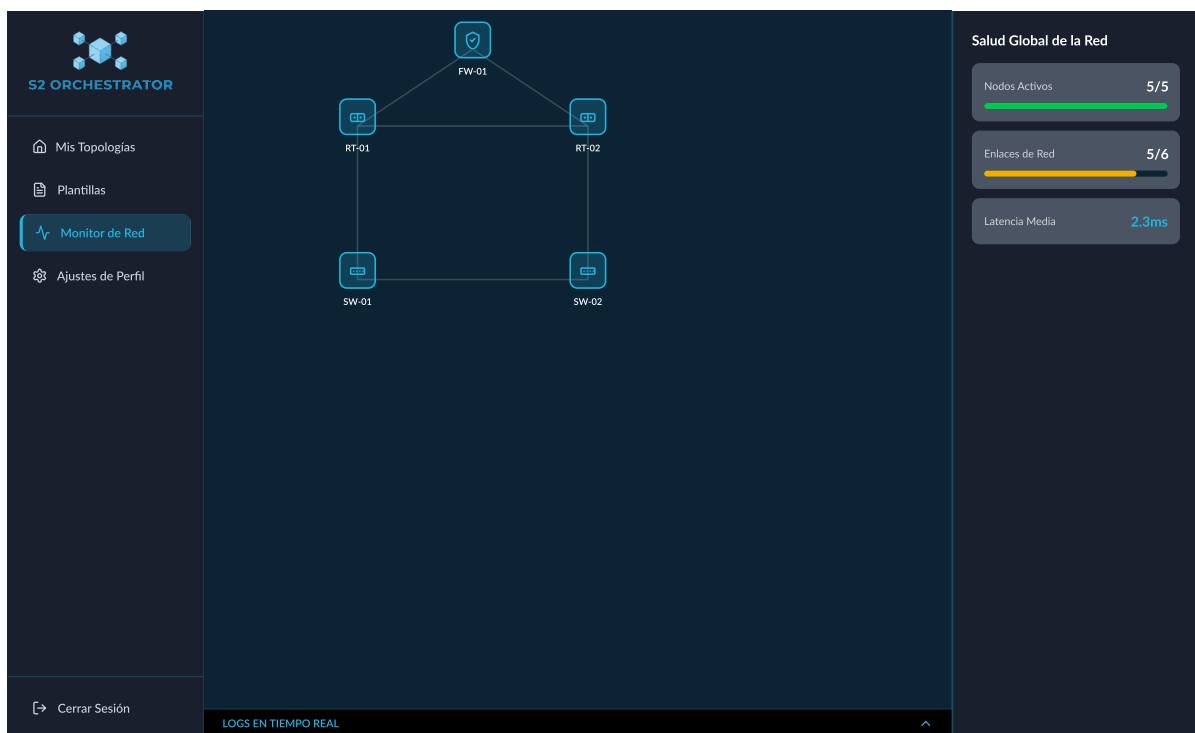


Figura 5.11: Vista de la monitorización de la red sin terminal

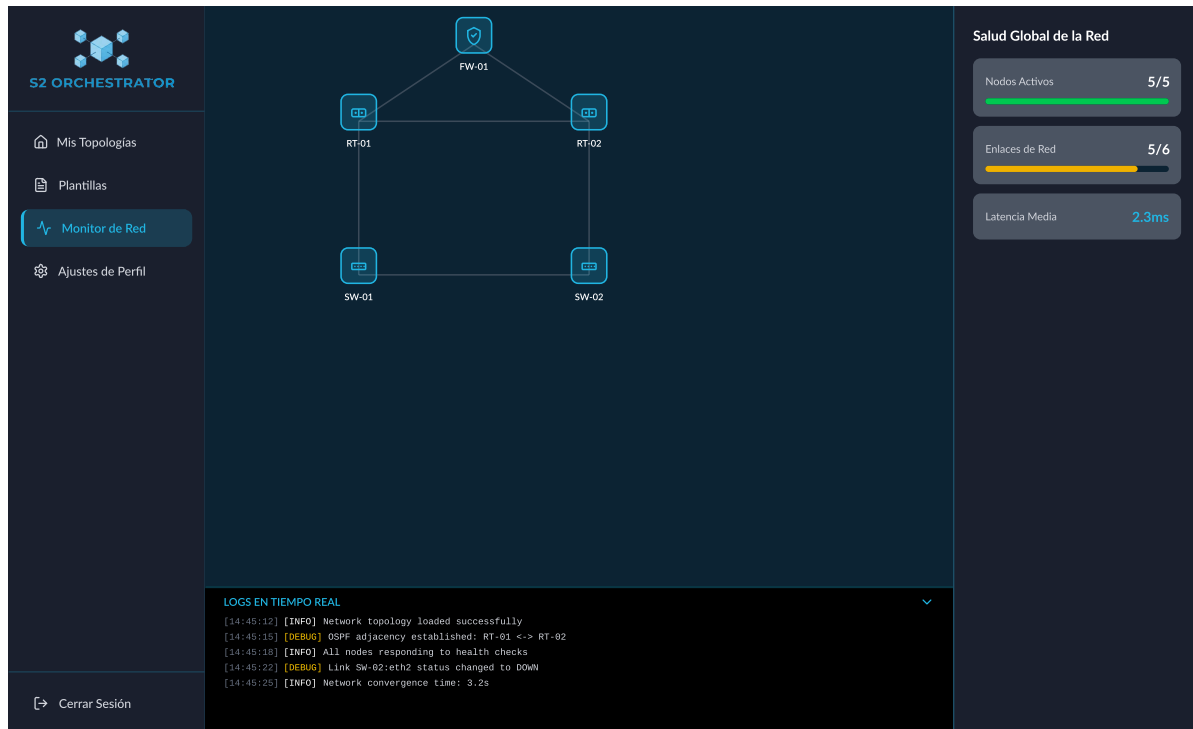


Figura 5.12: Vista de la monitorización de la red con terminal

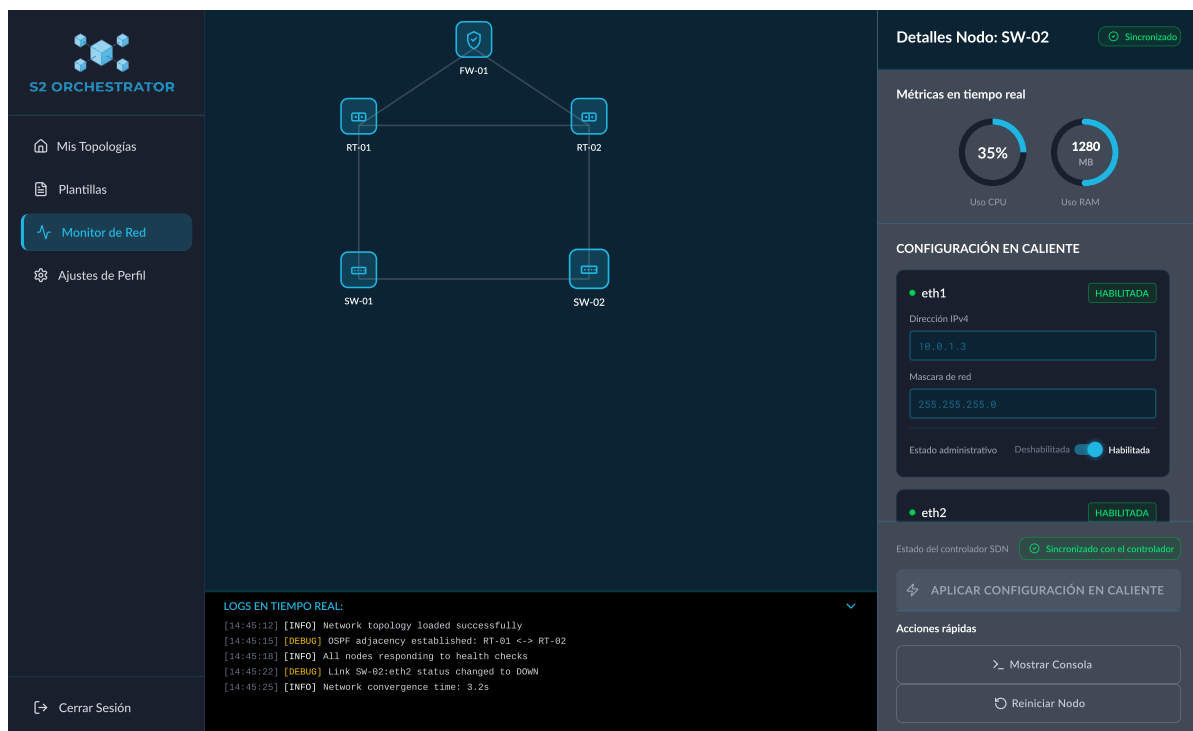


Figura 5.13: Vista de la monitorización de un nodo concreto

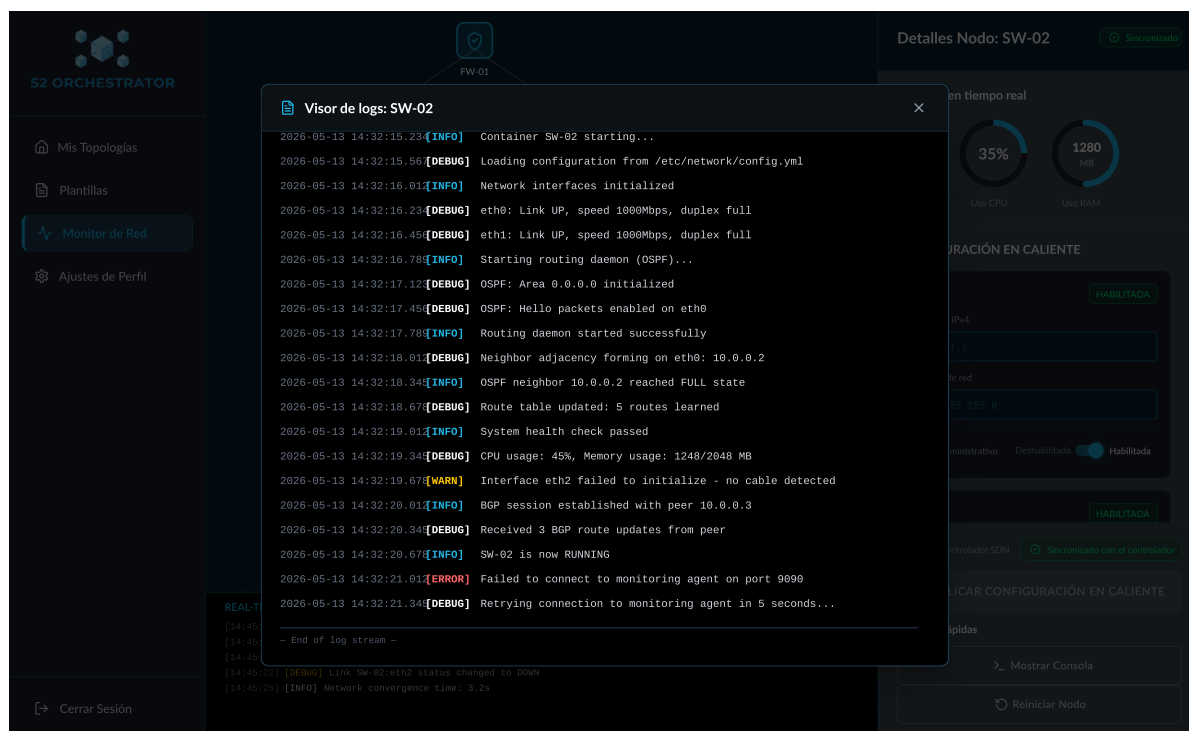


Figura 5.14: Vista de la consola de un nodo en concreto

5.7. Diagrama de despliegue

En la Figura 5.15 se ilustra el diagrama de despliegue del proyecto, el cual expone la distribución física y lógica de los componentes del sistema. Siguiendo de forma estricta los principios de desacoplamiento de las Redes Definidas por Software (SDN) detallados rigurosamente en la Sección 2.1, la arquitectura se desglosa en tres entornos de ejecución claramente diferenciados:

- **Entorno del Cliente (PC Usuario):** Representa el punto de acceso a la interfaz web por parte del usuario. Consiste en una máquina cliente que ejecuta un navegador web moderno, sobre el cual se despliega de forma aislada el Frontend de la aplicación, desarrollado en React y empaquetado mediante Vite. Este componente gestiona exclusivamente la lógica de presentación y la experiencia del usuario. Su comunicación con el servidor es completamente asíncrona, apoyándose en el envío de objetos estructurados (DTOs) mediante peticiones a la API HTTP bajo el protocolo HTTP, complementado con un canal de *WebSockets* dedicado a la recepción y visualización de logs de despliegue en tiempo real.
- **Plano de Control (Servidor Linux):** Constituye el núcleo inteligente de la aplicación y se ejecuta sobre un servidor basado en el sistema operativo Linux. En este entorno reside el Backend programado en Python, cuyo punto de entrada es un servicio API expuesto mediante el framework *FastAPI*. Este plano se encarga de procesar las solicitudes del cliente, verificar las políticas de seguridad y gestionar el ciclo de vida de los laboratorios. Cuenta con una base de datos relacional MySQL dedicada a consolidar de forma persistente los registros de los usuarios y los estados

de las topologías, así como un almacenamiento local que actúa como repositorio de plantillas y configuraciones en formato YAML mediante llamadas nativas de apertura y lectura de ficheros a nivel de sistema de archivos.

- **Plano de Datos e Infraestructura de Red:** Es el nivel inferior donde se materializan y ejecutan las topologías solicitadas. El plano de control interactúa directamente con Containerlab a través de llamadas al sistema por línea de comandos. Containerlab lee el archivo de configuración, computa la matriz de adyacencia y realiza las solicitudes pertinentes a la API interna del motor de Docker. Finalmente, el motor de Docker instancia los contenedores independientes (nodos de red) aislados en el espacio del usuario, mientras que el sistema operativo se encarga de crear los enlaces virtuales bidireccionales para interconectar los nodos lógicos, emulando con total precisión el comportamiento de una infraestructura de red física real.

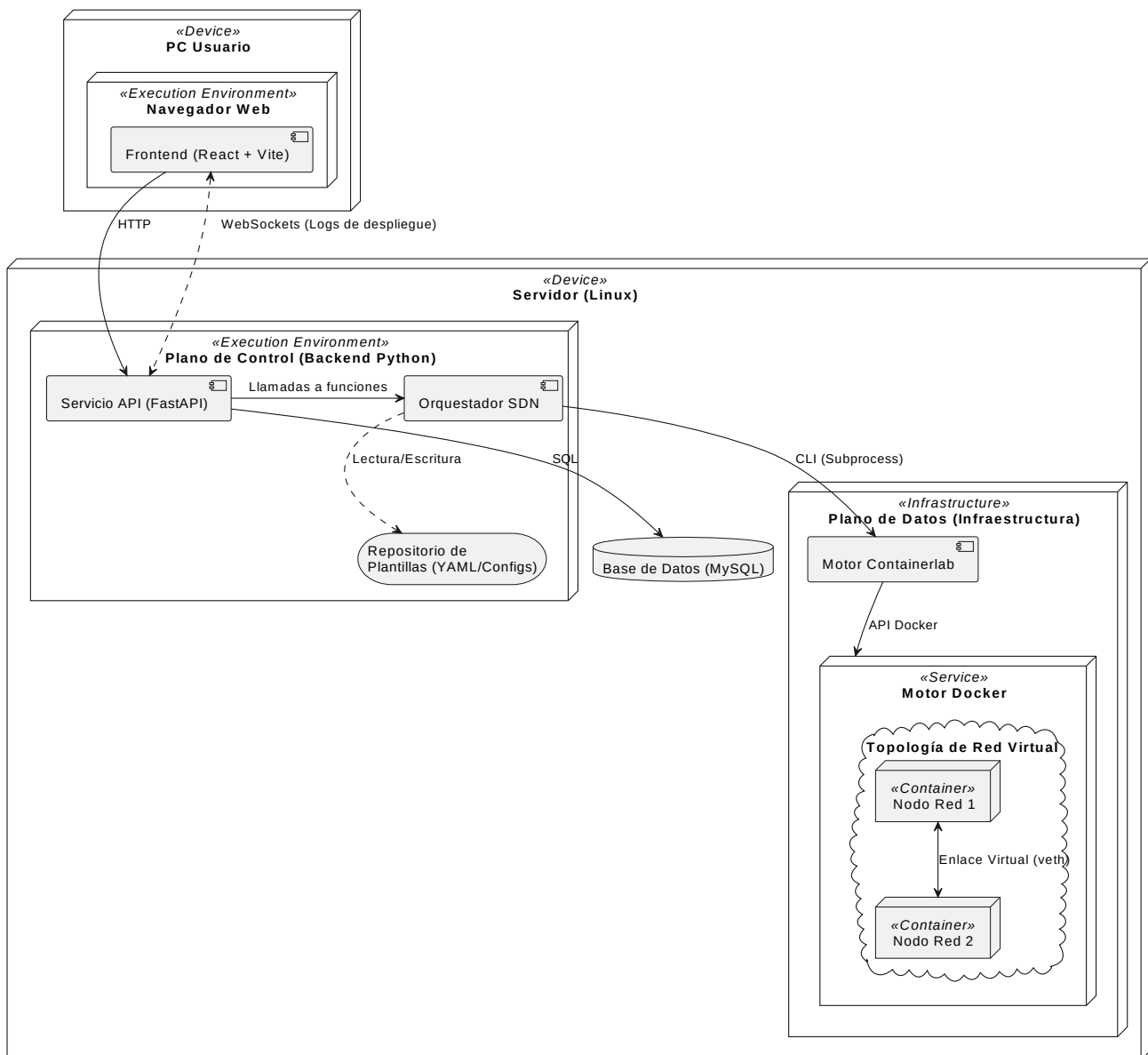


Figura 5.15: Diagrama de despliegue del sistema

Capítulo 6

Descripción de las iteraciones del sistema

En este capítulo se explica cómo se han implementado las funcionalidades principales del proyecto a lo largo del desarrollo. El contenido se estructura en torno a las sucesivas iteraciones del desarrollo, especificando para cada una de ellas los objetivos planteados, las tareas de programación realizadas en los scripts de Python y la interfaz de React, así como los imprevistos encontrados y las soluciones adoptadas.

6.1. Iteración 1: Núcleo del orquestador en CLI y Automatización de red

6.1.1. Objetivo

Desarrollar el motor básico en Python para interactuar con Containerlab mediante la línea de comandos, permitiendo validar, desplegar, destruir y operar topologías emuladas desde una interfaz de terminal centralizada.

6.1.2. Desarrollo y especificaciones de la implementación

El desarrollo de esta fase se concentró en la creación del programa principal *main.py*. Para evitar un diseño monolítico, el plano de control se dividió en módulos funcionales bien definidos.

- **Módulo de validación de entrada:** Antes de delegar el control a la infraestructura de contenedores, el programa lleva a cabo un proceso de validación de la ruta suministrada por el usuario a través de una lógica de control mediante la librería *pathlib* y el método *yaml.safe_load*. El sistema comprueba que el archivo existe, que no es un directorio y que el archivo tiene una extensión válida (.yaml, .yml, .clab.yaml o .clab.yml) impidiendo la ejecución ante nombres de archivos inválidos o vacíos que puedan ocasionar excepciones en tiempo de ejecución.
- **Módulo de orquestación del ciclo de vida (desplegar y destruir una topología):** Para la ma-

nipulación de los contenedores Docker a través de Containerlab, se descartaron funciones del sistema inseguras como *os.system* y se optó por encapsular los comandos mediante procesos hijo con la función *subprocess.run*.

- **Módulo de operación y diagnóstico de la red:** Una vez que la topología se ha desplegado, el sistema ofrece tres funcionalidades críticas para la administración del entorno sin necesidad de que el usuario abandone la aplicación:
 - **Inspección de estado:** Extrae dinámicamente el nombre de la red del archivo YAML y ejecuta un subprocesso con el comando *containerlab inspect*. El flujo de salida de Containerlab es volcado directamente en la terminal para facilitar la auditoría de las direcciones IP de las interfaces y el estado de salud de los nodos.
 - **Visualización del grafo:** Automatiza el arranque del servidor gráfico de Containerlab y, mediante la librería *webbrowser*, redirige de forma automática el navegador a la URL local *http://localhost:50080*, además, el código intercepta la excepción *KeyboardInterrupt* para asegurar un cierre controlado del servidor web al regresar al menú.
 - **Ejecución remota de comandos en los nodos:** Abstrae la potencia de *containerlab exec* ofreciendo un submenú. El usuario puede ejecutar comandos en toda la red de forma simultánea o aislar la ejecución de ese comando a un dispositivo en concreto aprovechando el sistema de etiquetas de Docker *-label clab-node-name=<nodo>*.

6.1.3. Contratiempos encontrados y soluciones adoptadas

Durante las pruebas de despliegue iniciales, se identificaron y resolvieron dos problemas relacionados con la robustez del sistema y el control de excepciones:

- **Pérdida de trazas de error en fallos de Containerlab:** Se detectó que si Containerlab fallaba debido a un error interno en la sintaxis del archivo YAML de la topología, las trazas de error se perdían en el flujo del sistema y la terminal se bloqueaba sin ofrecer ningún tipo de retroalimentación o *feedback* al usuario.

Como solución de ingeniería, se modificó la función de despliegue para capturar la excepción *subprocess.CalledProcessError*. A partir de esta captura, se diseñó un sistema de persistencia de los logs que escribe de forma automática tanto la salida estándar (*stdout*) como la salida de errores (*stderr*) en un directorio local bajo la nomenclatura *logs/despliegue[nombre].log* garantizando que el administrador pueda diagnosticar cualquier error en la infraestructura de red.

Listing 6.1: Captura de excepciones y persistencia de logs de despliegue

```
def desplegar_topologia(archivo_topologia):
    pathlib.Path("./logs").mkdir(exist_ok=True)
    nombre_log = f"logs/despliegue{pathlib.Path(archivo_topologia).stem}.log"
    comando = ["sudo", "containerlab", "deploy", "-t", str(archivo_topologia)]

    try:
        resultado = subprocess.run(comando, check=True, text=True, capture_output=True)
```

```
with open(nombre_log, "w") as archivo_log:
    archivo_log.write("--- SALIDA ESTANDAR ---\n")
    archivo_log.write(resultado.stdout)
guardar_topologia_activa(archivo_topologia)
return True
except subprocess.CalledProcessError as e:
    # Persistencia de las trazas de error ante un fallo de Containerlab
    with open(nombre_log, "w") as archivo_log:
        archivo_log.write("--- SALIDA ESTANDAR ---\n")
        archivo_log.write(e.stdout if e.stdout else "")
        archivo_log.write("--- SALIDA DE ERRORES ---\n")
        archivo_log.write(e.stderr if e.stderr else "")
    print(f"\n[ERROR] Fallo en el despliegue. Log guardado en {nombre_log}")
    return False
```

- **Inconsistencia de la topología por cierres imprevistos (Ctrl+C):** Se detectó un problema de consistencia si el usuario forzaba el cierre del sistema mediante una señal de teclado (*Ctrl+C*) o la terminal sufría una caída inesperada con la topología ya desplegada. Al reiniciar el programa *main.py*, se volvía a solicitar un archivo para desplegar la topología correspondiente; si el usuario introducía el mismo archivo, el sistema intentaba lanzar un nuevo despliegue, provocando un conflicto de colisión de nombres y puertos en la API Docker que bloqueaba la ejecución.

Para solventar este problema, se modificó la función de despliegue implementando un mecanismo de persistencia basado en las funciones *guardar_topologia_activa* y *hay_topologia_activa*. Al completarse el despliegue de una topología, el orquestador empleaba la función *tempfile.NamedTemporaryFile* para volcar una copia del archivo en el directorio *./tmp*. Al arrancar, el sistema verifica esta ruta, si detecta la existencia de un archivo, bloquea la solicitud de un nuevo archivo y redirige el flujo al menú postdespliegue. El archivo temporal solo se elimina cuando el usuario destruye la topología de forma controlada desde el menú.

Listing 6.2: Mecanismo de persistencia de la topología activa mediante ficheros temporales

```
def guardar_topologia_activa(archivo_topologia):
    with tempfile.NamedTemporaryFile(
        mode="w+t", delete=False, prefix="", suffix=".yaml", dir="./tmp"
    ) as tmp:
        with open(archivo_topologia, "r") as f:
            tmp.write(f.read())
        print(f"Backup temporal creado en: {tmp.name}")
    return tmp.name

def hay_topologia_activa():
    ruta_temporal = pathlib.Path("./tmp")
    ruta_temporal.mkdir(exist_ok=True)
    archivos_yaml = list(ruta_temporal.glob("*.yaml"))
    return archivos_yaml
```

6.2. Iteración 2: Diseño e integración de la persistencia en base de datos

6.2.1. Objetivo

El objetivo principal de esta etapa fue dotar al sistema de un mecanismo de persistencia mediante el diseño e implementación de una base de datos relacional, migrando el control de las topologías desde el modelo inicial de archivos temporales hacia un almacenamiento estructurado y permanente.

6.2.2. Desarrollo y especificaciones de la implementación

El trabajo técnico de esta iteración se centró en estructurar la capa de datos del sistema para permitir el registro, identificación y guardado de las redes emuladas:

- **Diseño del esquema de datos:** Se redactó el script SQL inicial para la creación de las tablas especificando las claves primarias y las restricciones necesarias para asociar cada despliegue con sus atributos en el sistema.

6.2.3. Contratiempos encontrados y soluciones adoptadas

Durante la implementación de la persistencia, se identificó un planteamiento técnico que obligó a replantear la estructura del almacenamiento:

- **Definición ineficiente del almacenamiento de los archivos de definición de las topologías:** En el planteamiento inicial del sistema, se consideró almacenar la configuración completa de las redes emuladas utilizando campos de tipo *TEXT* para volcar directamente todo el contenido de los archivos YAML dentro de la base de datos. Sin embargo, se detectó que este enfoque penalizaba considerablemente el rendimiento de las consultas y complicaba la edición externa de las topologías.

Como solución adoptada, se modificó la arquitectura del almacenamiento hacia un modelo de indexación por rutas. Se refactorizó el script de creación de la base de datos reemplazando los campos de tipo *TEXT* por columnas de tipo *VARCHAR* de longitud adecuada. De este modo, la base de datos solo almacena la ruta absoluta de la ubicación del fichero en el sistema, permitiendo que el orquestador interactúe con el sistema de archivos de forma ágil, directa y escalable.

A continuación, se presenta el fragmento de script SQL definitivo que refleja esta optimización de la persistencia relacional.

Listing 6.3: Script de creación del esquema relacional y optimización del almacenamiento

```
/* PROYECTO TFG: Orquestador de Red Corporativa SDN
Script de creacion de la Base de Datos persistente
```

```
*/

-- DEFINICION DEL ESQUEMA DE LA BASE DE DATOS
create database if not exists TFG;
use TFG;

-- TABLA DE USUARIOS.
create table if not exists Usuario (
    idUsuario int auto_increment primary key,
    idExterno varchar(255) unique,
    nombre varchar(255),
    correo varchar(255) unique,
    avatarURL varchar(512),
    fechaRegistro timestamp default current_timestamp,
    fechaUltimoAcceso timestamp null,
    rol enum('Admin','Usuario') default 'Usuario'
);

-- TABLA DE TOPOLOGIAS DEL USUARIO
create table if not exists Topologia(
    idTopologia int auto_increment primary key,
    idUsuario int,
    nombre varchar(100) not null,
    descripcion varchar(1024),
    esPlantilla boolean default false,
    rutaArchivoDefinicion varchar(512),
    estadoTopologia enum('Almacenada','Activa','En despliegue','En destruccion','Error') default
        'Almacenada',
    fechaCreacion timestamp default current_timestamp,
    mensajeError text null,
    foreign key (idUsuario) references Usuario(idUsuario) on delete set null
);
```

6.3. Iteración 3: Diseño de las topologías de red y creación de las plantillas corporativas

6.3.1. Objetivo

Diseñar, codificar y validar arquitecturas de red emuladas con el propósito de establecer un catálogo de plantillas base (archivos YAML) estandarizadas y listas para ser importadas y desplegadas por el orquestador.

6.3.2. Desarrollo y especificaciones de la implementación

Durante esta fase, el trabajo se alejó temporalmente de la programación React y Python para centrarse en la ingeniería de redes y la virtualización.

- **Diseño de escenarios SDN:** Se diseñaron múltiples topologías de prueba que incluyen conmutadores lógicos (Open vSwitch), enrutadores y terminales finales (contenedores ligeros Linux).

- **Codificación YAML:** Se tradujeron los esquemas lógicos a los archivos de definición soportados por Containerlab. En estos archivos se estandarizaron los parámetros de red, especificando los rangos de direccionamiento IP, los tipos de nodos (kind) y el mapeo de los enlaces lógicos (endpoints veth) entre los distintos equipos.
- **Validación de las plantillas:** Cada plantilla fue sometida a pruebas individuales para asegurar que los contenedores arrancaban correctamente, que los tiempos de despliegue fuesen óptimos y que existiera conectividad real entre los extremos de la topología.

6.3.3. Contratiempos encontrados y soluciones adoptadas

Durante las pruebas de las topologías desde la terminal de comandos, los principales problemas que se encontraron estaban relacionados con la configuración de la infraestructura virtual y el enrutamiento lógico de los equipos.

- **Creación previa de los puentes virtuales y conexión con el controlador SDN:** Se detectó que la red SDN no operaba correctamente si se delegaba todo el proceso de arranque a Containerlab. Se descubrió que era obligatorio crear los puentes virtuales en el sistema operativo antes de iniciar el despliegue de los contenedores. En esta fase de pruebas, se optó por realizar este paso de forma manual introduciendo los comandos en la terminal. Sin embargo, este hallazgo se tuvo en cuenta para la posterior fase de integración con la interfaz web, donde el backend fue programado para automatizar esta creación antes de cada despliegue.

Como muestra del trabajo de codificación realizado en esta fase, en el Código 6.4 se expone uno de los escenarios base implementados. Esta plantilla define una red SDN elemental compuesta por un controlador, un switch y tres nodos finales. En el archivo se aprecia la estandarización de las imágenes de contenedor empleadas, la inyección automática de la configuración de red en el arranque de cada nodo y la definición de la topología física mediante el emparejamiento de las interfaces virtuales.

Listing 6.4: Definición de la topología sdn-basico para Containerlab

```
name: sdn-basico
topology:
  nodes:
    controller:
      kind: linux
      image: osrg/ryu
      cmd: ryu-manager ryu.app.simple_switch_stp_13 ryu.app.ofctl_rest
      ports:
        - "6633:6633"
        - "8080:8080"
    s1:
      kind: ovs-bridge
    h1:
      kind: linux
      image: alpine:latest
      exec:
        - ip link set dev eth1 up
```

```
- ip addr add 192.168.1.10/24 dev eth1
- ip route add default via 192.168.1.1
- apk add curl

h2:
  kind: linux
  image: alpine:latest
  exec:
    - ip link set dev eth1 up
    - ip addr add 192.168.1.20/24 dev eth1
    - ip route add default via 192.168.1.1
    - apk add curl

s-web:
  kind: linux
  image: nginx:alpine
  exec:
    - ip link set dev eth1 up
    - ip addr add 192.168.1.30/24 dev eth1
    - ip route add default via 192.168.1.1
    - sh -c 'echo "<h1>Servidor Seguro y Redundante</h1>" > /usr/share/nginx/html/index.html'

links:
  - endpoints: ["controller:eth2","s1:s1-ctrl"]
  - endpoints: ["s1:s1-s-web","s-web:eth1"]
  - endpoints: ["s1:s1-h1","h1:eth1"]
  - endpoints: ["s1:s1-h2","h2:eth1"]
```

6.4. Iteración 4: Integración del sistema full-stack y validación del producto final

6.4.1. Objetivo

Integrar los componentes desarrollados (backend de FastAPI, plantillas de red y frontend en React) para ensamblar el Producto Mínimo Viable (MVP). El objetivo es lograr un ciclo de vida donde el usuario interactúe con una interfaz gráfica y el servidor gestione la infraestructura emulada de forma transparente.

6.4.2. Desarrollo y especificaciones de la implementación

Esta iteración final consolidó la arquitectura Full-Stack desacoplada, asegurando una comunicación fluida entre el plano de datos y el plano de control:

- **Diseño de la API asíncrona:** Se estructuró el backend en FastAPI utilizando programación asíncrona (*async/await*). Esto permite que el servidor gestione múltiples peticiones simultáneas sin bloquear el hilo principal, garantizando una alta disponibilidad del servicio.
- **Gestión de la consistencia entre capas (Integridad Sincronizada):** Se ha implementado un mecanismo de persistencia híbrida que garantiza la coherencia entre la gestión de los metadatos

del usuario y el control de acceso, alojado en la base de datos SQL, de la definición técnica de la red, que reside exclusivamente en el archivo YAML.

- **Arquitectura de componentes reactivos:** En el *frontend*, se adoptó un modelo de componentes funcionales con *hooks*, lo que permite que la interfaz reaccione en tiempo real a los cambios de estado (como el arranque de un contenedor o la caída de una interfaz).

6.4.3. Contratiempos encontrados y soluciones adoptadas

La fase de integración del sistema final planteó diversos desafíos técnicos derivados de la complejidad de la infraestructura.

- **Bloqueo de procesos por la gestión de los logs:** Durante la monitorización en tiempo real, se detectó que la ejecución de comandos de larga duración, como *docker logs -f* sobre los nodos, provocaba el bloqueo del hilo principal del backend al quedar a la espera indefinida de nueva salida (stdout).

La solución estuvo en la refactorización del motor de orquestación integrando la librería *subprocess* con una gestión de tiempos de espera (timeouts) y lectura no bloqueante. Se implementó una lógica de sondeo con límites de tiempo definidos, garantizando que el orquestador no quede suspendido ante procesos que no retornan inmediatamente, manteniendo así la disponibilidad de la API para otras peticiones concurrentes. En el Código 6.5 se muestra el núcleo del controlador asíncrono diseñado para esta tarea.

Listing 6.5: Lógica de lectura no bloqueante y tiempos de espera mediante asyncio

```
comando = ['docker', 'logs', '-f', '--tail', '0', nombre_contenedor]
proceso = await asyncio.create_subprocess_exec(comando, stdout=subprocess.PIPE, stderr=
    subprocess.STDOUT)

try:
    loop = asyncio.get_event_loop()
    while True:
        try:
            # Lectura asincrónica con tiempo límite (timeout) para evitar bloqueos del
            # hilo principal
            linea = await asyncio.wait_for(loop.run_in_executor(None, proceso.stdout.
                readline), timeout=1.0)
        except asyncio.TimeoutError:
            continue # Si se agota el tiempo, se continua el bucle sin bloquear el
                servidor

        if not linea:
            break
        await websocket.send_json(parsear_linea_logs(linea))
```

- **Limitaciones en la obtención de las métricas de red nativas:** Se identificó la ausencia de comandos estandarizados en la infraestructura virtual para consultar métricas complejas, como la latencia punto a punto o el estado detallado de las interfaces en tiempo real.

Como solución, se implementó una capa de diagnóstico activo. Ante la carencia de una API de métricas, el sistema realiza sondeos mediante ping automatizado, calculando los tiempos de respuesta (RTT) a partir del análisis de la salida de texto. Asimismo, para determinar el estado de las interfaces, se desarrolló un algoritmo de inspección que analiza la salida de comandos de nivel de sistema, extrayendo el estado operativo de forma programática. Este enfoque permite inferir la salud de la red mediante la medición directa de la conectividad, mitigando la falta de visibilidad nativa en los contenedores. En el código 6.6 se expone la función encargada de esta extracción de métricas.

Listing 6.6: Algoritmo de diagnóstico activo y extracción de latencia

```
def medir_ping(origen, destino, nombre_topologia):
    contenedor_origen = f"clab-{nombre_topologia}-{origen}"
    try:
        # Ejecucion del diagnostico limitando el conteo y el tiempo de respuesta
        cmd = ["docker", "exec", contenedor_origen, "ping", "-c", "4", "-W", "1",
              destino]
        resultado = subprocess.run(cmd, capture_output=True, text=True, timeout=6)

        if resultado.returncode != 0:
            return "N/A"

        # Algoritmo de inspeccion: extraccion del RTT medio mediante expresiones
        # regulares
        match = re.search(r'round-trip min/avg/max = [\d\.]+/([\d\.]+)/', resultado.
                          stdout)
        if match:
            return f"{match.group(1)}ms"

    except subprocess.TimeoutExpired:
        return "Timeout"
    except Exception:
        return "Error"
```

La comunicación entre el cliente y el servidor se centraliza en una API detallada en la Tabla 6.1. En este listado se definen los 20 *endpoints* desarrollados, especificando su propósito, la ruta de acceso y los parámetros de entrada. Esta estructura garantiza que todas las funcionalidades descritas en los requisitos funcionales sean accesibles de forma estandarizada a través del plano de control.

Tabla 6.1: Definición de los endpoints de la API y WebSockets

Método	Ruta HTTP	Descripción	Parámetros
POST	/auth/google	Valida el token OAuth2 de Google, registra al usuario y devuelve el JWT interno.	Body: token
GET	/api/usuario/perfil	Recupera los datos básicos del usuario (nombre, correo, rol y avatar).	Header: JWT

6.4. Iteración 4: Integración del sistema full-stack y validación del producto final

Método	Ruta HTTP	Descripción	Parámetros
POST	/api/usuario/perfil	Actualiza el nombre del usuario en el sistema.	Body: nombre
POST	/api/usuario/avatar	Sustituye la imagen de perfil actualizando el almacenamiento físico y la BD.	Form: avatarURL
POST	/api/usuario/eliminar	Elimina por completo la cuenta del usuario de la base de datos.	Header: JWT
GET	/api/topologias	Lista todas las topologías creadas y almacenadas por el usuario autenticado.	Header: JWT
GET	/api/plantillas	Devuelve el catálogo de escenarios estandarizados y prediseñados.	Header: JWT
GET	/api/topologias/activa	Comprueba si el usuario tiene actualmente alguna infraestructura virtual en ejecución.	Header: JWT
POST	/api/topologias/importar	Extrae y almacena una nueva topología desde un archivo YAML, ZIP o TAR.	Form: nombre, desc, archivo
POST	/api/topologias/{id}/toggle	Despliega una red almacenada o destruye una red activa mediante subprocesos.	Path: id_topologia
POST	/api/topologias/{id}/detener	Ejecuta una parada de emergencia sobre los contenedores de una topología.	Path: idTopologia
POST	/api/topologias/{id}/clonar	Duplica la definición física y lógica de una red existente o plantilla.	Path: id_topologia
POST	/api/topologias/{id}/borrar	Elimina los registros de MySQL y los ficheros YAML del sistema operativo.	Path: id_topologia
GET	/api/topologias/{id}/descargar	Permite la exportación del archivo <code>.clab.yml</code> original hacia el cliente.	Path: id_topologia

Descripción de las iteraciones del sistema

Método	Ruta HTTP	Descripción	Parámetros
GET	/api/topologias/{id}/estado	Inspecciona la red y devuelve sus métricas, nodos, enlaces y coordenadas.	Path: idTopologia
WS	/ws/logs/{id}/{nodo}	Establece un canal WebSocket bidireccional para el volcado de logs en tiempo real.	Path: id, nodo Query: token
POST	/api/{id}/{nodo}/comando	Inyecta y ejecuta instrucciones shell de forma arbitraria dentro de un contenedor.	Body: comando
POST	/api/{id}/{nodo}/reiniciar	Fuerza el reinicio de un nodo aislado empleando <i>containerlab restart</i> .	Path: id, nodo
POST	/api/{id}/{nodo}/configurar	Reescribe el YAML inyectando nuevas IPs/Máscaras y redepliega la infraestructura.	Body: ConfigurarNodo
GET	/api/salud	Monitoriza la disponibilidad de la API y del servicio de Docker/Containerlab subyacente.	Ninguno

6.5. Estructura del proyecto

El código fuente del sistema se ha organizado siguiendo una arquitectura modular para separar claramente las responsabilidades del cliente, el servidor y la infraestructura. A continuación se detalla la estructura principal de directorios:

Listing 6.7: Estructura de directorios del proyecto

```
containerlab-orchestrator/  
|-- backend/
```

```

|   |-- app/
|   |   |-- main.py
|   |   |-- services/
|   |       |-- orquestador.py
|   |-- archivos/
|   |-- avatars/
|   |-- logs/
|   |-- topologias/
|   |-- .env
|-- frontend/
|   |-- src/
|   |   |-- assets/
|   |   |-- components/
|   |   |-- App.jsx
|   |   |-- index.css
|   |   |-- main.jsx
|   |-- index.html
|-- scripts/

```

A continuación del Listado 6.7, se detalla la función de los directorios y archivos claves del proyecto:

backend/: Directorio donde reside el código del servidor y la lógica principal de la aplicación.

- **app/main.py:** Es el archivo central de la API. Aquí se definen todos los *endpoints* que utilizan el *frontend* para enviar o recibir datos. Además, gestiona el inicio de sesión seguro de los usuarios mediante tokens y establece las conexiones en tiempo real para transmitir los logs a la web.
- **app/services/orquestador.py:** Actúa como el “motor” del sistema. Contiene las funciones que se comunican directamente con el sistema operativo para ejecutar los comandos de Docker y Containerlab, se encarga del trabajo pesado de desplegar y destruir las redes virtuales, leer el consumo de CPU y RAM de los nodos, calcular latencias mediante ping y generar las coordenadas espaciales para dibujar el mapa de la topología.
- **archivos/ y avatars/:** Carpetas utilizadas por el servidor para guardar de forma local los archivos YAML de las topologías importadas por los usuarios y sus fotos de perfil.
- **logs/:** Almacena los registros de ejecución de los despliegues, fundamentales para la depuración y monitorización.
- **topologias/:** Contiene las topologías predefinidas y plantillas base listas para ser desplegadas.
- **.env:** Archivo de configuración que almacena de forma segura las variables secretas (credenciales de la base de datos y claves de Google).

frontend/: Directorio que contiene todo el código visual e interactivo de la página web, desarrollado con la librería React.

- **assets/:** Almacena recursos estáticos como imágenes y logotipos.
- **src/components/:** Carpeta donde se almacenan los componentes visuales tales como vistas de gestión (Login.jsx o MisTopologias.jsx), elementos de control de redes (como

Descripción de las iteraciones del sistema

`ConsolaNodo.jsx` o `Monitor.jsx`) y componentes reutilizables como tarjetas y barras laterales.

- `App.jsx`: Componentes raíz que define la estructura global y las rutas de la aplicación.
- `index.css` y `main.jsx`: Ficheros de estilos globales y punto de entrada que inicializa la aplicación.
- `index.html`: Fichero base que carga la aplicación web en el navegador.

`scripts/`: Carpeta que contiene algunos scripts del sistema. Destaca el archivo `creacionBaseDatos.sql` que contiene las sentencias exactas para generar la estructura de tablas de la base de datos desde cero y el archivo `poblar_plantillas.sql` que contiene las sentencias necesarias para insertar las topologías predefinidas (plantillas) en la base de datos solo si estas no existen previamente.

Capítulo 7

Pruebas

En este capítulo se describen las pruebas llevadas a cabo, las cuales constituyen la fase de validación del sistema. El objetivo principal de estas pruebas es comprobar que el funcionamiento del software cumple con los requisitos previamente analizados y, por consiguiente, que cumple con los estándares de calidad, rendimiento y funcionalidad necesarios para su puesta en marcha en un entorno operativo real.

Además de la validación puramente técnica y funcional, el éxito de una plataforma orientada al usuario depende en gran medida de su accesibilidad y experiencia de usuario. Por este motivo, en la sección final de este capítulo se documentan también las pruebas de usabilidad realizadas, cuyo propósito es evaluar de forma cualitativa y cuantitativa lo intuitiva que resulta la interfaz y el grado de satisfacción de los usuarios al interactuar con el sistema.

7.1. Pruebas de aceptación

Para llevar a cabo la validación de forma sistemática y rigurosa, se han diseñado un conjunto de casos de prueba (CP) que están directamente vinculados a los Criterios de Aceptación (CA) definidos en la Sección 4.2.

Las pruebas aquí recogidas se tratan de pruebas de Sistema y Aceptación (End-to-End), las cuales permiten evaluar la aplicación en su conjunto desde la perspectiva del usuario final, garantizando que todos los componentes de la arquitectura (interfaz web, base de datos y motor de orquestación) se integran e interactúan correctamente.

Dado que el proyecto se rige por una metodología iterativa e incremental, la verificación del software no se ha relegado exclusivamente al final del proyecto. Durante el desarrollo, al final de cada iteración, se han ido realizando pruebas parciales correspondientes a los módulos implementados en dicha iteración. Sin embargo, los casos de prueba documentados en este capítulo representan la validación final e integral del sistema, es decir, la comprobación definitiva de que todos los incrementos desarrollados funcionan en conjunto y cumplen con los requisitos planteados.

A continuación, se detallan mediante tablas los diferentes escenarios ejecutados, documentando la acción realizada, el comportamiento esperado del sistema y el resultado empírico obtenido.

7.1.1. Gestión de identidad

CP01	Registro
Descripción	El usuario se registra en el sistema mediante el proveedor de identidad externo.
Criterio de aceptación asociado	CA01
Acción	Desde la pantalla de inicio de sesión, el usuario pulsa Iniciar sesión con Google y se autentica mediante Google.
Resultado esperado	El usuario es autenticado correctamente y redirigido al panel principal.
Resultado obtenido	Correcto. Usuario registrado correctamente y datos almacenados en la BBDD.

Tabla 7.1: CP01. Registro

CP02	Inicio de sesión
Descripción	El usuario inicia sesión mediante el proveedor de identidad externo.
Criterio de aceptación asociado	CA01
Acción	Desde la pantalla de inicio de sesión, el usuario pulsa Iniciar sesión con Google y se autentica con Google.
Resultado esperado	El usuario es autenticado correctamente y redirigido al panel principal.
Resultado obtenido	Correcto. Sesión iniciada y perfil del usuario cargado correctamente.

Tabla 7.2: CP02. Inicio de sesión

Pruebas

CP02-1	Acceso no autorizado a rutas protegidas
Descripción	Un usuario intenta acceder a URLs del sistema sin haber iniciado sesión.
Criterio de aceptación asociado	CA01
Acción	Un usuario sin sesión iniciada intenta acceder directamente a la URL del dashboard (ej: <i>/dashboard</i>) o a un endpoint de la API.
Resultado esperado	El sistema deniega el acceso y redirige automáticamente a la página de <i>Inicio de sesión</i> .
Resultado obtenido	Correcto. El sistema detecta la ausencia de sesión y bloquea la navegación.

Tabla 7.3: CP02-1. Acceso no autorizado a rutas protegidas

CP03	Modificación del perfil
Descripción	El usuario modifica el nombre o el avatar de su perfil de usuario.
Criterio de aceptación asociado	CA01
Acción	Desde la pantalla del perfil, el usuario modifica su nombre o avatar y guarda los cambios.
Resultado esperado	El sistema actualiza la base de datos y refleja los cambios en la interfaz.
Resultado obtenido	Correcto. Datos actualizados correctamente.

Tabla 7.4: CP03. Modificación del perfil

CP03-1	Modificación del perfil (Formato del avatar no valido)
Descripción	El usuario modifica el avatar de su perfil de usuario introduciendo archivos no permitidos.
Criterio de aceptación asociado	CA01
Acción	Desde la pantalla del perfil, el usuario modifica su avatar introduciendo un archivo no permitido y guarda los cambios.
Resultado esperado	El sistema bloquea la carga del archivo y muestra un mensaje de error indicando que el formato del archivo no es válido.
Resultado obtenido	Correcto. El sistema deniega la subida del fichero detectando el formato incorrecto.

Tabla 7.5: CP03-1. Modificación del perfil (Formato del avatar no valido)

7.1.2. Orquestación y despliegue

CP04	Carga de topología YAML válida
Descripción	El usuario importa el archivo YAML o zip de una topología.
Criterio de aceptación asociado	CA02
Acción	Desde la pantalla de importar topología, el usuario rellena los datos de la topología y sube el fichero correspondiente.
Resultado esperado	El sistema valida la estructura del archivo, lo analiza correctamente y lo guarda en la base de datos, quedando disponible para su despliegue.
Resultado obtenido	Correcto. El sistema confirma la importación y la topología aparece en el listado de topologías.

Tabla 7.6: CP04. Carga de topología YAML válida

CP04-1	Carga de topología YAML inválida (Sintaxis incorrecta)
Descripción	El usuario importa el archivo YAML o zip de una topología con una sintaxis incorrecta.
Criterio de aceptación asociado	CA02
Acción	Desde la pantalla de importar topología, el usuario rellena los datos de la topología y sube el fichero correspondiente con una sintaxis incorrecta.
Resultado esperado	El sistema rechaza el archivo y muestra un mensaje de error detallando que la sintaxis del archivo no es correcta.
Resultado obtenido	Correcto. El sistema interpreta el error y muestra un mensaje.

Tabla 7.7: CP04-1. Carga de topología YAML inválida (Sintaxis incorrecta)

CP04-2	Carga de topología YAML inválida (Formato no válido)
Descripción	El usuario importa el archivo de una topología con un formato no válido.
Criterio de aceptación asociado	CA02
Acción	Desde la pantalla de importar topología, el usuario rellena los datos de la topología y sube el fichero de la topología en un formato no válido.
Resultado esperado	El sistema rechaza el archivo y muestra un mensaje de error detallando que el formato del archivo no es el correcto.
Resultado obtenido	Correcto. El sistema interpreta el error y muestra un mensaje.

Tabla 7.8: CP04-2. Carga de topología YAML inválida (Formato no válido)

CP05	Despliegue y validación de tiempos
Descripción	El usuario realiza el despliegue de una topología y se comprueba el tiempo que tarda en desplegarse.
Criterio de aceptación asociado	CA03
Acción	Desde la pantalla topologías, el usuario selecciona el botón para desplegar una topología con cinco nodos y esta debe desplegar en menos de 60 segundos.
Resultado esperado	El sistema despliega los cinco nodos en menos de 60 segundos.
Resultado obtenido	Correcto. Tras realizar 10 ejecuciones controladas , el tiempo medio de despliegue es de unos 10,5 segundos, cumpliendo holgadamente con el límite de 60 segundos establecido.

Tabla 7.9: CP05. Despliegue y validación de tiempos

CP05-1	Despliegue y validación de tiempos (Fallo en el despliegue)
Descripción	El usuario realiza el despliegue de una topología pero se produce un error durante el proceso y, por lo tanto, la topología no llega a desplegarse correctamente.
Criterio de aceptación asociado	CA03
Acción	Desde la pantalla topologías, el usuario selecciona el botón para desplegar una topología con cinco nodos y provoca intencionadamente un fallo para que la topología no llegue a desplegarse.
Resultado esperado	El sistema detecta el error, aborta el proceso y la topología queda en estado "ERROR".
Resultado obtenido	Correcto. El sistema aborta la operación, libera los recursos y deja la topología en estado "ERROR".

Tabla 7.10: CP05-1. Despliegue y validación de tiempos (Fallo en el despliegue)

Pruebas

CP06	Parada de topología y eliminación de recursos
Descripción	El usuario selecciona la opción de detener la topología desplegada.
Criterio de aceptación asociado	CA05
Acción	Desde la pantalla topologías, el usuario selecciona el botón para detener la topología que se encuentra en estado “Activo”.
Resultado esperado	El sistema detiene los nodos, elimina contenedores, redes y todos los recursos asociados y marca la topología como “Almacenada”.
Resultado obtenido	Correcto. Los recursos del host han sido liberados y la topología se encuentra en estado “Almacenada”.

Tabla 7.11: CP06. Parada de topología y eliminación de recursos

CP07	Reinicio de nodo
Descripción	El usuario selecciona el reinicio de uno de los nodos de la topología desplegada.
Criterio de aceptación asociado	CA05
Acción	Desde la pantalla de monitorización, el usuario selecciona un nodo y pulsa el botón de reiniciar.
Resultado esperado	El sistema detiene y vuelve a levantar el contenedor. El estado del nodo pasa temporalmente a “Inactivo” y vuelve a estar “Activo”.
Resultado obtenido	Correcto. El nodo se ha reiniciado con éxito y la conectividad del resto de la red permanece intacta.

Tabla 7.12: CP07. Reinicio de nodo

CP08	Modificación de los parámetros de un nodo
Descripción	El usuario modifica algunos de los parámetros del nodo.
Criterio de aceptación asociado	CA05
Acción	Desde la pantalla de monitorización, el usuario selecciona un nodo y mediante el formulario modifica algunos parámetros del mismo.
Resultado esperado	El sistema aplica la nueva configuración, notifica el éxito de la operación y refleja el cambio en la vista de monitorización.
Resultado obtenido	Correcto. Parámetros actualizados y validados por el orquestador.

Tabla 7.13: CP08. Modificación de los parámetros de un nodo

CP08-1	Modificación de los parámetros de un nodo (Formato no válido)
Descripción	El usuario modifica algunos de los parámetros del nodo introduciéndolos en un formato no válido.
Criterio de aceptación asociado	CA05
Acción	Desde la pantalla de monitorización, el usuario selecciona un nodo y mediante el formulario modifica los parámetros del mismo con un formato no aceptado.
Resultado esperado	El sistema bloquea la acción, muestra un mensaje de error al usuario y no realiza cambios en la configuración.
Resultado obtenido	Correcto. El sistema detecta la validación fallida y mantiene la configuración original.

Tabla 7.14: CP08-1. Modificación de los parámetros de un nodo (Formato no válido)

7.1.3. Monitorización y telemetría

CP09	Visualización de logs en tiempo real
Descripción	El usuario visualiza los logs de despliegue de la topología en la consola de monitorización.
Criterio de aceptación asociado	CA04
Acción	El usuario se dirige a la vista de monitorización una vez la topología se ha desplegado y visualiza en la consola los logs en tiempo real.
Resultado esperado	El sistema establece una conexión y muestra los logs según vayan apareciendo sin necesidad de recargar la página.
Resultado obtenido	Correcto. La conexión se ha establecido correctamente y los logs fluyen dinámicamente en la consola.

Tabla 7.15: CP09. Visualización de logs en tiempo real

CP10	Monitorización de métricas de nodos
Descripción	El usuario visualiza los métricas de un nodo concreto.
Criterio de aceptación asociado	CA04
Acción	El usuario desde de la vista de monitorización, seleccionado un nodo y visualiza las métricas correspondientes al nodo.
Resultado esperado	El sistema muestra el consumo de CPU y RAM con actualización periódica en tiempo real.
Resultado obtenido	Correcto. El sistema muestra correctamente las métricas de los nodos con actualizaciones periódicas.

Tabla 7.16: CP10. Monitorización de métricas de nodos

CP11	Monitorización de métricas de salud de la red
Descripción	El usuario visualiza las métricas de salud de la red.
Criterio de aceptación asociado	CA04
Acción	El usuario accede a la vista de monitorización y visualiza las métricas de salud correspondientes a la red.
Resultado esperado	El sistema muestra la latencia y unas gráficas del número de enlaces activos con respecto al número de enlaces totales y de nodos activos con respecto a los nodos totales.
Resultado obtenido	Correcto. El sistema muestra correctamente las gráficas y la latencia, ambas actualizándose de manera periódica.

Tabla 7.17: CP11. Monitorización de métricas de salud de la red

7.1.4. Gestión de escenarios

CP12	Importación de plantilla predefinida
Descripción	El usuario importa una plantilla desde la biblioteca de plantillas a su espacio de trabajo.
Criterio de aceptación asociado	CA06
Acción	El usuario accede al catálogo de plantillas, selecciona un modelo predefinido y pulsa el botón “Usar plantilla”.
Resultado esperado	La topología se instancia correctamente en el espacio del usuario, manteniendo la estructura de nodos y enlaces definidos en la plantilla.
Resultado obtenido	Correcto. La plantilla ha sido cargada y está disponible para su despliegue en el listado de las topologías

Tabla 7.18: CP12. Importación de plantilla predefinida

Pruebas

CP13

Clonación de escenarios

Descripción

El usuario realiza una copia exacta de un escenario propio para su edición paralela o pruebas.

Criterio de aceptación asociado CA06

Acción

El usuario selecciona una topología existente en su espacio y pulsa el botón de clonar.

Resultado esperado

El sistema genera un nuevo escenario con una configuración idéntica al original, asegurando la integridad de los datos.

Resultado obtenido

Correcto. Escenario duplicado correctamente, el original permanece inalterado.

Tabla 7.19: CP13. Clonación de escenarios

CP14

Consulta del historial de topologías

Descripción

El usuario visualiza la lista de los escenarios creados o importados previamente.

Criterio de aceptación asociado CA06

Acción

El usuario navega a la sección de historial de topologías en el panel principal.

Resultado esperado

Se visualiza un listado con todas las topologías mostrando información relevante como nombre, fecha de creación, estado.

Resultado obtenido

Correcto. El historial se carga correctamente y permite la navegación por todos los registros previos.

Tabla 7.20: CP14. Consulta del historial de topologías

CP15	Exportación completa de topología
Descripción	El usuario descarga la configuración completa de una topología en un archivo comprimido.
Criterio de aceptación asociado	CA07
Acción	Desde el panel de topologías, el usuario selecciona la opción de exportar o descargar la topología deseada.
Resultado esperado	El sistema genera y proporciona un archivo comprimido que contiene el archivo YAML y los ficheros de configuración asociados a los nodos del escenario.
Resultado obtenido	Correcto. El archivo se genera correctamente y la descarga se inicia sin errores, conteniendo todos los archivos de configuración requeridos.

Tabla 7.21: CP15. Exportación completa de topología

7.1.5. Validación de requisitos no funcionales

CP16	Tiempos de carga de la interfaz
Descripción	El sistema debe de garantizar una carga rápida de la interfaz web para asegurar la disponibilidad continua.
Criterio de aceptación asociado	CA08
Acción	Acceder a las distintas vistas del dashboard (Listado de topologías, catálogo de plantillas, monitorización) bajo una conexión de red estándar.
Resultado esperado	El tiempo de carga de cualquier vista o componente principal debe de ser inferior a tres segundos.
Resultado obtenido	Correcto. Las distintas vistas de la interfaz registraron tiempos de carga de entre 0,6 y 0,8 segundos , cumpliendo holgadamente con el tiempo máximo exigido de unos 3 segundos.

Tabla 7.22: CP16. Tiempos de carga de la interfaz

Pruebas

CP17	Eficiencia en el flujo de despliegue
Descripción	El usuario despliega la red mediante un flujo optimizado de máximo tres pasos.
Criterio de aceptación asociado	CA08
Acción	El usuario selecciona la topología, configura los parámetros básicos y ejecuta el botón de despliegue.
Resultado esperado	El despliegue se inicia satisfactoriamente habiendo completado el flujo en tres pasos o menos.
Resultado obtenido	Correcto. Flujo optimizado, el sistema permite el despliegue efectivo en menos de tres pasos.

Tabla 7.23: CP17. Eficiencia en el flujo de despliegue

CP18	Aislamiento de acceso entre usuarios
Descripción	El sistema garantiza que los recursos de un usuario son inaccesibles para otros usuarios mediante validación de sesión.
Criterio de aceptación asociado	CA08
Acción	El usuario intenta acceder a un endpoint de la API o a una vista de monitorización de una topología vinculada a un ID de un usuario distinto al de la sesión activa.
Resultado esperado	El sistema deniega el acceso con un error de autorización garantizando que el usuario solo puede interactuar con sus propios recursos.
Resultado obtenido	Correcto. El sistema valida la propiedad del recurso en cada petición, impidiendo el acceso cruzado.

Tabla 7.24: CP18. Aislamiento de acceso entre usuarios

7.2. Pruebas de usabilidad

Como se ha mencionado anteriormente, la validación del usuario no se limita exclusivamente al correcto funcionamiento técnico del código, sino que abarca también la experiencia de usuario (UX).

La usabilidad se define como la medida en la que un producto puede ser utilizado por usuarios específicos con efectividad, eficiencia y satisfacción [104]. Para medir estos tres pilares y garantizar que la interfaz sea intuitiva y que el flujo de trabajo resulte natural, se han llevado a cabo pruebas con

usuarios reales.

7.2.1. Perfil de los participantes

Dado que el sistema desarrollado es una herramienta especializada, se seleccionó un grupo formado por 5 participantes (U1 a U5) con formación avanzada en Ingeniería de Telecomunicaciones e Ingeniería Informática. Por lo tanto, todos los participantes cuentan con conocimientos previos en redes, lo que permite evaluar la herramienta desde la perspectiva de su público objetivo real: operadores de red e ingenieros de infraestructura.

A estos usuarios se les pidió que interactuaran con la herramienta sin asistencia previa ni manuales de usuario para evaluar la curva de aprendizaje mediante los siguientes escenarios prácticos simulando un entorno de trabajo real:

Escenario 1: Primer acceso y personalización. Se indicó al usuario: “Acabas de entrar a trabajar como administrador de red en la empresa. Inicia sesión en el sistema con tu cuenta de Google, entra en tu perfil, cambia tu nombre a ‘Admin Principal’ y ponte una foto de perfil (avatar) que tengas en tu escritorio”. Objetivo: Analizar la fluidez durante el primer contacto con la plataforma y personalizar los parámetros del perfil de usuario.

Escenario 2: Importación y despliegue. Se indicó al usuario: “El departamento de arquitectura te ha enviado el diseño de una nueva topología. Súbelo al sistema con el nombre de ‘Red Sucursal’ y una breve descripción. Una vez subido, ponlo en marcha y confirma cuándo esté 100 % activo y los contenedores estén corriendo”. Objetivo: Comprobar la facilidad para importar una nueva topología, iniciar su despliegue y verificar de forma visual que la red se ha iniciado correctamente.

Escenario 3: Monitorización y diagnóstico integrado. Se indicó al usuario: “La red ya está funcionando, pero el servidor web ha reportado una incidencia. Entra en el panel de monitorización y haz dos cosas: primero, abre los logs del nodo ‘s-web’ para ver qué está pasando; segundo, utiliza la interfaz para hacer un ping desde el nodo ‘h1’ hacia ‘s-web’ y comprobar la conectividad”. Objetivo: Evaluar la facilidad para localizar las herramientas de diagnóstico de la red y comprobar que la información de monitorización se muestra de manera clara.

Escenario 4: Gestión de escenarios y liberación de recursos. Se indicó al usuario: “Las pruebas han terminado. Detén completamente la topología en ejecución. Como nos ha servido de base, crea un clon o copia de esta topología para tenerla guardada, y luego elimina la topología original para liberar espacio”. Objetivo: Validar que el proceso para duplicar un escenario, detener su ejecución y borrar la topología original resulta intuitivo y seguro para el usuario.

7.2.2. Resultados cualitativos y observaciones

Dada la familiaridad técnica de los usuarios, no se experimentaron bloqueos críticos durante los escenarios. No obstante, las pruebas revelaron puntos de fricción menores muy valiosos para futuras

iteraciones:

- En el despliegue (Escenario 2), aunque el sistema permite modificar parámetros de los nodos una vez están en ejecución desde la vista de monitorización, dos usuarios sugirieron poder realizar estos ajustes antes de lanzar el despliegue, para agilizar así la configuración inicial de la red.
- En el diagnóstico (Escenario 3), los usuarios no tuvieron problemas en encontrar la opción de ejecutar comandos en el nodo para lanzar el ping. Por otro lado, respecto a la consola de logs, se sugirió incluir una barra de filtrado por palabras clave (ej.“ERROR”), ya que el flujo constante de logs de texto en topologías grandes dificulta la lectura.
- En la limpieza del entorno (Escenario 4), el mecanismo de seguridad requerida para eliminar la topología se reconoció como un acierto. Respecto al proceso de clonación, los participantes sugirieron habilitar la opción de editar el nombre de la nueva topología creada ya que el sistema actual asigna de forma automática un nombre basado en el nombre de la topología original seguido del número de la versión y resulta algo rígido para la organización personal del usuario.

Evaluación cuantitativa: System Usability Scale (SUS)

Para respaldar empíricamente las observaciones, los 5 participantes completaron el cuestionario estandarizado **System Usability Scale (SUS)** [105]. Este método consta de 10 afirmaciones que se valoran mediante una escala Likert del 1 (Totalmente en desacuerdo) al 5 (Totalmente de acuerdo).

En la Tabla 7.25, se recogen las respuestas en valores 1-5 aportadas por cada uno de los usuarios (U1-U5) para cada una de las 10 preguntas del cuestionario [106]:

Pregunta del cuestionario SUS	U1	U2	U3	U4	U5
1. Creo que me gustaría usar este sistema con frecuencia. (+)	5	4	4	5	5
2. Encontré el sistema innecesariamente complejo. (-)	1	2	1	2	2
3. Pensé que el sistema era fácil de usar. (+)	4	4	4	5	5
4. Creo que necesitaría el apoyo de un técnico para poder usar este sistema. (-)	1	2	2	1	1
5. Encontré que las diversas funciones del sistema estaban bien integradas. (+)	4	3	4	4	4
6. Pensé que había demasiada inconsistencia en este sistema. (-)	1	2	2	2	3
7. Imagino que la mayoría de las personas aprenderían a usar este sistema muy rápidamente. (+)	5	4	4	4	4
8. Encontré el sistema muy incómodo de usar. (-)	2	2	2	1	2
9. Me sentí muy seguro(a) usando el sistema. (+)	4	4	5	4	4
10. Necesité aprender muchas cosas antes de poder empezar a usar este sistema. (-)	2	2	2	2	3

Tabla 7.25: Respuestas en escala Likert (1-5) de los participantes al test SUS.

Análisis de resultados Aplicando la fórmula de normalización del SUS, donde a las preguntas de orientación positiva (impares) se les resta 1 a la respuesta del participante y a las preguntas de orientación negativa (pares) se resta la respuesta del participante a 5 y multiplicando la suma final por 2,5 [106], se obtuvieron las siguientes puntuaciones individuales sobre 100:

- **U1:** 87,5
- **U2:** 72,5
- **U3:** 80,0
- **U4:** 85,0
- **U5:** 77,5

El sistema obtiene una puntuación media de 80,5 sobre 100. En la escala de aceptabilidad del método SUS, un valor superior a 68 indica que el sistema es usable, y superar la barrera de los 80 puntos lo sitúa en el rango de excelente. Estos datos demuestran que, a pesar de que los usuarios más críticos demandan una mayor flexibilidad en la personalización de ciertos flujos, la interfaz logra abstraer eficazmente la complejidad de la orquestación subyacente.

Capítulo 8

Conclusiones

8.1. Conclusiones

Al evaluar el desarrollo final de la plataforma con respecto a los objetivos e hitos propuestos al inicio de este Trabajo Fin de Grado, se puede determinar que estos se han alcanzado con éxito. A continuación, se detallan los logros obtenidos para cada uno de los objetivos específicos estipulados:

8.1.1. Objetivos del proyecto

- **Desarrollar un flujo de automatización capaz de aprovisionar infraestructuras de red funcionales a partir de definiciones lógicas estructuradas:**
 - *Hito alcanzado:* Se diseñó e implementó un motor de ejecución en el backend que procesa archivos en formato YAML, un formato de definición de datos que permite describir la topología de forma legible y estructurada. El motor emplea FastAPI como orquestador central, un framework de alto rendimiento que actúa como el cerebro del sistema: valida la integridad de las configuraciones y gestiona los procesos de forma asíncrona, garantizando una alta disponibilidad del servicio incluso durante operaciones intensivas. Dicha capa interpreta las especificaciones lógicas y las traduce automáticamente en comandos nativos de Containerlab, encapsulando la complejidad operativa de la consola. FastAPI orquesta de forma segura la interacción con la CLI (la interfaz de línea de comando de Linux), transformando definiciones de usuarios en infraestructuras de red reales en cuestión de segundos, sin necesidad de intervención manual directa.
- **Habilitar un entorno de gestión que facilite el despliegue y la operación de la red, ofreciendo un asistente guiado que abstraiga la complejidad técnica para distintos perfiles de usuario:**
 - *Hito alcanzado:* Se construyó una interfaz web reactiva y moderna desarrollada con React ideada para generalizar el acceso a infraestructuras de red mediante la eliminación total de la

dependencia de la terminal de comandos. Esta interfaz actúa como una capa de abstracción visual que simplifica el ciclo de vida completo de un laboratorio:

- **Gestión del ciclo de vida:** El usuario dispone de un panel de control centralizado para el aprovisionamiento, ejecución, parada y eliminación de topologías centralizando acciones que, de forma manual, requerirían múltiples comandos del sistema operativo y una gestión compleja de las rutas de archivos.
- **Asistente de operación intuitivo:** Se han integrado mecanismos de clonación y descarga de topologías, permitiendo el versionado ágil de laboratorios y portabilidad del conocimiento.
- **Abstracción del error:** Al encapsular la ejecución técnica bajo una capa de control lógica, el sistema impide la introducción de errores sintácticos o inconsistencias. Esto permite que el usuario opere infraestructuras de red con garantías de estabilidad, delegando en el sistema la validación de la configuración antes de su despliegue.
- **Visibilidad operativa en tiempo real:** Más allá del despliegue, la interfaz ofrece un plano de control que monitoriza el estado de la salud de los nodos y el flujo de logs. Esto permite una supervisión activa del laboratorio sin necesidad de consultar múltiples consolas o archivos, ofreciendo una visión integral y centralizada del estado del sistema.

■ **Implementar un modelo de persistencia híbrido basado en la Fuente Única de Verdad (SSOT):**

- *Hito alcanzado:* Se ha diseñado un modelo de datos donde la responsabilidad de la verdad (SSOT) se encuentra dividida para garantizar la consistencia del sistema. Mientras que la base de datos relacional MySQL actúa como el repositorio central de metadatos administrativos (gestión de usuarios, trazabilidad de las topologías y estado operativo del ciclo de vida), el archivo YAML se ha consolidado como la Fuente Única de Verdad (SSOT) técnica. Esa decisión de diseño garantiza que la configuración de la red (nodos, enlaces y parámetros operativos) resida en un único fichero fuente, evitando los riesgos de inconsistencia y los fallos de sincronización que conlleva replicar estructuras complejas en tablas relacionales. De este modo, cualquier cambio en la infraestructura se refleja directamente en el archivo origen, asegurando que el motor de orquestación siempre trabaje con la definición más reciente y fiable del laboratorio.

■ **Mejorar la eficiencia en el aprovisionamiento de la infraestructura, reduciendo la intervención manual mediante la generación automática de configuraciones de red:**

- *Hito alcanzado:* Se ha integrado un catálogo de escenarios maestros (ej. SND Básico, SDN Alta Disponibilidad) que permite al usuario importar entornos de red listos para operar. Para cualquier usuario, esto funciona de forma similar a una plantilla de proyecto: en lugar de diseñar la topología y configurar la conectividad entre nodos desde cero, el usuario selecciona un escenario predefinido que ya contiene la lógica de red necesaria para demostrar ese concepto técnico.

■ **Implementar un sistema de monitorización y gestión operativa centralizada para la visualización y control de los nodos:**

- **Hito alcanzado:** Se ha dotado a la plataforma de un panel de control avanzado que permite una supervisión técnica integral de la infraestructura. Esta funcionalidad se articula en un pilar:
 - **Monitorización en tiempo real:** El sistema integra un código visual detallado que refleja la situación operativa de toda la infraestructura. Por una parte, permite identificar que nodos se encuentran activos y por otra, el sistema extiende esta supervisión a los enlaces, mostrando gráficamente qué interfaces de los nodos se encuentran operativas y cuáles están deshabilitadas o caídas. Esto permite al usuario detectar problemas de conectividad desde el panel, sin necesidad de realizar pruebas o comandos de verificación manuales en cada dispositivo.

8.1.2. Objetivos personales y de aprendizaje

En lo que respecta a la adquisición de nuevas competencias y a mi crecimiento personal, he consolidado los siguientes logros:

- **Desarrollo de lógica de negocio específica para el control de infraestructuras de red:** Se ha consolidado la capacidad de aplicar lógica de programación sobre el plano de control y datos de redes virtualizadas. Esto incluye el diseño de algoritmos para el procesamiento y normalización de archivos de topología, el mapeo dinámico de interfaces virtuales a recursos de almacenamiento en el sistema operativo, y la construcción de una API estructurada que traduce peticiones de usuario en comandos de red precisos, garantizando la consistencia del plano de datos en entornos virtualizados.
- **Experiencia en herramientas de orquestación:** Se reforzó el dominio en la gestión de escenarios de red virtualizados basados en contenedores aislados (Docker) mediante el despliegue automático con Containerlab en sistemas operativos Linux.
- **Dominio del paradigma NetDevOps:** Se ha integrado con éxito la metodología *NetDevOps*, que consiste en la aplicación de principios de ingeniería de software en la gestión de redes. En lugar de tratar a la red como un conjunto estático de dispositivos configurados manualmente, el proyecto adopta tres pilares clave:
 - **Infraestructura como Código (IaC):** La topología y los parámetros de red se definen mediante archivos YAML estructurados. Esto permite que el diseño de red sea versionable, auditable y fácilmente reproducible, tratando la configuración de red exactamente igual que el código fuente de una aplicación.
 - **Automatización del ciclo de vida:** Se ha reemplazado la configuración manual en consola por un motor de orquestación que automatiza el despliegue y la destrucción de la infraestructura. Esto asegura que cada entorno de red sea consistente y esté libre de errores de configuración humana, siguiendo el principio de despliegue continuo.
 - **Arquitectura desacoplada:** Al separar la capa de presentación (*frontend* en React) de la capa de control (*backend* en FastAPI), se ha logrado una arquitectura *Full-Stack* que permite

que la lógica de red evolucione independientemente de la interfaz de usuario, garantizando la escalabilidad y mantenimiento del sistema a largo plazo.

- **Planificación y gestión de proyectos:** Se afrontó con autonomía el ciclo de vida completo del desarrollo de software, desde la especificación técnica de requisitos y el diseño arquitectónico de la base de datos hasta la resolución de problemas de autenticación, persistencia de los datos en MySQL y control de asincronía en las peticiones API.

En resumen, este Trabajo Fin de Grado ha permitido consolidar una herramienta operativa y funcional que da respuesta a la necesidad actual de modernizar y agilizar el despliegue de infraestructuras de red. Al combinar la potencia de emulación ligera de Containerlab con un entorno web accesible y centralizado (desarrollado con FastAPI y React), la plataforma se presenta como una solución valiosa para hacer accesible el uso de tecnologías *NetDevOps*, facilitando el aprendizaje y la gestión de escenarios de red tanto en entornos académicos como profesionales. Esto se consigue mediante la abstracción de la complejidad técnica: el sistema sustituye la configuración manual y el uso de la terminal por una interfaz intuitiva que automatiza el ciclo de vida de los nodos, permitiendo que el usuario despliegue escenarios complejos con un solo clic. Esta automatización elimina la barrera de entrada técnica, en entornos académicos, facilita el aprendizaje al incluir un catálogo de plantillas predefinidas, lo que permite al estudiante centrarse en la comprensión de conceptos técnicos sin necesidad de profundizar en la creación de archivos de configuración de topologías o sintaxis complejas. Por otra parte, en el ámbito profesional, el sistema garantiza la consistencia y auditabilidad necesarias para una gestión eficiente de los recursos. La experiencia acumulada a lo largo del ciclo de desarrollo de este proyecto no solo demuestra el potencial de la automatización de las redes, sino que consolida un pilar fundamental para el inicio de mi trayectoria como ingeniero de software y redes.

8.2. Líneas de trabajo futuras

A pesar de haber cumplido satisfactoriamente los objetivos iniciales y disponer de una plataforma totalmente operativa para la gestión de laboratorios, el margen de evolución del sistema es bastante amplio. Debido a las restricciones temporales propias del desarrollo de un Trabajo Fin de Grado, se han dejado fuera del alcance ciertas características.

A continuación, se describen las propuestas de desarrollo futuro, clasificadas entre la incorporación de nuevas funcionalidades operativas y mejoras técnico-estructurales:

8.2.1. Ampliación de funcionalidades y mejoras en la experiencia del usuario

- **Evaluación analítica de usabilidad:** Partiendo de las evaluaciones preliminares, se plantea escalar las pruebas de usabilidad involucrando a una muestra de usuarios más extensa. El objetivo es complementar los datos actuales midiendo factores analíticos de interacción y aplicando modelos matemáticos de Interacción Persona-Ordenador (HCI). Mediante la aplicación de principios como la ley de Fitts, para optimizar la ubicación y el tamaño de los controles de la interfaz, o la ley

de Hick, para medir y minimizar la carga cognitiva en la toma de decisiones, se podrán identificar objetivamente los cuellos de botella en la navegación y refinar la interfaz con un rigor técnico y analítico mayor.

- **Editor visual interactivo de topologías:** Actualmente, la importación de escenarios se realiza mediante la carga y validación de archivos en formato YAML planos. Una evolución natural y de gran valor para la usabilidad sería implementar un lienzo interactivo en el frontend, lo que permitiría a los usuarios arrastrar, soltar y conectar nodos (routers, switches) de forma visual, generando el archivo de configuración de Containerlab de manera automatizada a nivel de servidor. Si bien ya existen herramientas especializadas en el diseño gráfico de redes que generan dichos ficheros, integrar estas capacidades dentro de la plataforma permitiría un flujo de trabajo unificado, evitando la dependencia de software externo y centralizando todo el ciclo de vida de la infraestructura en un único entorno.
- **Integración de monitorización en tiempo real:** Aunque la plataforma refleja el estado operativo de los nodos, se propone conectar el plano de datos de los contenedores con herramientas de monitorización analíticas como Grafana ¹ o Prometheus ². De esta manera, se podrían capturar las métricas de las interfaces virtuales de Linux facilitando la visualización de gráficas de consumo de ancho de banda y rendimiento a lo largo del tiempo directamente desde el panel de control del usuario.
- **Gestión multiusuario con aislamiento en entornos corporativos:** Si bien el sistema actual permite gestionar topologías de forma independiente mediante la persistencia de ficheros YAML, el despliegue se realiza sobre un espacio de nombres de red compartido en el servidor anfitrión. La mejora propuesta consiste en evolucionar hacia un modelo de aislamiento de red profundo: el sistema garantizaría que cada usuario opere en un entorno de red totalmente aislado. Esto no solo evitaría colisiones de direccionamiento IP entre laboratorios de distintos usuarios, sino que proporcionaría un aislamiento total del tráfico, permitiendo una convivencia segura y eficiente en entornos corporativos de alta densidad.
- **Sistema de autenticación híbrido:** El acceso actual a la plataforma está ligado exclusivamente a Google Auth. Como línea futura, se plantea expandir este sistema hacia un modelo híbrido que incorpore un registro tradicional mediante correo y contraseña cifrada, o la integración de otros proveedores institucionales (como cuentas universitarias o GitHub). Esto eliminará la dependencia de un único proveedor y garantizará el acceso a usuarios que no dispongan de cuentas de Google.
- **Biblioteca de plantillas comunitarias:** La sección actual de plantillas funciona como un catálogo estático precargado. Se propone evolucionarlo hacia un entorno colaborativo donde los usuarios puedan publicar sus propias topologías de red y descargar las creadas por otros miembros de la plataforma. Este sistema requeriría implementar un control de versiones para los archivos YAML y expandir el buscador actual (basado en títulos) mediante la incorporación de filtros avanzados por autor, valoraciones de la comunidad y etiquetas de tecnologías (ej. BGP, OSPF, VLAN), destacando así los escenarios de red más eficientes y mejor documentados.

¹Grafana: <https://grafana.com/>

²Prometheus: <https://prometheus.io/>

8.2.2. Mejoras técnicas y de calidad del software

- **Implementación de una batería de pruebas automatizadas:** El aseguramiento de la calidad de la plataforma se ha realizado mediante la ejecución de pruebas de caja negra operando directamente sobre la interfaz web. Una línea futura de trabajo sería desarrollar una arquitectura de pruebas unitarias y de integración para verificar de forma automática los endpoints de FastAPI y validar el correcto flujo de los componentes de React.
- **Limpieza y análisis de código:** Siguiendo las directrices de métricas de calidad del software (*Clean Code*) [107], se plantea una revisión del código del servidor y del cliente asistida por herramientas de análisis estático, permitiendo corregir el código duplicado, eliminar deuda técnica debido a los plazos académicos y mejorar la legibilidad, comprensión y facilidad de mantenimiento del código en general.
- **Refactorización de los estados y mejoras en el frontend:** A medida que el número de escenarios en ejecución escala, la carga de datos puede saturar la renderización. Se plantea migrar la lógica de estados hacia herramientas de gestión global de estados, tales como **Zustand** [108] o **Redux Toolkit** [109]. Estas librerías permiten mantener un único almacén de datos accesible globalmente, lo que optimiza la frecuencia de las peticiones API y mejora significativamente la reactividad de la interfaz, asegurando que solo los componentes que dependen de un dato específico se actualicen cuando este cambie.
- **Robustecimiento de la seguridad y el control de sesiones:** Se propone implementar un ciclo de vida estricto para los tokens devueltos por Google, gestionando su almacenamiento mediante cookies seguras para mitigar los ataques de robo de sesión. Además, se plantea el cifrado simétrico de las credenciales privadas de los laboratorios o claves API que los usuarios puedan almacenar en la base de datos MySQL, concluyendo con el despliegue definitivo de la plataforma bajo el protocolo seguro HTTPS (*Hypertext Transfer Protocol Secure*) para blindar todas las comunicaciones.

8.3. Valoración personal

Desde una perspectiva personal, este trabajo ha sido el reto más exigente y, a la vez, el más gratificante de mi etapa universitaria. Este TFG comenzó en octubre partiendo de un tema por completo desconocido para mí, pero motivado por dos razones principales: la primera, que se alineaba con el trabajo que iba a desarrollar en mis prácticas de empresa y, la segunda, el deseo de salir de mi zona de confort y no limitarme a realizar lo típico o usual.

La vida está compuesta de etapas muy diferenciadas entre sí: la universitaria y la profesional. Esta última se caracteriza por la existencia de problemas de gran magnitud a los que deberé enfrentarme de manera independiente y, precisamente en ese aspecto, este proyecto ha marcado un punto de inflexión. Enfrentarme a una idea inicial que no tenía un rumbo fijo, y en la que fui adaptando el sistema con las tecnologías y conocimientos que adquiriría a lo largo del tiempo, me ha obligado a madurar no solo como

Conclusiones

desarrollador sino también como persona; me ha enseñado a “sacarme las castañas del fuego” por mí mismo cuando la solución no era evidente y el proyecto parecía imposible.

Gracias a este proceso de investigación y desarrollo autónomo, he dejado de depender de las pautas guiadas de las asignaturas para adquirir una capacidad de resolución de problemas propia. He mejorado mis habilidades y conocimientos en ciertos lenguajes y he aprendido otros nuevos, pero eso no se equipara con la madurez profesional y la seguridad obtenida al saber que puedo analizar, estructurar y resolver problemas técnicos complejos en el mundo laboral. Al fin y al cabo, surgen nuevas tecnologías y las que ya conocías quedan obsoletas, mientras que las *soft skills* permanecen y marcan la diferencia entre un profesional y otro.

Referencias

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [2] Ponemon Institute, "Cost of a data breach report," IBM Security, Tech. Rep., 2023.
- [3] M. Casado, T. Koponen, S. Shenker, and A. Tootoonchian, "Fabric: A retrospective on evolving sdn," in *Proc. 1st Workshop Hot Topics Software Defined Netw. (HotSDN '12)*. ACM, 2012, pp. 85–90. [Online]. Available: <https://yuba.stanford.edu/~casado/fabric.pdf>
- [4] Open Networking Foundation (ONF). Software-defined networking (sdn) definition. Sin fecha de publicación. [Online]. Available: <https://opennetworking.org/sdn-definition/>
- [5] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Harlow, Reino Unido: Pearson Education, 2017, cap. 5.
- [6] IBM. (2024) ¿qué es la automatización de red? IBM Topics. [Online]. Available: <https://www.ibm.com/es-es/topics/network-automation>
- [7] Cisco Systems. (2024) Cisco intersight: Hybrid cloud operations platform. Plataforma que ejemplifica el uso de capas de abstracción y automatización para la gestión de infraestructura heterogénea. [Online]. Available: <https://www.cisco.com/site/us/en/products/computing/hybrid-cloud-operations/intersight-platform/index.html>
- [8] —, "2024 global networking trends report: Business agility through cloud-first operations," Cisco Systems, Inc., Technical Report, 2024. [Online]. Available: https://www.cisco.com/c/dam/global/en_uk/solutions/enterprise-networks/2024-global-networking-trends.pdf
- [9] Kyndryl. (2026) Sdn frente a las redes tradicionales: ¿cuáles son las diferencias? [Online]. Available: <https://www.kyndryl.com/mx/es/learn/sdn-vs-traditional-networking>
- [10] Red Hat. (2026) ¿qué es el networking definido por software (sdn)? [Online]. Available: <https://www.redhat.com/es/topics/hyperconverged-infrastructure/what-is-software-defined-networking>
- [11] Trevenque Group. (2026) Redes definidas por software o sdn: qué son y ventajas. [Online]. Available: <https://www.trevenque.es/blog/posts-cloud/redes-definidas-por-software-o-sdn-que-son-y-ventajas/>

- [12] Ciberseguridad Blog. (2026) Seguridad en las redes definidas por software (sdn). [Online]. Available: <https://ciberseguridad.blog/seguridad-en-las-redes-definidas-por-software-sdn/>
- [13] IBM. (2026) ¿qué es el plano de control (control plane)? [Online]. Available: <https://www.ibm.com/es-es/think/topics/control-plane>
- [14] Cloudflare. (2026) ¿qué es el plano de control (control plane)? [Online]. Available: <https://www.cloudflare.com/es-es/learning/network-layer/what-is-the-control-plane/>
- [15] Eclass Virtual. (2026) Los planos de datos, control y gestión para el ccna 200-301. [Online]. Available: <https://eclassvirtual.com/los-planos-de-datos-control-y-gestion-para-el-ccna-200-301/>
- [16] Open Networking Foundation, “SDN architecture, issue 1,” Open Networking Foundation (ONF), Tech. Rep. TR-502, June 2014. [Online]. Available: https://opennetworking.org/wp-content/uploads/2013/02/TR_SDN_ARCH_1.0_06062014.pdf
- [17] Geekflare. (2026) ¿qué es sdn (software-defined networking)? arquitectura y beneficios. [Online]. Available: <https://geekflare.com/es/software-defined-networking/>
- [18] Ciphersafety. (2026) Redes sdn: ¿qué son y cómo funcionan? [Online]. Available: <https://ciphersafety.com/redes-sdn-que-son/>
- [19] FS.com. (2026) Redes definidas por software (sdn): tipos, ventajas y aplicaciones. [Online]. Available: <https://www.fs.com/es/blog/software-defined-networking-sdn-types-advantages-and-applications-4870.html>
- [20] Wikipedia. (2026) Redes definidas por software. [Online]. Available: https://es.wikipedia.org/wiki/Redes_definidas_por_software
- [21] Network Academy. (2026) Sd-wan underlay vs overlay routing. [Online]. Available: <https://www.networkacademy.io/ccie-enterprise/sdwan/underlay-vs-overlay-routing>
- [22] J. Gentry. (2014, May) eBay in production: Migration from Nova-network to Neutron. Superuser. [Online]. Available: <https://superuser.openinfra.org/articles/eBay-in-production-migration-from-nova-network-to-neutron/>
- [23] Juniper Networks. (2024) Gap inc. case study. Juniper Networks. [Online]. Available: <https://www.juniper.net/mx/es/customers/gap-case-study.html>
- [24] RedesTelecom. (2014, mar) Las redes SDN llegan a la universidad. RedesTelecom. [Online]. Available: <https://www.redestelecom.es/comunicaciones/las-redes-sdn-llegan-a-la-universidad/>
- [25] V. Bahl. (2024, June) Born in the research lab a decade ago, SWAN continues to accelerate networking in the Microsoft Cloud. Microsoft Research Blog. [Online]. Available: <https://www.microsoft.com/en-us/research/blog/born-in-the-research-lab-a-decade-ago-swan-continues-to-accelerate-networking-in-the-microsoft-cloud/>
- [26] InfSeg. (2016) Comencemos con hypervirtualización. [Online]. Available: <https://infseg.com/index.php/2016/02/07/comencemos-con-hypervirtualizacion/>

- [27] Red Hat. (2026) ¿qué es la virtualización nativa de contenedores? [Online]. Available: <https://www.redhat.com/es/topics/containers/what-is-container-native-virtualization>
- [28] ——. (2024) ¿qué es la infraestructura como código (IaC)? [Online]. Available: <https://www.redhat.com/es/topics/automation/what-is-infrastructure-as-code-iac>
- [29] Museos Abiertos. (2026) 1.5. modelos de datos y sus formatos de serialización. [Online]. Available: <https://docs.museosabiertos.org/es/datos-vinculados/modelado/1-introduccion/1-5-modelos-de-datos-y-sus-formatos-de-serializacion>
- [30] Cisco. (2026) ¿qué es la programación de redes? [Online]. Available: <https://gblogs.cisco.com/la/que-es-la-programacion-de-redes/>
- [31] CiscoZine. (2010) The PPDIOO network lifecycle. [Online]. Available: <https://www.ciscozine.com/the-ppdioo-network-lifecycle/>
- [32] GNS3 coTechnologies. (2024) Gns3 documentation. [Online]. Available: <https://docs.gns3.com/>
- [33] EVE-NG Ltd. (2024) Eve-ng professional documentation. [Online]. Available: <https://www.eve-ng.net/index.php/documentation/>
- [34] I. Pepelnjak. (2024) netlab: Virtual networking labs made easy. [Online]. Available: <https://netlab.tools/>
- [35] HashiCorp. (2026) Terraform: Infrastructure as code. HashiCorp. [Online]. Available: <https://www.terraform.io/>
- [36] Cisco Systems. (2024) Cisco modeling labs (cml) - data sheet. [Online]. Available: <https://www.cisco.com/c/en/us/products/cloud-systems-management/modeling-labs/index.html>
- [37] Cisco DevNet. (2024) Nextui framework: A network-specific topology visualization library. [Online]. Available: <https://developer.cisco.com/site/ne-xt/>
- [38] Layer5 Authors. (2024) Meshery: The cloud native manager. [Online]. Available: <https://meshery.io/>
- [39] Sentries. (2023) Diferencias entre el desarrollo iterativo e incremental. Sentries Blog. [Online]. Available: <https://sentries.io/blog/diferencias-entre-desarrollo-iterativo-e-incremental/>
- [40] N. Jerez. (2019) Diseño iterativo, la metodología que perfeccionará tus proyectos. Sueños Gráficos (Medium). [Online]. Available: <https://medium.com/sue~{n}osgraficos/dise~{n}o-iterativo-la-metodologia-que-perfeccionara-tus-proyectos-21034b0d277e>
- [41] Agile Academy. Qué es kanban. Sin fecha de publicación. [Online]. Available: <https://www.agile-academy.com/es/diccionario-agil/kanban/>
- [42] Consuunt. (2023) El método moscow: prioriza las tareas de tu proyecto. [Online]. Available: <https://www.consuunt.es/el-metodo-moscow/>

- [43] Scrum Manager. (2023) Gold-plating: evitando el exceso en proyectos y negocios. Scrum Manager Blog. [Online]. Available: <https://www.scrummanager.com/blog/2023/11/goldplating-evitando-el-exceso-en-proyectos-y-negocios/>
- [44] Project Management Institute, *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)*, 6th ed. Newtown Square, PA, EE. UU.: Project Management Institute, 2017.
- [45] Playbook Team. (2023, jul) How to calculate risk exposure: Formulas and examples. Playbook Resources. [Online]. Available: <https://playbookteam.com/resources/blog/calculate-risk-exposure>
- [46] Glassdoor. (2026) Sueldos para ingeniero devops en España. [Online]. Available: https://www.glassdoor.es/Sueldos/ingeniero-devops-sueldo-SRCH_KO0,16.htm
- [47] Working Days. (2026) Días laborables en españa en 2026. [Online]. Available: <https://workingdays.info/es/dias-laborables/es/2026>
- [48] Microsoft. (2024) Microsoft Teams: Online Meetings and Video Conferencing. [Online]. Available: <https://www.microsoft.com/en-us/microsoft-teams/group-chat-software>
- [49] ——. (2024) Microsoft 365 Education: Cloud solutions for schools. [Online]. Available: <https://www.microsoft.com/es-es/education/products/office>
- [50] ——. (2024) Microsoft Outlook: Personal Email and Calendar Software. [Online]. Available: <https://www.microsoft.com/en-us/microsoft-365/outlook/>
- [51] Overleaf. (2024) Overleaf: Real-time Collaborative Writing and Publishing Tools. [Online]. Available: <https://www.overleaf.com/about>
- [52] Git Project Authors. (2024) Git - About Version Control. [Online]. Available: <https://git-scm.com/about>
- [53] GitLab Inc. (2024) Gitlab: The DevSecOps Platform. [Online]. Available: <https://about.gitlab.com/company/>
- [54] GitHub, Inc. (2024) Github: Where the world builds software. [Online]. Available: <https://github.com/about>
- [55] Atlassian. (2024) Bitbucket: Solución de git para equipos que usan jira. [Online]. Available: <http://bitbucket.org/product/es>
- [56] The Linux Foundation. (2024) The Linux Kernel Archives. [Online]. Available: <https://www.kernel.org/>
- [57] Canonical Ltd. (2024) Official Ubuntu Documentation. [Online]. Available: <https://help.ubuntu.com/>
- [58] Kubernetes Authors. (2024) Minikube Drivers: HyperKit. [Online]. Available: <https://minikube.sigs.k8s.io/docs/drivers/hyperkit/>
- [59] Microsoft. (2024) Windows Subsystem for Linux Documentation. [Online]. Available: <https://learn.microsoft.com/en-us/windows/wsl/>

- [60] Containerlab Authors. (2024) Containerlab: Orchestrate networking labs with Docker. [Online]. Available: <https://containerlab.dev/>
- [61] The Kubernetes Authors. (2024) Comunicación entre el nodo y el plano de control. [Online]. Available: <https://kubernetes.io/es/docs/concepts/architecture/control-plane-node-communication/>
- [62] Red Hat. (2024) Red Hat OpenShift: La plataforma de aplicaciones de la nube híbrida. [Online]. Available: <https://www.redhat.com/es/technologies/cloud-computing/openshift>
- [63] Kathará Project Authors. (2024) Kathará: Quick Example and Usage. [Online]. Available: <https://github.com/KatharaFramework/Kathara/tree/main#quick-example>
- [64] Docker Inc. (2024) Docker Engine overview. [Online]. Available: <https://docs.docker.com/engine/>
- [65] Podman Authors. (2024) Podman: A tool for managing containers and pods. [Online]. Available: <https://podman.io/>
- [66] O. Ben-Kiki, C. Evans, and I. döt Net. (2021) YAML Data Serialization Standard 1.2. [Online]. Available: <https://yaml.org/spec/1.2.2/>
- [67] ECMA International. (2024) Introducing JSON. [Online]. Available: <https://www.json.org/json-es.html>
- [68] W3C. (2024) Extensible Markup Language (XML) 1.0. [Online]. Available: <https://www.w3.org/TR/xml/>
- [69] T. Preston-Werner. (2024) TOML: Tom's Obvious Minimal Language. [Online]. Available: <https://toml.io/en/>
- [70] Oracle Corporation. (2024) MySQL 8.4 Reference Manual. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/>
- [71] ——. (2024) MySQL Connector/Python Developer Guide. [Online]. Available: <https://dev.mysql.com/doc/connector-python/en/>
- [72] Amazon Web Services (AWS). (2024) Diferencias entre MySQL y PostgreSQL. [Online]. Available: <https://aws.amazon.com/es/compare/the-difference-between-mysql-vs-postgresql/>
- [73] MongoDB Inc. (2024) The MongoDB 7.0 Manual: Data Modeling Introduction. [Online]. Available: <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>
- [74] Ecosistema Startup. (2023) Concurrencia en SQLite: Importancia y soluciones para Founders. [Online]. Available: <https://ecosistemastartup.com/concurrencia-en-sqlite-importancia-y-soluciones-para-founders/>
- [75] GeeksforGeeks. (2024) Python for DevOps. [Online]. Available: <https://www.geeksforgeeks.org/devops/python-for-devops/>
- [76] Python Software Foundation. (2024) What is Python? Executive Summary. [Online]. Available: <https://www.python.org/doc/essays/blurbs/>

- [77] Profile Software Solutions. (2023) ¿qué es Go y qué usos tiene el lenguaje de Google? [Online]. Available: <https://profile.es/blog/que-es-go-y-que-usos-tiene/>
- [78] Free Software Foundation. (2024) Bash Reference Manual. [Online]. Available: <https://www.gnu.org/software/bash/>
- [79] OpenJS Foundation. (2024) Acerca de Node.js. [Online]. Available: <https://nodejs.org/es/about>
- [80] Meta Platforms, Inc. (2024) React: The library for web and native user interfaces. [Online]. Available: <https://react.dev/>
- [81] —. (2024) Thinking in React. [Online]. Available: <https://react.dev/learn/thinking-in-react>
- [82] Google. (2024) What is Angular? [Online]. Available: <https://angular.dev/overview>
- [83] E. You. (2024) Introduction to Vue.js. [Online]. Available: <https://vuejs.org/guide/introduction.html>
- [84] Mozilla Resources. (2024) Introduction to the DOM. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- [85] Microsoft. (2024) Visual Studio Code: Code Editing. Redefined. [Online]. Available: <https://code.visualstudio.com/>
- [86] JetBrains. (2024) IDEs vs. Code Editors: Which One Is Right for You? [Online]. Available: <https://blog.jetbrains.com/webstorm/2024/03/ides-vs-code-editors/>
- [87] The Eclipse Foundation. (2024) Eclipse IDE. [Online]. Available: <https://www.eclipse.org/ide/>
- [88] JetBrains. (2024) PyCharm: The Python IDE for Professional Developers. [Online]. Available: <https://www.jetbrains.com/pycharm/>
- [89] Sublime HQ. (2024) Sublime Text: The sophisticated text editor for code, markup and prose. [Online]. Available: <https://www.sublimetext.com/>
- [90] GitHub. (2022) Sunsetting Atom. [Online]. Available: <https://github.blog/2022-06-08-sunsetting-atom/>
- [91] Vim Community. (2024) Vim: The ubiquitous text editor. [Online]. Available: <https://www.vim.org/>
- [92] Free Software Foundation. (2024) GNU nano: The friendly text editor. [Online]. Available: <https://www.nano-editor.org/>
- [93] Tesify. (2024) Cómo declarar el uso de ia en tu tfg paso a paso y sin riesgos. [Online]. Available: <https://tesify.es/declarar-uso-ia-tfg-paso-a-paso-sin-riesgos>
- [94] Google. (2026) Gemini: Modelos de inteligencia artificial. [Online]. Available: <https://gemini.google.com/>
- [95] Anthropic. (2026) Claude code: Ai coding agent documentation. [Online]. Available: <https://code.claude.com/docs/>

- [96] Microsoft. (2022) The mvvm pattern. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>
- [97] A. Shvets. (2024) Patrones de diseño. Refactoring.Guru. [Online]. Available: <https://refactoring.guru/es/design-patterns>
- [98] Microsoft. (2023) Azure application architecture guide. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/>
- [99] KeepCoding. (2024, apr) Patrón de inyección de dependencias | keepcoding bootcamps. [Online]. Available: <https://keepcoding.io/blog/patron-de-inyeccion-de-dependencias/>
- [100] M. Fowler. (2023) Data transfer object. [Online]. Available: <https://martinfowler.com/eaCatalog/dataTransferObject.html>
- [101] A. Shvets. (2024) Observer. Refactoring.Guru. [Online]. Available: <https://refactoring.guru/design-patterns/observer>
- [102] Microsoft. (2026) Protocolo de concesión de código de autorización de oauth 2.0. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-auth-code-flow>
- [103] D. Torres Burriel. (2021) Minimalismo en el diseño ux. Torresburriel Estudio. [Online]. Available: <https://torresburriel.com/weblog/minimalismo-en-el-diseno-ux/>
- [104] HubSpot. (2026) Qué es la usabilidad web. HubSpot España. [Online]. Available: <https://blog.hubspot.es/website/que-es-usabilidad-web>
- [105] J. Brooke, *SUS: A 'Quick and Dirty' Usability Scale*. CRC Press, 1996, pp. 189–194. [Online]. Available: <https://www.taylorfrancis.com/chapters/edit/10.1201/9781498710411-35/sus-quick-dirty-usability-scale-john-brooke>
- [106] Teacup Lab. (2026) Qué es la escala sus y cómo usarla para medir la usabilidad. Teacup Lab. [Online]. Available: <https://www.teacuplab.com/es/blog/que-es-la-escala-sus-y-como-usarla-para-medir-la-usabilidad/>
- [107] Viewnext. (2026) Clean code: qué es y por dónde empezar. Viewnext. [Online]. Available: <https://www.viewnext.com/clean-code-que-es-y-por-donde-empezar/>
- [108] Poimandres. (2026) Zustand: Bear necessities for state management in react. [Online]. Available: <https://zustand-demo.pmnd.rs/>
- [109] Redux.js.org. (2026) Redux toolkit: The official, opinionated, batteries-included toolset for efficient redux development. [Online]. Available: <https://redux-toolkit.js.org/>

Anexos

Anexo A

Manual de despliegue

Este manual detalla las instrucciones necesarias para preparar el entorno de trabajo y llevar a cabo el despliegue completo de la plataforma, abarcando desde los requisitos previos de software y configuración de dependencias, hasta los procesos específicos para la puesta en marcha tanto del frontend como del backend

A.1. Requisitos previos

Para garantizar el despliegue y correcto funcionamiento del sistema, es necesario cumplir con las especificaciones técnicas detalladas a continuación.

■ Requisitos de infraestructura y orquestación en el host:

- **SO:** Distribución Linux (Ubuntu 22.04 LTS o superior recomendada)
- **Docker:** Motor de contenedores Docker instalado y en ejecución
- **Containerlab:** Versión estable más reciente para la orquestación de nodos de red
- **Privilegios:** El usuario que ejecute el backend debe tener permisos de ejecución para comandos de Docker (sudo o pertenencia al grupo docker)

■ Requisitos para el frontend:

- **Node.js:** Versión 20.19.0 o superior
- **npm:** Versión 10.0.0 o superior
- **Gestor de paquetes:** npm
- **Framework:** React 19.2.5
- **Herramientas de construcción:** Vite 8.0.10 con el plugin de TailwindCSS para Vite

■ Requisitos para el backend:

- **Python:** Versión 3.10 o superior

- **Gestor de dependencias:** pip (se recomienda el uso de un entorno virtual)
- **Framework:** FastAPI 0.136.1
- **Servidor ASGI:** Uvicorn 0.46.0
- **Base de datos:** MySQL 8.0
- **Cliente de la base de datos:** mysql-connector-python 9.7.0

A.2. Instalación de herramientas y configuración

A.2.1. Preparación del entorno (HOST)

Primero, asegúrese de disponer de las herramientas necesarias para la orquestación de contenedores y de los lenguajes de programación requeridos:

■ Instalación de Docker:

- Instale el motor de Docker siguiendo la documentación oficial para su distribución Linux.
- Añada su usuario al grupo de Docker para evitar el uso de sudo en cada petición: `sudo usermod -aG docker $USER` (requiere iniciar la sesión).

■ Instalación de Containerlab:

- Utilice el script de instalación automática: `bash -c "$(curl -sL https://containerlab.dev/setup)"`

■ Entorno Backend (Python):

- Clone el repositorio <https://gitlab.inf.uva.es/fraalva/containerlab-orchestrator.git> en su máquina local.
- Cree y active un entorno virtual para gestionar las librerías: `python3 -m venv venv source venv/bin/activate`
- Instale las dependencias necesarias: `pip install -r requirements`

A.2.2. Configuración de las variables de entorno

En la raíz del backend, dentro de la carpeta backend, existe un archivo `.env` que debe ser modificado para la configuración de la base de datos y la autenticación. Modifique el fichero existente de la siguiente manera.

```
GOOGLE_CLIENT_ID=tu_cliente_id_de_google  
JWT_SECRET=secreto_para_la_autenticacion_de_jwt  
DB_HOST=localhost  
DB_USER=usuario_base_de_datos  
DB_PASSWORD=contraseña_usuario_base_datos
```

A.2.3. Configuración de la base de datos

- Ejecución del script de creación de la base de datos que se encuentra dentro de la carpeta script en la carpeta base del proyecto:

```
mysql -u root -p <creacionBaseDatos.sql
```

- Opcionalmente, para poblar la base de datos con las plantillas de topologías preconfiguradas, se puede ejecutar el script de inicialización de datos:

```
mysql -u root -p <poblar_plantillas.sql
```

- Creación de usuario en la base de datos y concesión de privilegios sobre la base de datos:

```
CREATE USER 'usuario_base_datos'@'localhost' IDENTIFIED BY 'tu_contraseña'; GRANT  
ALL PRIVILEGES ON TFG.* TO 'usuario_base_datos'@'localhost'; FLUSH PRIVILEGES;
```

Las credenciales (usuario y contraseña) especificadas en el archivo `.env` deben coincidir exactamente con las que se configuren en la base de datos.

A.2.4. Instalación del Frontend

- **Instalación:** `npm install`

A.3. Despliegue del proyecto

Una vez configurado el entorno y la base de datos según se detalla en el apartado anterior, el despliegue de la plataforma se realiza mediante la ejecución de los servicios del frontend y el backend. Para asegurar la persistencia de ambos servicios durante su uso, se recomienda mantener ambas terminales activas.

- **Lanzamiento del Frontend** El frontend debe iniciarse primero para preparar el entorno de la interfaz de usuario.
 - Acceda a la carpeta *frontend*.

- Inicie el entorno de desarrollo de Vite: `npm run dev`.

La interfaz será accesible a través de la dirección proporcionada por Vite: `http://localhost:5173`

- **Lanzamiento del Backend** Una vez que el frontend está activo, inicie el backend para establecer la conexión con la base de datos y habilitar los endpoints de la API.

- Acceda a la carpeta `backend`
- Active el entorno virtual mediante el comando: `source venv/bin/activate`
- Ejecute el servidor con Uvicorn: `uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`

El sistema estará disponible en `http://localhost:8000`

A.4. Verificación del despliegue

Tras el despliegue, el sistema puede validarse siguiendo estos pasos:

- **Comprobación de la API:** Acceda a `http://localhost:8000/docs` para visualizar la documentación interactiva Swagger y verificar que los endpoints están activos.
- **Comprobación de la interfaz:** Acceda a `http://localhost:5173` y confirme que el inicio de sesión mediante Google Auth redirige correctamente al dashboard.

Anexo B

Manual de uso

En este anexo se describe el funcionamiento de la aplicación desde el punto de vista de los usuarios. Mediante capturas de pantalla, se mostrarán las principales funcionalidades de la aplicación, así como los flujos de acciones que los usuarios deben seguir para completar las tareas principales.

B.1. Inicio de sesión

Para acceder a S2 Orchestrator, el usuario debe dirigirse a la URL de la aplicación, donde se presenta la pantalla de inicio de sesión (ver Figura B.1).

En esta pantalla se muestra el nombre del sistema, una breve descripción de su propósito (sistema de orquestación de redes) y el estado actual de los servicios principales:

- **API Status:** Indica si la API del backend se encuentra operativa.
- **Engine:** Indica si el motor de emulación de redes (Containerlab) está listo para su uso.

El acceso a la aplicación se realiza mediante autenticación con una cuenta de Google, pulsando el botón “*Continuar con Google*”. Una vez completado el proceso de autenticación, el usuario es redirigido a la pantalla principal de la aplicación

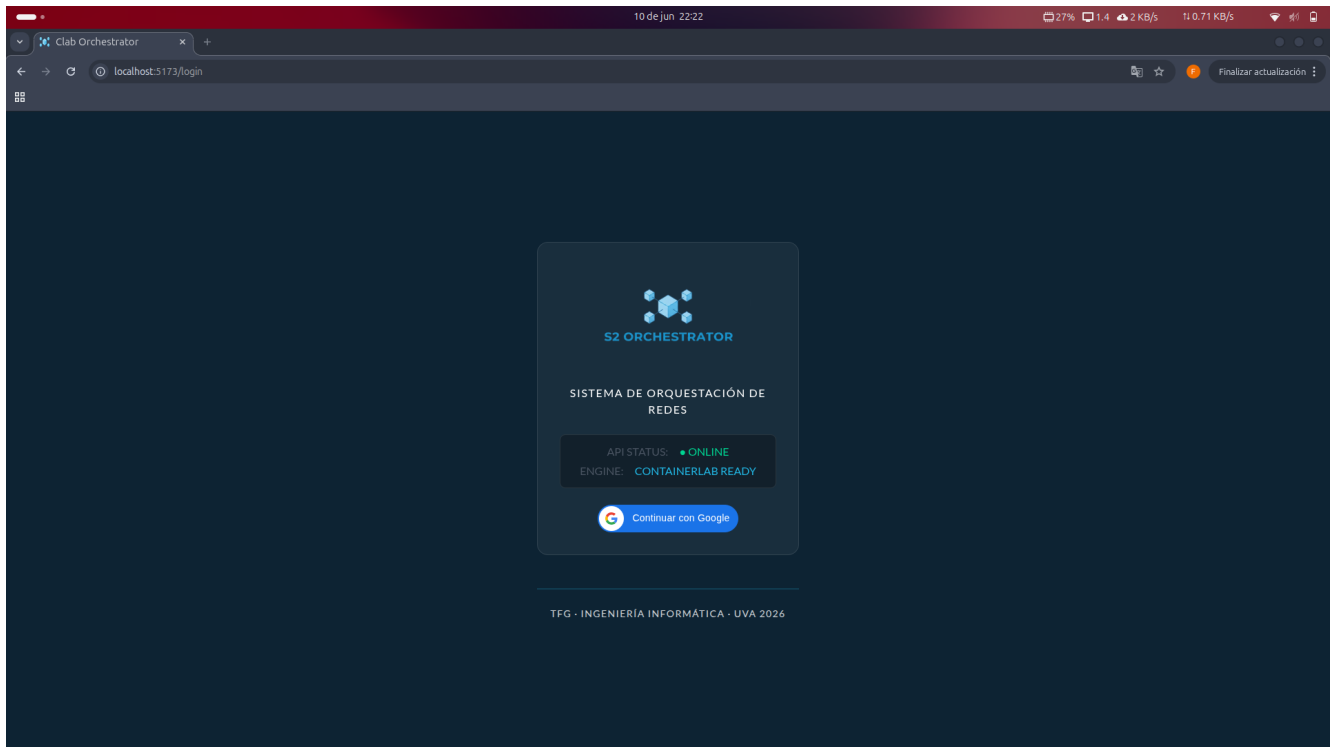


Figura B.1: Pantalla de inicio de sesión de S2 Orchestrator.

B.2. Pantalla principal: Mis topologías

Tras iniciar sesión, el usuario accede a la pantalla principal de la aplicación, correspondiente a la sección “*Mis topologías*” (ver Figura B.2). Esta es la vista por defecto y muestra un listado con todas las topologías creadas por el usuario.

En la parte superior se encuentra el botón “+ Nueva Topología”, que permite crear una nueva topología desde cero. Justo debajo se sitúa un buscador que permite filtrar las topologías por nombre, así como un menú desplegable para ordenar el listado (por defecto, por fecha de creación, de más reciente a más antigua o viceversa y por nombre, en orden alfabético ascendente o descendente).

Cada elemento de la lista corresponde a una topología guardada y muestra la siguiente información:

- **Nombre** de la topología.
- **Estado**, indicando si la topología está actualmente *Almacenada*, *En despliegue*, *Activa* o *En destrucción*.
- **Descripción** con un breve resumen de los elementos que componen la red y la finalidad de la topología.
- **Fecha de creación**.

A la derecha de cada topología se muestran una serie de acciones rápidas, representadas mediante iconos:

- **Desplegar/Detener** (icono verde/icono amarillo): Inicia el despliegue de la topología en el motor de emulación si la topología está en estado *Almacenada* o detiene y elimina los recursos asignados si esta se encuentra en estado *Activa*. Este botón se muestra deshabilitado mientras la topología se encuentra en los estados *En despliegue* o *En destrucción*, así como cuando ya existe otra topología activa, dado que únicamente se permite tener una topología desplegada.
- **Descargar** (icono naranja): Permite descargar el archivo de definición de la topología. Se encuentra deshabilitado mientras la topología se encuentra en los estados *En despliegue* o *En destrucción*.
- **Eliminar** (icono rojo): Elimina la topología de forma permanente. Se encuentra deshabilitado cuando la topología se encuentra en estado *Activa*, *En despliegue* o *En destrucción*, ya que no es posible eliminar una topología que se encuentra desplegada o en proceso de cambio de estado.
- **Duplicar** (icono azul): Crea una copia de la topología. Se encuentra deshabilitado mientras la topología se encuentra en los estados *En despliegue* o *En destrucción*.

Por último, en el menú lateral izquierdo se encuentran los distintos apartados de la aplicación: *Mis Topologías*, *Plantillas*, *Monitor de Red* (Solo disponible cuando hay una topología activa), *Ajustes de Perfil*, así como la opción para cerrar sesión.

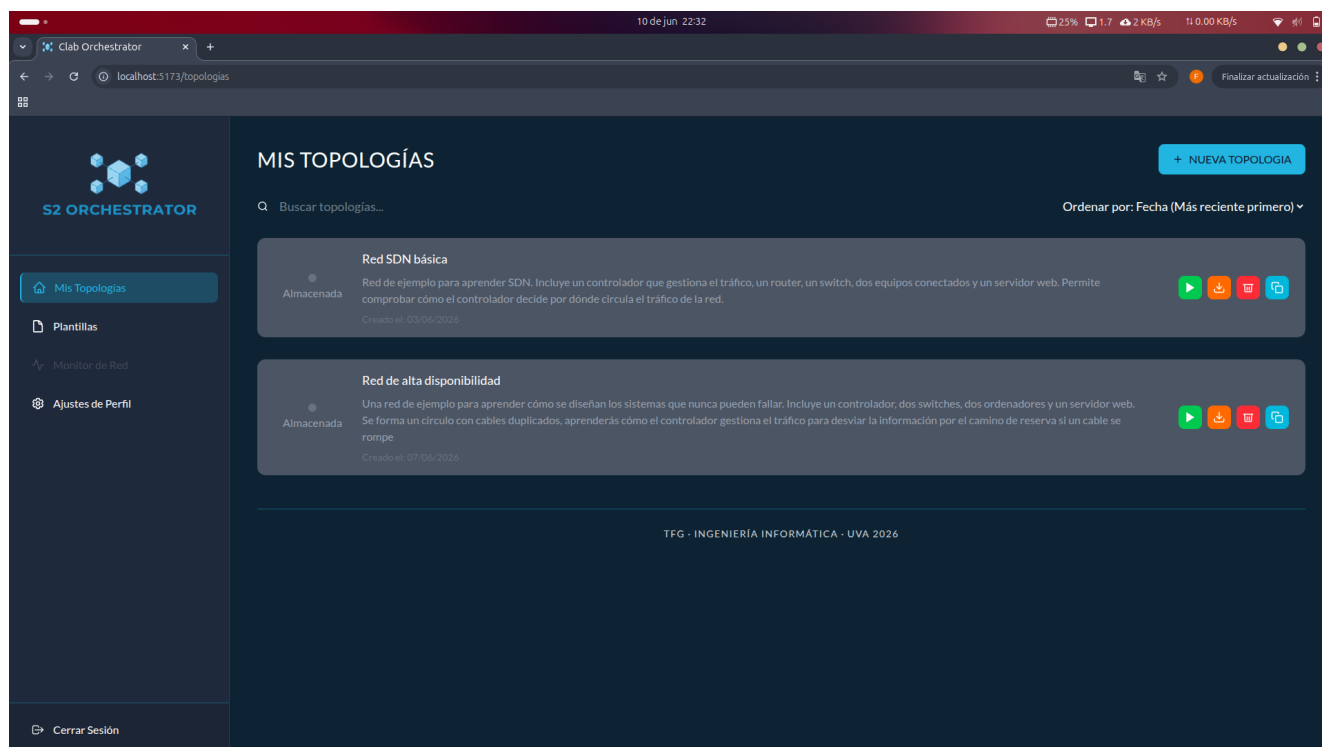


Figura B.2: Pantalla principal de la aplicación: Mis Topologías.

B.3. Creación de una nueva topología

Al pulsar el botón “+ Nueva Topología” desde la pantalla principal, se abre una ventana modal que permite crear una nueva topología de red (ver figura B.3).

En este formulario, el usuario debe completar los siguientes campos:

- **Nombre de la topología:** Nombre que se mostrará posteriormente en el listado de *Mis Topologías*.
- **Descripción:** Texto donde se describe el propósito y los componentes principales de la topología de red.
- **Archivo de Configuración (YAML):** Zona de carga donde el usuario puede arrastrar o seleccionar un archivo de definición de la topología en formato `.clab.yaml`, `.yaml` o `.yml`, correspondiente a la sintaxis de Containerlab. Adicionalmente, también se admite la carga de archivos comprimidos (`.zip`, `.tar.gz` o `.tar.*`) que contengan la topología junto con archivos adicionales necesarios para su despliegue (por ejemplo, configuraciones o ficheros de arranque de los dispositivos).

Al cargar el archivo, la aplicación verifica automáticamente la sintaxis YAML, notificando al usuario si existe algún error en el fichero antes de continuar.

Una vez completados los campos, el usuario puede pulsar el botón “Importar” para crear la topología y añadirla al listado, o “Cancelar” para cerrar la ventana sin guardar los cambios.

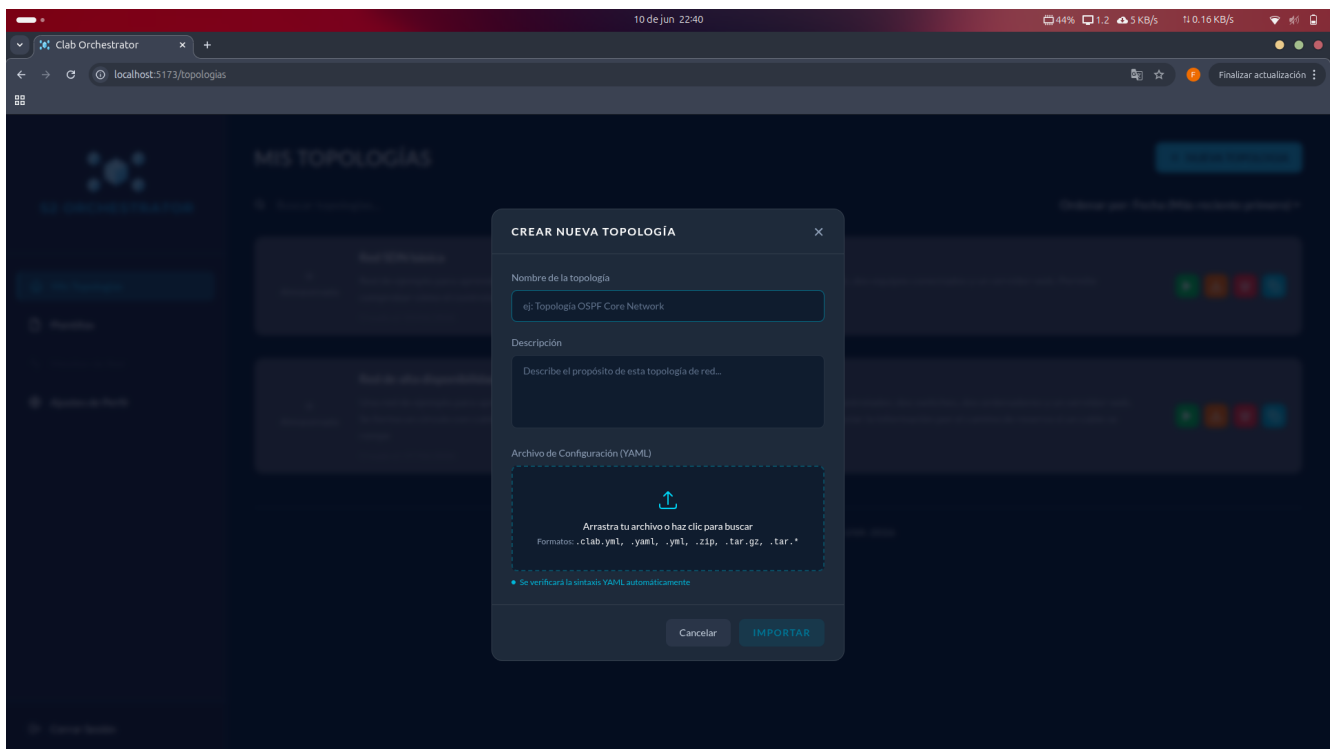


Figura B.3: Ventana de creación de una nueva topología.

B.4. Eliminación de una topología

Al pulsar el icono de *Eliminar* sobre una de las topologías listadas en *Mis Topologías*, se muestra una ventana modal de confirmación que advierte al usuario (ver figura B.4).

En esta ventana se indica que la acción es irreversible, y que, al eliminar la topología se borrarán los registros correspondientes de la base de datos.

Para evitar eliminaciones accidentales, el usuario debe escribir el nombre exacto de la topología en el campo de confirmación. El botón “*Eliminar*” permanecerá deshabilitado hasta que el texto introducido coincida con el nombre de la topología a eliminar. Alternativamente, el usuario puede pulsar “*Cancelar*” para cerrar la ventana sin realizar ningún cambio.

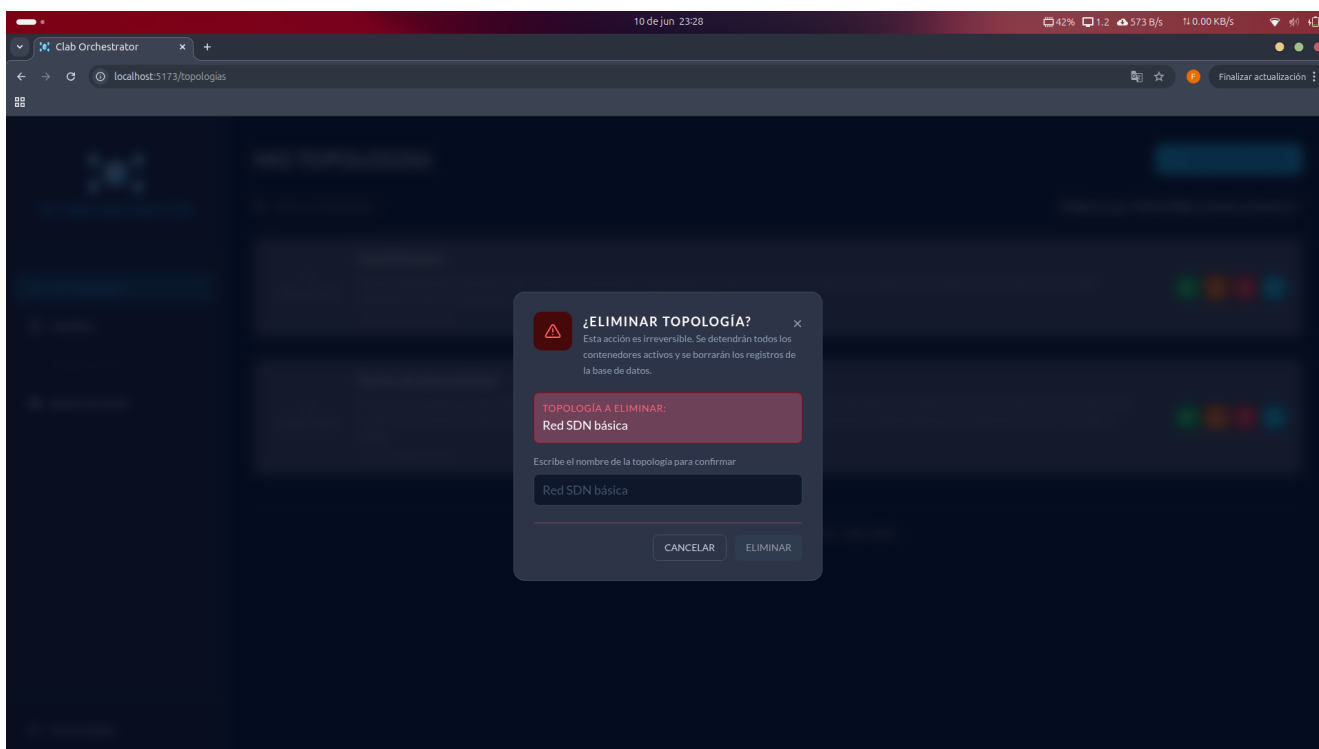


Figura B.4: Ventana de confirmación para la eliminación de una topología.

B.5. Biblioteca de plantillas

La sección *Plantillas* ofrece un conjunto de escenarios de red preconfigurados que permiten desplegar una topología de forma rápida sin necesidad de crearla desde cero (ver figura B.5).

Cada plantilla se presenta en una tarjeta que incluye su nombre, la fecha de creación y una descripción detallada de los dispositivos que la componen y del objetivo del escenario.

Para utilizar una plantilla, el usuario debe pulsar el botón “*Usar Plantilla*” de la tarjeta correspondiente, lo que añadirá una copia de dicha topología al listado de *Mis Topologías*, quedando disponible para

su despliegue.

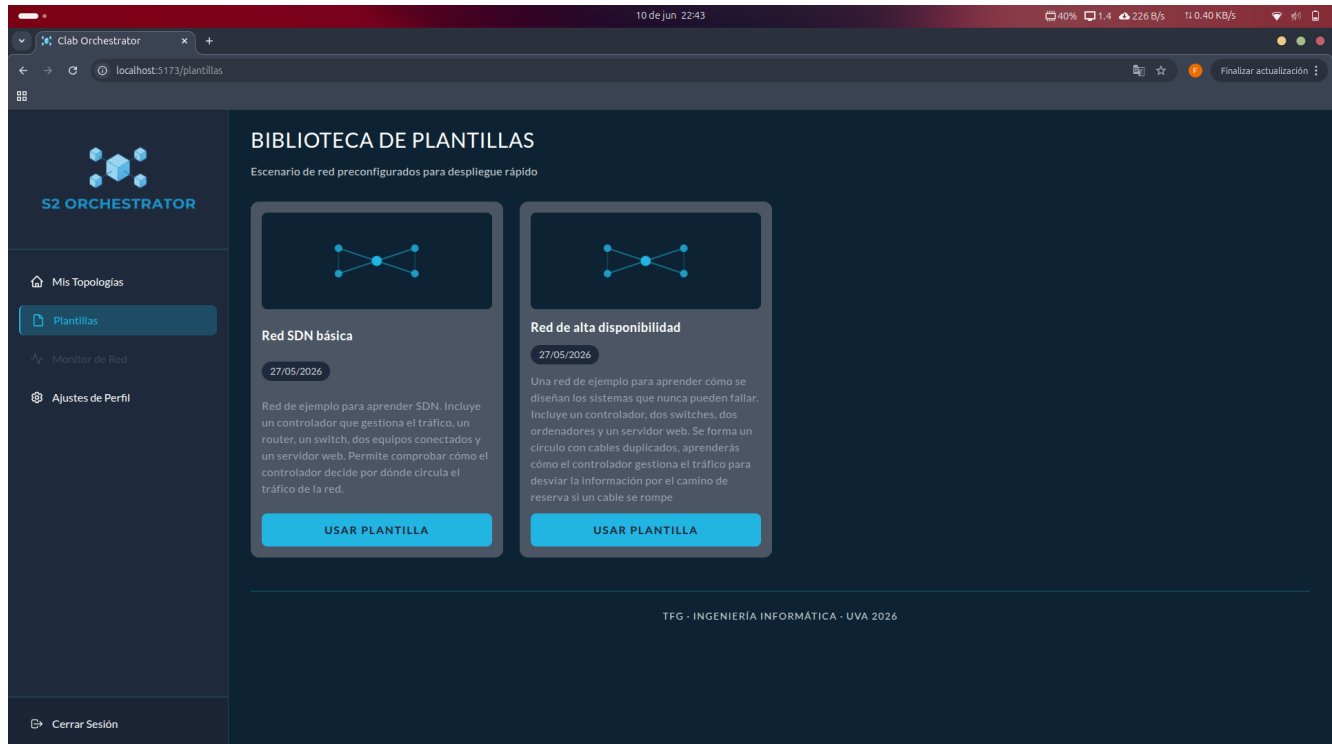


Figura B.5: Biblioteca de plantillas de topologías preconfiguradas.

B.6. Ajustes de Perfil

La sección *Ajustes de Perfil*, accesible desde el menú lateral, permite al usuario consultar y modificar la información asociada a su perfil (ver figura B.6).

Desde esta pantalla es posible:

- **Cambiar el avatar:** Imagen de perfil asociada a la cuenta del usuario, mediante el botón “*Cambiar Avatar*”.
- **Modificar el nombre completo:** Nombre que identifica al usuario dentro de la aplicación.
- **Consultar el correo electrónico:** Dirección asociada a la cuenta, autenticada a través de Google y, por tanto, no editable.
- **Consultar el rol predeterminado:** Rol del usuario dentro del sistema, el cual es asignado por el administrador y no puede ser modificado por el propio usuario.

En la parte inferior de la pantalla se encuentran los siguientes botones de acción:

- **Eliminar Cuenta:** Elimina permanentemente la cuenta del usuario y todos los datos asociados a la misma.

- **Guardar Cambios:** Confirma y almacena las modificaciones realizadas en el perfil.
- **Cancelar:** Descarta los cambios realizados y restaura los valores previos.

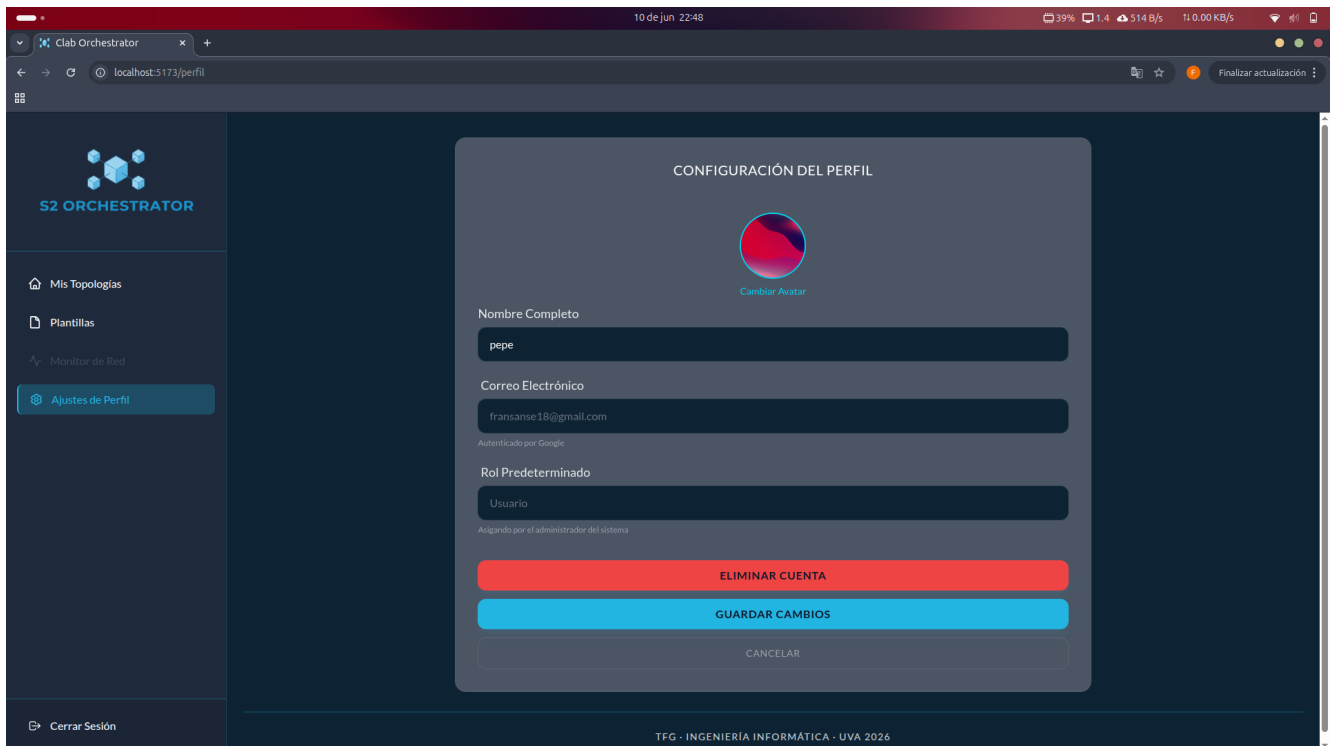


Figura B.6: Pantalla de ajustes del perfil de usuario.

B.7. Monitor de Red

La sección *Monitor de Red* permite visualizar en tiempo real el estado de una topología desplegada (ver figura B.7). Esta sección se divide en tres áreas principales:

- **Visualización de la topología:** Representación gráfica interactiva de los nodos que componen la red y los enlaces existentes entre ellos, generada a partir de la topología desplegada.
- **Salud Global de la Red:** Panel lateral derecho que muestra métricas del estado de la red, entre las que se incluyen:
 - **Nodos Activos:** Número de nodos activos respecto al total de nodos en la topología.
 - **Enlaces de Red:** Número de enlaces activos respecto al total de enlaces.
 - **Latencia Media:** Latencia promedio medida entre los nodos de la red.
- **Logs:** Consola inferior donde se muestran, de forma cronológica, los mensajes generados durante el proceso de despliegue y ejecución de la topología, incluyendo la creación de contenedores, redes *Docker* y enlaces entre dispositivos. Dispone en la parte derecha del icono de una flecha que permite mostrar u ocultar este panel.

Esta sección resulta especialmente útil para comprobar que el despliegue de la topología se ha realizado correctamente y para diagnosticar posibles problemas durante la creación de los nodos o enlaces.

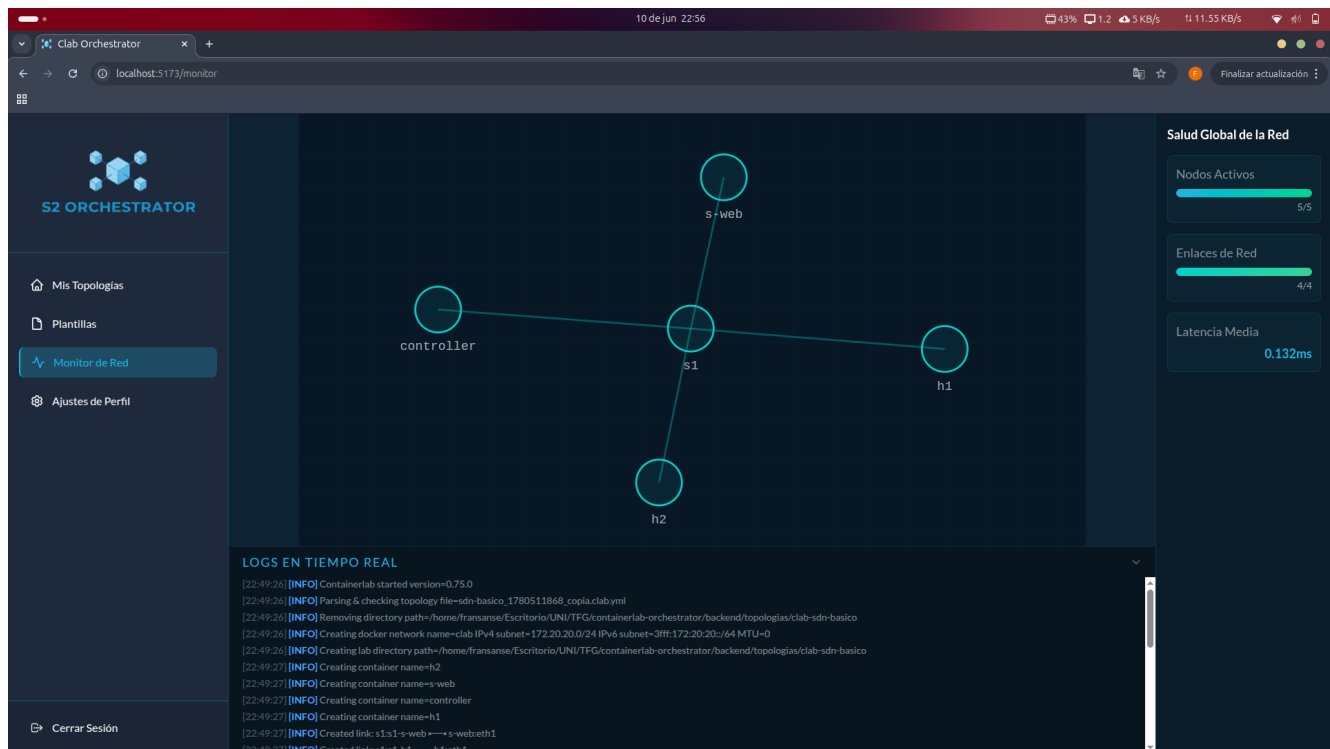


Figura B.7: Pantalla del monitor de red con la topología desplegada y los logs.

B.8. Detalles de un nodo

Al pulsar sobre cualquiera de los nodos representados en la visualización de la topología, el panel lateral derecho cambia para mostrar información detallada y opciones de gestión específicas del nodo seleccionado (ver figura B.8). En la parte superior del panel se indica el nombre del nodo y su estado de sincronización con el sistema.

Este panel se organiza en los siguientes bloques:

- **Métricas en Tiempo Real:** Muestra el porcentaje de uso de CPU y la cantidad de memoria RAM consumida por el nodo en el momento actual.
- **Configuración en Caliente:** Permite consultar y modificar la configuración de red de la interfaz del nodo sin necesidad de reiniciarlo. Incluye los siguientes campos:
 - **Dirección IPv4 y Máscara de red** de la interfaz seleccionada.
 - **Estado del controlador SDN:** Indica si la configuración del nodo está sincronizada con el controlador.

Los cambios realizados se aplican mediante el botón “Aplicar configuración en caliente”.

- **Acciones Rápidas:** Ofrece accesos directos para interactuar con el nodo, entre los que se incluyen:
 - **Mostrar consola:** Abre una consola para visualizar los logs del nodo.
 - **Reiniciar nodo:** Permite reiniciar el contenedor asociado al nodo seleccionado.
 - **Ejecutar comando:** Permite introducir y ejecutar un comando concreto sobre el nodo (por ejemplo, `ip addr show eth1`) y consultar su resultado.

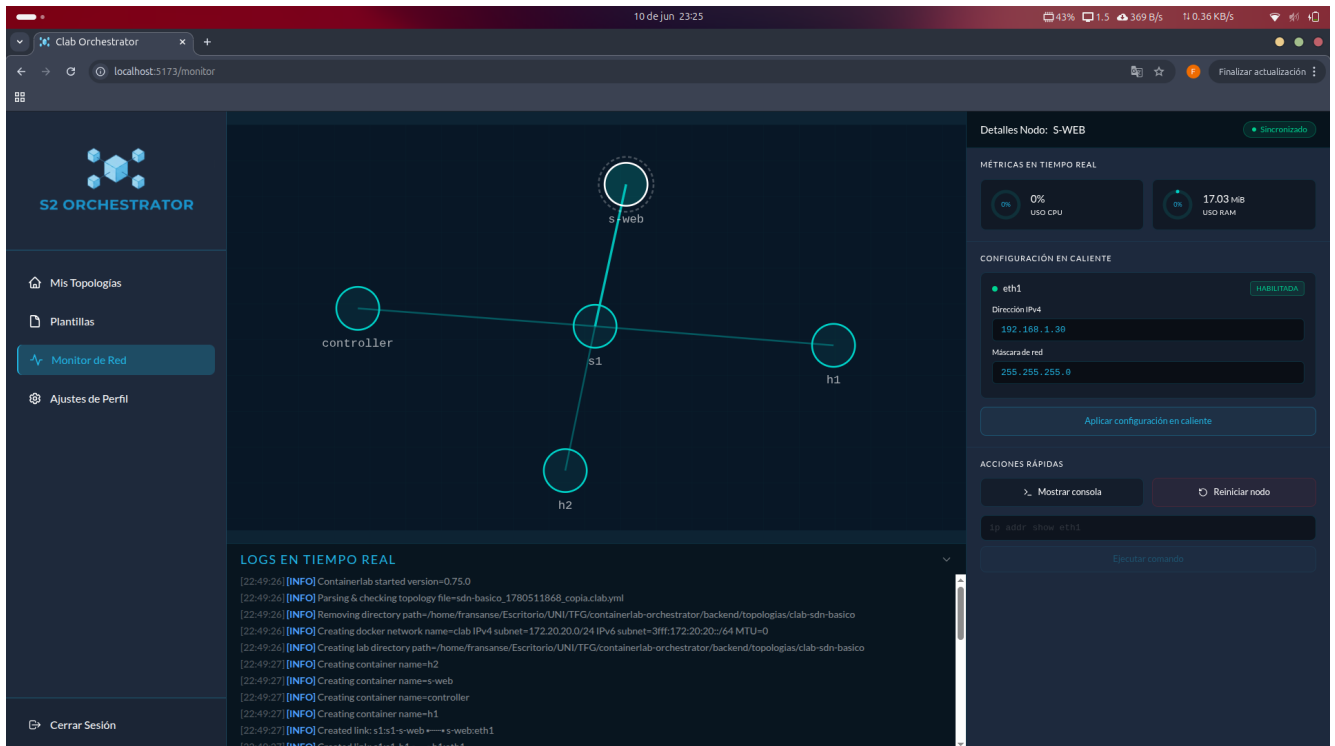


Figura B.8: Panel de detalles y configuración de un nodo seleccionado.

Cabe destacar que el contenido de este panel varía en función del tipo de nodo seleccionado. Por ejemplo, en el caso de los conmutadores (*switches*), el bloque *Configuración en Caliente* no se muestra, ya que estos dispositivos no disponen de una interfaz de red configurable de forma individual desde la aplicación. Asimismo, el campo de *Ejecutar comando* dentro de *Acciones Rápidas* sugiere comandos diferentes según el tipo de dispositivo (por ejemplo, `ovs-vsctl show` para un conmutador OVS frente a `ip addr show eth1` para un equipo final).

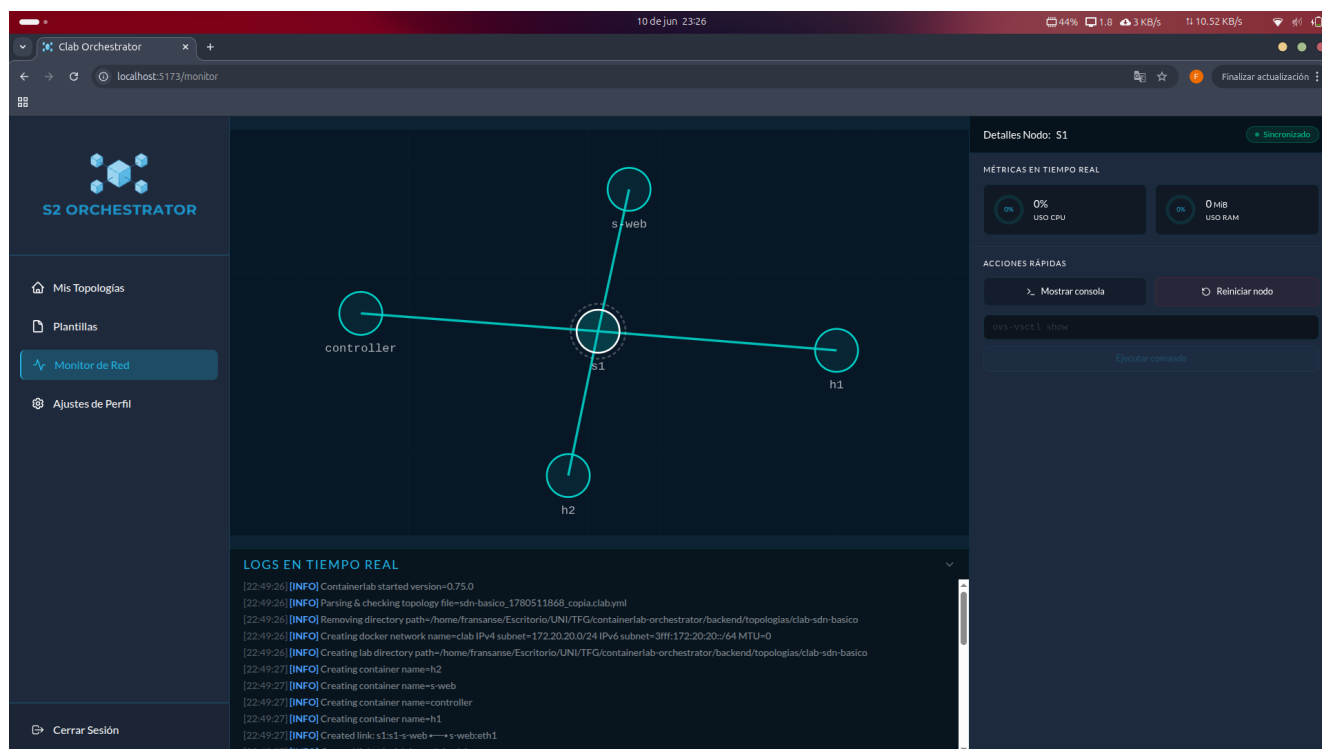


Figura B.9: Panel de detalles de un nodo de tipo conmutador (switch).

B.9. Consola del nodo

Al pulsar el botón “*Mostrar consola*” dentro del panel de detalles de un nodo, se abre una ventana modal denominada “*Visor de logs*”, que muestra los registros generados por el contenedor correspondiente a dicho nodo (ver figura B.10).

Estos registros permiten consultar información relevante sobre el proceso de arranque y funcionamiento interno del nodo, como la configuración de servicios, mensajes de inicialización o eventos relacionados con su ejecución. Al final del listado se muestra el indicador “- *End of log stream* -”, señalando que se han mostrado todos los registros disponibles hasta el momento.

Cabe destacar que la disponibilidad de estos registros depende del tipo de nodo. En el caso de nodos como conmutadores (*switches*), al no generar registros propios accesibles desde la aplicación, el visor muestra un mensaje de advertencia indicando que el nodo no dispone de registros previos (ver figura B.11).

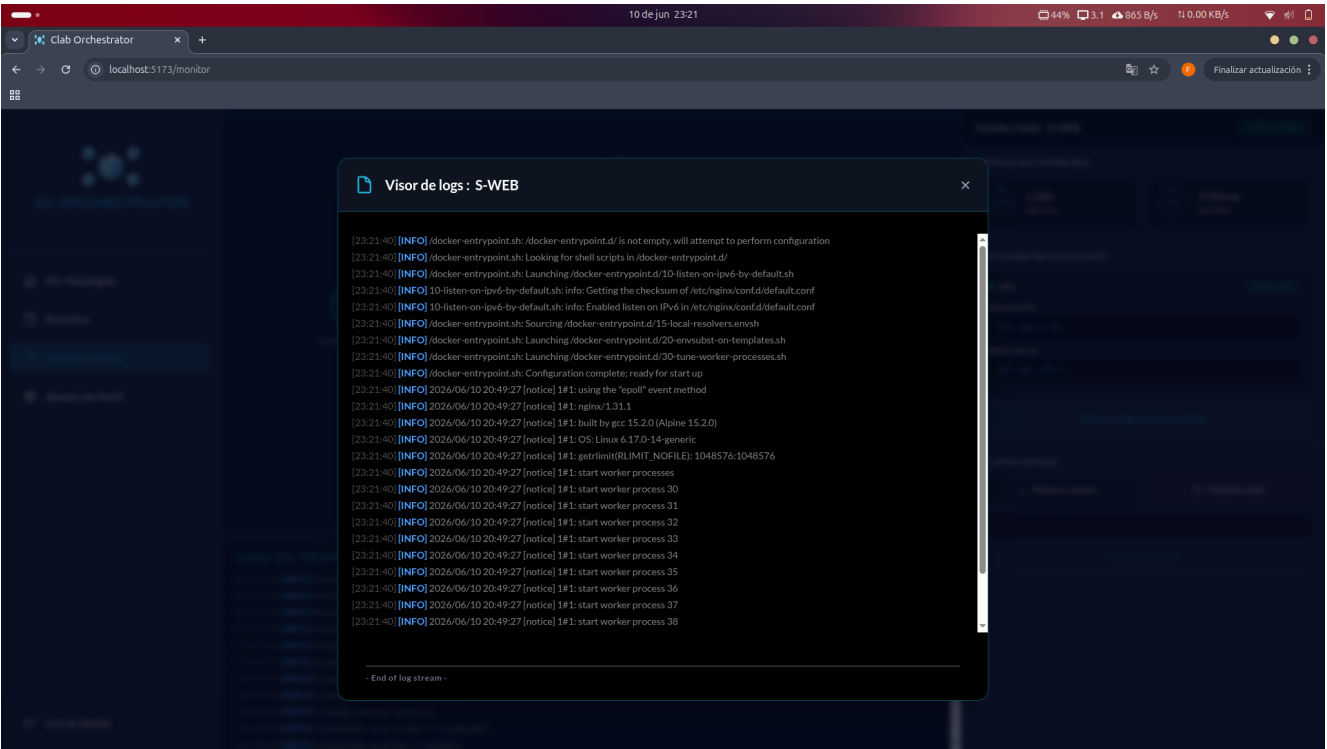


Figura B.10: Consola de un nodo con registros disponibles.

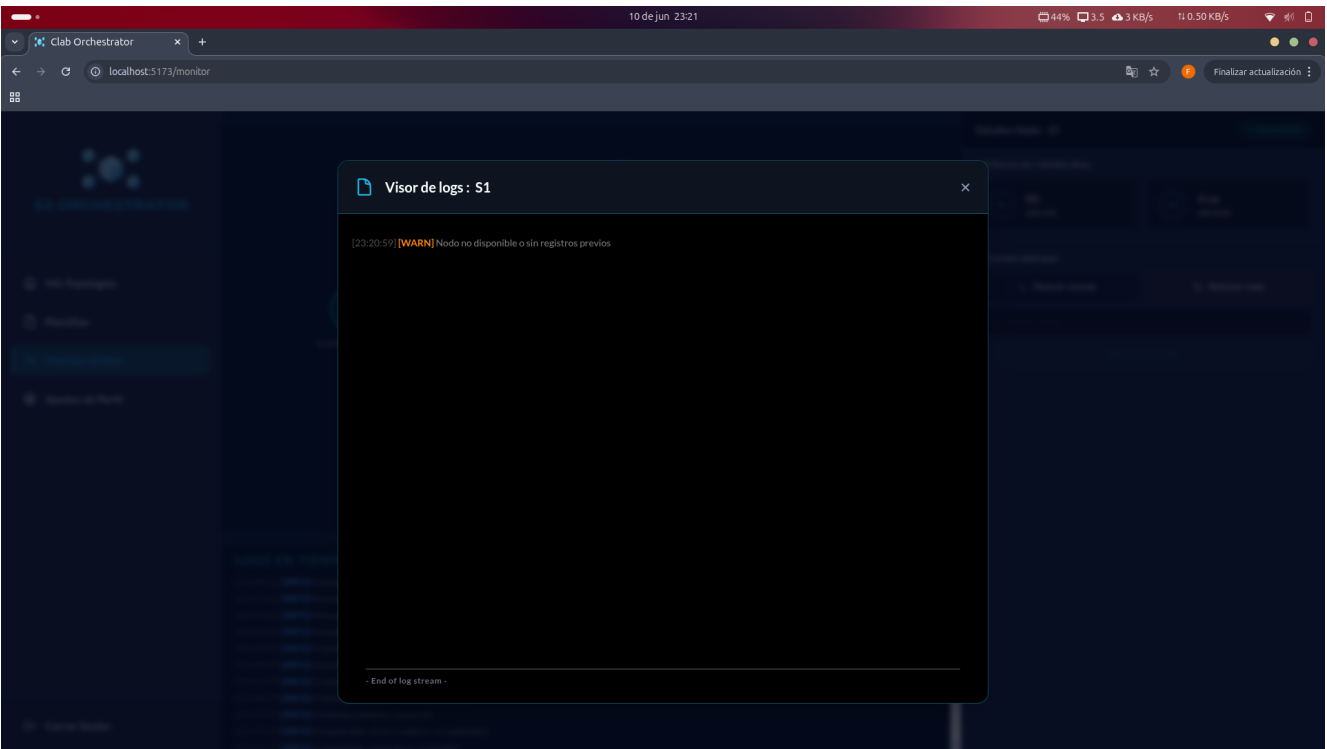


Figura B.11: Consola de un nodo sin registros disponibles.

Anexo C

Repositorio del código fuente

Para facilitar la revisión y consulta del trabajo realizado, el código fuente completo está disponible en el siguiente enlace:

`https://gitlab.inf.uva.es/fraalva/containerlab-orchestrator.git`